

TP de Biométrie — Statistiques Semestre 5

Benoît Simon-Bouhet

lundi 14 septembre 2026

Table des matières

Introduction	6
Objectifs	6
Pré-requis	7
Organisation	8
Volume de travail	8
Modalités d’enseignement	9
Comment communiquer	11
Progression conseillée	12
Séance 1 — Comparer une moyenne à une valeur théorique	13
Séance 2 — Comparer des données appariées	13
Séance 3 — Comparer deux échantillons indépendants	14
Séance 4 — Estimer et comparer des proportions	14
Séance 5 — Tester la qualité de l’ajustement	15
Quelques mots sur les IA génératives	15
Évaluation(s)	17
Licence	20
 1 Comparaison de la moyenne d’une population à une valeur théorique	 21
1.1 Pré-requis	21
1.2 Contexte	22
1.3 Importation et mise en forme des données . .	23
1.4 Exploration statistique des données	26
1.4.1 Position et dispersion	26
1.4.2 Incertitude	29
1.5 Exploration graphique des données	30
1.5.1 Histogramme	30
1.5.2 Diagramme de densité	32
1.6 Le test paramétrique	33
1.6.1 Conditions d’application	34
1.6.2 Signification de la p -value	36
1.6.3 Réalisation du test de Student et interprétation	40
1.7 L’alternative non paramétrique	42

1.8	Les notions d'erreur et de puissance statistique	44
1.9	Bilan	46
1.10	Exercice d'application	47
2	Comparaison de moyennes : deux échantillons appariés	48
2.1	Pré-requis	48
2.2	Contexte	49
2.3	Importation et mise en forme des données . .	51
2.4	Exploration statistique des données	55
2.5	Exploration graphique des données	57
2.5.1	Avec un stripchart	58
2.5.2	Avec des histogrammes facettés	59
2.5.3	Avec des diagrammes de densité facettés	59
2.5.4	Avec des boîtes à moustaches	60
2.5.5	Avec un nuage de points appariés . . .	62
2.6	Le test paramétrique	64
2.6.1	Procédure	64
2.6.2	Conditions d'application	65
2.6.3	Réalisation du test et interprétation .	68
2.7	L'alternative non paramétrique	70
2.8	Exercice d'application	72
3	Comparaison de moyennes : deux échantillons indépendants	73
3.1	Pré-requis	73
3.2	Contexte	74
3.3	Importation et mise en forme des données . .	76
3.4	Exploration statistique des données	78
3.5	Exploration graphique des données	81
3.5.1	Avec un stripchart	81
3.5.2	Avec des histogrammes facettés	82
3.5.3	Avec des diagrammes de densité facettés	83
3.5.4	Avec des boîtes à moustaches	84
3.6	Le test paramétrique	86
3.6.1	Conditions d'application	86
3.6.2	Réalisation du test et interprétation .	92
3.7	L'alternative non paramétrique	94
3.8	L'autre alternative non paramétrique	95
3.9	Exercices d'application	96
3.9.1	La taille des hommes et des femmes .	96
3.9.2	La longueur du bec des manchots Adélie	96
3.10	Tests bilatéraux et unilatéraux	97
3.10.1	Principe	97

3.10.2	Un exemple pas à pas	99
3.10.3	Exercice d'application	104
4	Comparaison de proportions	105
4.1	Pré-requis	105
4.2	Contexte	106
4.3	Importation et mise en forme des données . .	108
4.4	Cas 1 : comparer une proportion observée à une proportion théorique	110
4.4.1	Question statistique et données utiles	110
4.4.2	Exploration statistique des données . .	111
4.4.3	Exploration graphique des données . .	118
4.4.4	Test d'hypothèse : comparer une pro- portion à une valeur théorique	121
4.5	Cas 2 : comparer deux proportions indépen- dantes	124
4.5.1	Question statistique et données utiles	125
4.5.2	Exploration statistique des données . .	126
4.5.3	Exploration graphique des données . .	127
4.5.4	Test d'hypothèse : comparer deux pro- portions	131
4.5.5	Du test de deux proportions à la table de contingence	133
4.5.6	Conditions d'application du test du chi carré	135
4.5.7	Le cas des petits effectifs : test exact de Fisher	136
4.6	Comment choisir le bon test ?	138
4.7	Exercices d'application	139
5	Qualité de l'ajustement	140
5.1	Pré-requis	140
5.2	Contexte	141
5.3	Contexte scientifique et données	142
5.4	Le modèle proportionnel	147
5.5	Exploration statistique des données	149
5.6	Exploration graphique	153
5.7	Le test du chi-deux de qualité de l'ajustement	155
5.8	Degrés de liberté et p -value	158
5.9	Conditions d'application	159
5.10	Magnitude de l'effet	160
5.11	Le cas de deux catégories	161
5.12	Ajustement à une loi binomiale	164
5.13	Ajustement à une loi de Poisson	170

5.14	Comment choisir et interpréter un test de qualité de l'ajustement ?	177
5.15	Conclusion	178
5.16	Exercices d'application	179
Références		180

Introduction

Objectifs

Ce livre rassemble les contenus, exemples et exercices des travaux pratiques qui illustrent les cours magistraux (CM) et les travaux dirigés (TD) de **Biométrie**, dans l'EC *Outils pour l'étude et la compréhension du vivant 4* du semestre 5 de la licence Sciences de la Vie de La Rochelle Université. Ces travaux pratiques sont assurés par **Benoît Simon-Bouhet**. Les cinq chapitres de cet ouvrage sont consacrés aux tests statistiques de comparaison de moyennes et de proportions, aux tests de qualité de l'ajustement, ainsi qu'à la démarche d'analyse des données à mettre en place avant leur utilisation.

À l'issue du travail sur les cinq chapitres de ce livre, vous devriez être capables de faire les choses suivantes dans **RStudio** :

- Mettre en forme des tableaux de données : renommer des variables, en créer de nouvelles, modifier des variables existantes, filtrer des lignes, sélectionner des colonnes, etc. (vu au semestre 3)
- Explorer des jeux de données en produisant des graphiques appropriés selon la nature des variables disponibles, leur nombre et l'objectif recherché (vu au semestre 3)
- Explorer des jeux de données en produisant des résumés statistiques de variables de différentes natures (numériques continues ou catégorielles) et pour différentes modalités de facteurs d'intérêt (vu au semestre 4)
- Calculer des indices de position (moyennes, médianes...), de dispersion (écarts-types, variances, IQR...) et d'incertitude (erreurs standard, intervalles de confiance...) pour plusieurs sous-groupes de vos jeux de données, et les représenter sur des graphiques adaptés (vu au semestre 4)

- Choisir et formuler des hypothèses adaptées à la question scientifique posée (hypothèses bilatérales ou unilatérales)
- Choisir les tests statistiques permettant de répondre à une question scientifique précise selon la nature de la question posée et la nature des variables à disposition
- Réaliser les tests usuels de comparaison de moyennes (*t* de Student à 1 ou 2 échantillons, appariés ou indépendants...)
- Vérifier les conditions d'application des tests (graphiquement et avec des tests appropriés tels que les tests de Shapiro-Wilk et de Levene), et le cas échéant, choisir une alternative adaptée (test de Welch, de Wilcoxon...)
- Estimer une proportion et son intervalle de confiance
- Comparer une proportion à une valeur théorique et comparer les proportions de deux groupes indépendants
- Construire et interpréter une table de contingence, examiner les effectifs attendus et choisir un test adapté
- Comparer une distribution d'effectifs observés à une distribution attendue sous un modèle théorique, avec un test du χ^2 de qualité de l'ajustement
- Distinguer un test du χ^2 d'indépendance d'un test du χ^2 de qualité de l'ajustement
- Interpréter correctement les résultats des tests pour répondre aux questions scientifiques posées

Pré-requis

Pour atteindre les objectifs fixés ici, et compte tenu du volume horaire restreint qui est consacré aux TP de Biométrie au S5, je suppose que vous possédez un certain nombre de pré-requis. En particulier, vous devriez avoir à ce stade une bonne connaissance de l'interface du logiciel **RStudio**, et vous devriez être capables :

1. de créer un **Rproject** et un script d'analyse dans **RStudio**
2. d'importer des jeux de données issus de tableurs dans **RStudio**
3. d'effectuer des manipulations de données simples (sélectionner des variables, trier des colonnes, filtrer des lignes, créer de nouvelles variables, etc.)

4. de produire des graphiques de qualité, adaptés à la fois aux variables dont vous disposez et aux questions auxquelles vous souhaitez répondre
5. de réaliser une exploration statistique de vos données en calculant des indices de position, de dispersion et d'incertitude pour plusieurs sous-groupes de vos données

 Si ces pré-requis ne sont pas maîtrisés

Mettez-vous à niveau de toute urgence en lisant attentivement :

- [le livre en ligne de Biométrie du semestre 3](#)
- [le livre en ligne de Biométrie du semestre 4](#)

Vous aurez de toutes façons besoin de vous référer à ces 2 documents régulièrement au cours de ces TP. Donc gardez-les à portée de main et apprenez à naviguer rapidement dans leurs chapitres et sous-chapitres pour retrouver facilement les informations dont vous avez besoin.

Organisation

Volume de travail

Les travaux pratiques de biométrie auront lieu entre le 14 septembre et le 9 décembre 2026. Durant cette période, vous n'aurez jamais plus d'un créneau d'1h30 de TP par semaine pour chaque groupe, et deux séances consécutives sont souvent espacées de deux semaines ou plus. Vous devrez donc prendre l'habitude de prévoir, dans vos emplois du temps, des plages de travail personnel régulières afin de ne pas oublier ce que vous avez appris entre deux séances de TP.

Les séances de TP ont lieu soit en salle MSI 217, soit dans les salles du Pôle Communication Multimédia Réseau (PCMR), dans les salles Sc1 à Sc7 ou en salle Info Trans.

Au total, chaque groupe aura 9 séances de TP de 1h30 au semestre 5. **Cinq de ces séances, soit 7h30, seront as-**

surées par mes soins (Benoît Simon-Bouhet) et porteront sur les contenus de ce livre. Les quatre autres séances seront assurées par Florian Orgeret et pour lesquelles un autre livre en ligne sera disponible. Pour atteindre les objectifs présentés ici, vous devrez prévoir du travail personnel régulier entre les 5 séances de TP décrites ici. Vos prédécesseurs consacraient en moyenne entre une et trois heures de travail personnel pour chaque séance de TP prévue dans leur emploi du temps. Comme pour les semestres précédents, Ce travail personnel et régulier est essentiel pour progresser dans cette matière. J'insiste sur l'importance de faire l'effort dès maintenant : vous allez en effet avoir des enseignements qui reposent sur l'utilisation de ces logiciels jusqu'à la fin du S6 (y compris pendant vos stages et, très vraisemblablement, dans vos futurs masters également). C'est donc maintenant qu'il faut acquérir des automatismes, cela vous fera gagner énormément de temps ensuite.

Attention, mes séances et celles de Florian Orgeret sont intercalées au cours du semestre. Pour les TP de l'EC outils, deux séances consécutives dans votre emploi du temps ne correspondent donc pas nécessairement au même intervenant ni au même travail à préparer. Avant chaque TP, vérifiez le nom de l'intervenant, la salle et les consignes de préparation associées à la séance.

Important

Soyez vigilants lorsque vous consultez vos emplois du temps : les deux intervenants utilisent des supports et des modalités pédagogiques différents. Les consignes de ce livre concernent uniquement mes séances. Pour celles de Florian Orgeret, consultez les documents et les consignes qu'il vous fournit.

Modalités d'enseignement

Pour suivre cet enseignement, vous pourrez utiliser les ordinateurs de l'université, mais je ne peux que vous encourager à utiliser vos propres ordinateurs, sous Windows, Linux ou MacOS. Lors de vos futurs stages et pour rédiger vos comptes rendus de TP, vous utiliserez le plus souvent vos propres ordinateurs, autant prendre dès maintenant de bonnes habitudes

en installant les logiciels dont vous aurez besoin tout au long de votre licence. Si vous n'avez pas suivi les cours de biométrie de L2 et que les logiciels R et RStudio ne sont pas encore installés sur vos ordinateurs, suivez [la procédure décrite ici](#). Si vous ne possédez pas d'ordinateur, manifestez-vous rapidement auprès de moi car des solutions existent (prêt par l'université, travail sur tablette ou chromebook via [Posit cloud...](#)).

! Séances assurées par Benoît Simon-Bouhet

L'essentiel du contenu de ce livre peut être abordé en autonomie, à distance, grâce aux explications et exercices proposés ici, aux ressources mises à disposition sur Moodle et à votre ordinateur personnel. **Les cinq séances que j'assure fonctionnent comme des permanences d'accompagnement : votre présence en salle est facultative, mais le travail demandé est obligatoire.**

Plus que des séances de TP classiques, considérez plutôt qu'il s'agit de **permanences non-obligatoires** : si vous pensez avoir besoin d'aide, si vous avez des points de blocage ou des questions sur le contenu de ce document ou sur les exercices demandés, alors venez poser vos questions lors des séances de TP. Vous ne serez d'ailleurs pas tenus de rester pendant 1h30 : si vous obtenez une réponse en 10 minutes et que vous préférez travailler ailleurs, vous serez libres de repartir. De même, si vous n'avez pas de difficulté de compréhension, que vous n'avez pas de problème avec les exercices de ce livre en ligne ni avec les quiz Moodle, votre présence n'est pas requise. Si vous souhaitez malgré tout venir en salle de TP, pas de problème, vous y serez toujours les bienvenus.

Ce fonctionnement très souple a de nombreux avantages :

- vous vous organisez comme vous le souhaitez
- vous ne venez que lorsque vous en avez vraiment besoin
- celles et ceux qui se déplacent reçoivent une aide personnalisée
- vous travaillez sur vos ordinateurs
- les effectifs étant réduits, c'est aussi plus confortable pour moi !

Toutefois, pour que cette organisation fonctionne, cela demande de la rigueur de votre part, en particulier sur la régularité du travail que vous devez fournir. Si la présence en salle de TP n'est pas requise, **le travail demandé est bel et bien obligatoire** ! Si vous venez en salle de TP sans avoir travaillé en amont, vous ne tirerez pas grand-chose des séances puisque vous passerez votre temps à lire et suivre ce livre en ligne, choses que vous pouvez très bien faire chez vous. Votre temps, celui de vos collègues et le mien sera bien mieux utilisé si vous avez travaillé en amont des séances. Aussi, n'attendez pas plusieurs semaines pour commencer : prévoyez des plages régulières de travail personnel dès le début du semestre, y compris les semaines où aucune séance n'est programmée avec moi.

Je vous laisse donc une grande liberté d'organisation. À vous d'en tirer le maximum et de faire preuve du sérieux nécessaire. Le rythme auquel vous devriez avancer est présenté dans la partie suivante "Progression conseillée" ci-dessous.

! Séances assurées par Florian Orgeret : attention !

Le fonctionnement en permanence facultative concerne uniquement mes séances. La présence aux séances de Florian Orgeret est obligatoire, sauf indications contraires de sa part. Ces séances étant intercalées dans vos emplois du temps, vérifiez systématiquement quel intervenant assure votre prochain TP et préparez le travail correspondant.

Comment communiquer

Lors de votre travail sur les chapitres et exercices de ce livre, vous pouvez me contacter directement par email. Joignez, si nécessaire, votre code, les messages d'erreur obtenus et des captures d'écran permettant de comprendre votre difficulté.

Sachez toutefois qu'afin d'éviter de recevoir les mêmes questions de nombreuses fois, mes réponses pourront être adressées à l'ensemble de la promotion. Ainsi, tout le monde en profite, et tout le monde progresse en même temps.

Ce document est fait pour vous permettre d'avancer en autonomie et vous ne devriez normalement pas avoir beaucoup besoin de moi si votre lecture est attentive. L'expérience montre en effet que la plupart du temps, il suffit de lire correctement les paragraphes précédents et/ou suivants pour obtenir la réponse à ses questions. Lors des séances de TP au format "permanence", j'essaie de rester disponible par mail, mais je réponds évidemment en priorité à celles et ceux qui sont physiquement présents en salle. Cela signifie néanmoins que **même si votre groupe n'est pas en TP, vos questions ont des chances d'être lues et de recevoir des réponses lorsque j'assure une permanence pour un autre groupe**. Vous pouvez d'ailleurs vous-même envoyer vos questions directement à toute la promotion : vous aurez ainsi plus de chances que quelqu'un vous réponde rapidement.

Progression conseillée

Si vous avez suivi le document de prise en main de R et RStudio lors de l'EC de biométrie du semestre 3 de la licence SV de La Rochelle, vous savez que pour apprendre à utiliser ces logiciels, il faut faire les choses soi-même, ne pas avoir peur des messages d'erreurs (il faut d'ailleurs apprendre à les déchiffrer pour comprendre d'où viennent les problèmes), essayer maintes fois, se tromper beaucoup, recommencer, et surtout, ne pas se décourager. J'utilise ce logiciel presque quotidiennement depuis près de 20 ans et à chaque session de travail, je rencontre des messages d'erreur. Avec suffisamment d'habitude, on apprend à les déchiffrer et on corrige les problèmes en quelques secondes. Ce livre est conçu pour vous faciliter la tâche, mais ne vous y trompez pas, vous rencontrerez des difficultés, et c'est normal. C'est le prix à payer pour profiter de la puissance du meilleur logiciel permettant d'analyser des données, de produire des graphiques de qualité et de réaliser toutes les statistiques dont vous aurez besoin d'ici la fin de vos études et au-delà.

Pour que cet apprentissage soit le moins problématique possible, prenez les choses dans l'ordre : lisez successivement les chapitres 1 à 5 et faites les exercices d'application au fur et à mesure de votre lecture.

! Comment lire cette progression

La progression ci-dessous concerne uniquement les cinq séances que j’assure, à raison d’une séance par chapitre. La “troisième séance”, par exemple, désigne votre troisième séance avec moi, même si vous avez entre-temps suivi des séances avec Florian Orgeret. Vérifiez donc le nom de l’intervenant dans votre emploi du temps pour identifier la séance et le chapitre correspondants.

Séance 1 — Comparer une moyenne à une valeur théorique

La première séance est consacrée au chapitre Chapitre 1. À partir de l’exemple des températures corporelles, vous devrez mettre en œuvre toutes les étapes d’une analyse de données : importer et mettre en forme les données, les examiner à l’aide de statistiques descriptives (Section 1.4) et de graphiques adaptés (Section 1.5), vérifier les conditions d’application du test (Section 1.6.1), puis réaliser le test de Student à un échantillon ou son alternative non paramétrique de Wilcoxon.

L’objectif est de comprendre la démarche autant que de maîtriser les commandes de **RStudio**. Vous devrez être capables de formuler les hypothèses, d’interpréter la p -value et de rédiger une conclusion répondant à la question posée. Vous devrez également distinguer l’estimation et son incertitude du résultat d’un test, et comprendre les notions d’erreurs de type I et II et de puissance statistique (Section 1.8). L’exercice d’application (Section 1.10) vous permettra de reproduire cette démarche sur un nouveau jeu de données.

Séance 2 — Comparer des données appariées

La deuxième séance est consacrée au chapitre Chapitre 2. À partir de l’exemple sur la testostérone et l’immunocompétence, vous apprendrez à reconnaître des observations appariées, à organiser les données aux formats large et long et à représenter graphiquement le lien entre les deux mesures réalisées sur chaque individu.

Vous devrez comprendre pourquoi l'analyse porte sur les différences individuelles et pourquoi la condition de normalité doit être vérifiée sur ces différences (Section 2.6.2). Vous réaliserez le test de Student sur données appariées et étudierez son alternative non paramétrique. Vous devrez être capables d'interpréter les résultats en tenant compte du sens dans lequel les différences sont calculées, puis d'appliquer cette démarche à la comparaison des températures du corps et du cerveau des autruches.

Séance 3 — Comparer deux échantillons indépendants

La troisième séance est consacrée au chapitre Chapitre 3. À partir de l'exemple des lézards cornus, vous devrez comparer deux groupes indépendants, en examinant séparément les données de chaque groupe et en estimant les moyennes et leur incertitude.

Vous apprendrez à vérifier les conditions d'application des tests et à comprendre leur robustesse (Section 3.6.1), puis à choisir entre le test de Student, le test de Welch et le test de Wilcoxon sur la somme des rangs. Vous devrez interpréter le résultat du test et la différence entre les groupes à l'aide des estimations et des intervalles de confiance. Cette séance permettra également de distinguer tests bilatéraux et unilatéraux (Section 3.10), de justifier le choix d'une hypothèse unilatérale et de comprendre le rôle de l'ordre des groupes dans la syntaxe du test.

Séance 4 — Estimer et comparer des proportions

La quatrième séance est consacrée au chapitre Chapitre 4. À partir des données sur les calmars, vous devrez passer des observations individuelles aux effectifs et aux proportions, comprendre en quoi une proportion est une moyenne et estimer son incertitude à l'aide d'un intervalle de confiance.

Vous apprendrez à comparer une proportion à une valeur théorique avec le test binomial exact (Section 4.4), puis à comparer les proportions de deux groupes indépendants (Section 4.5). Vous devrez également construire et interpréter une

table de contingence, faire le lien entre comparaison de proportions et test du χ^2 d'indépendance, examiner les effectifs attendus et comprendre dans quelles situations utiliser le test exact de Fisher (Section 4.5.7). L'objectif est de savoir choisir le test adapté et de rédiger une conclusion fondée sur les proportions estimées, leur incertitude et les résultats du test.

Séance 5 — Tester la qualité de l'ajustement

La cinquième et dernière séance est consacrée au chapitre Chapitre 5. Vous étendrez la démarche de comparaison de proportions à une distribution entière d'effectifs. À partir de l'exemple des poissons récifaux et des phases lunaires, vous devrez définir un modèle proportionnel (Section 5.4), calculer les probabilités et les effectifs attendus sous l'hypothèse nulle, puis réaliser un test du χ^2 de qualité de l'ajustement (Section 5.7).

Vous apprendrez à examiner les conditions d'application, les contributions des différentes catégories au χ^2 et la magnitude des écarts observés. Vous étudierez également l'ajustement à une loi binomiale (Section 5.12) et à une loi de Poisson (Section 5.13), ainsi que la prise en compte des paramètres estimés dans le calcul des degrés de liberté (Important 1). Vous devrez être capables de distinguer un test de qualité de l'ajustement d'un test d'indépendance et de formuler une conclusion qui porte précisément sur le modèle théorique testé.

Quelques mots sur les IA génératives

J'attire votre attention sur l'utilisation des IA génératives. La plupart des modèles conversationnels (Large Language Models, LLMs) tels que ChatGPT, Gemini, Claude... sont très bons pour générer du code. Je vous mets néanmoins en garde : si vous ne comprenez pas ce qui est expliqué dans ces pages, vous ne pourrez pas juger de la pertinence du code que les IA pourraient vous fournir. Vous pouvez donc vous faire aider par l'IA pour corriger votre code par exemple, ou pour obtenir des suggestions, mais **il est de votre responsabilité de comprendre ce que vous faites et ce que l'IA**

fait pour vous ! Non seulement c'est ce qui vous fera gagner du temps à terme (acceptez de passer du temps maintenant, pour comprendre les notions et concepts, afin de gagner beaucoup de temps plus tard), mais surtout, c'est ce qui vous évitera de faire beaucoup d'erreurs ! Encore cette année, la plupart de nos étudiants de M1 ont utilisé l'IA pour calculer un indice statistique un peu exotique. Presque aucun ne s'est rendu compte que le code produit par l'IA ne faisait pas ce qui était demandé. Par conséquent, toute une partie de leur compte rendu de TP n'avait aucun sens et ils ont donc été lourdement pénalisés à cause de leur confiance aveugle en l'IA. Produire soi-même un code, et demander l'aide de l'IA pour corriger un bug ou demander des améliorations précises, c'est très différent de ne produire aucun code et de demander à l'IA de tout faire. Dans un cas, on fait travailler son cerveau, et l'IA joue le rôle d'un partenaire qui fournit des suggestions souvent pertinentes. Dans l'autre, le cerveau reste au repos, on n'apprend rien, et donc, on perd son temps...

Vous devez donc faire preuve d'esprit critique vis-à-vis de ces outils, qui peuvent être très utiles, mais qui demandent la plus grande vigilance et qui se trompent régulièrement. Or, pour faire preuve d'esprit critique, vous devez être en mesure de comprendre le code fourni par ces IA. Si ça n'est pas le cas, reprenez les explications de ce livre en ligne : tout ce dont vous avez besoin s'y trouve noir sur blanc.

Par ailleurs, **si vous utilisez l'IA pour générer du code, vous devez l'indiquer clairement**, et ça doit rester relativement rare. Quelle(s) partie(s) de vos scripts sont réellement de vous, et quelles parties ont été générées par une IA ? À votre niveau, je vous assure qu'il m'est très facile de savoir si du code a été produit par une IA ou par vous. Si vous n'indiquez pas que du code que vous intégrez à vos scripts n'est pas de vous, c'est de la fraude qui peut faire l'objet de sanctions importantes (passage devant le conseil de discipline de l'établissement par exemple). Donc encore une fois, vous pouvez utiliser ces outils, mais à la double condition que (i) ça soit fait à bon escient (assurez-vous que ce qui est produit par l'IA est pertinent, que ça a du sens, et que ça reste rare), et (ii), vous indiquiez clairement dans vos scripts quelles lignes ont été produites à l'aide d'une IA (et laquelle).

Enfin, je me permets une dernière mise en garde. De plus en plus d'études démontrent un effet négatif de l'utilisation

excessive des LLMs sur la qualité des apprentissages chez les étudiants. Par exemple, [cet article \(Stadler et al. 2024\)](#) montre que

Bien que les LLM puissent réduire la charge cognitive associée à la collecte d'informations au cours d'une tâche d'apprentissage, ils ne favorisent pas un engagement profond avec le contenu étudié, pourtant nécessaire à un apprentissage durable et de qualité.

De même, [Kosmyna et al. \(2025\)](#) ont étudié, par électroencéphalogramme, les performances cognitives de 3 groupes de participants lors d'exercices de productions écrites (rédaction de synthèses scientifiques). Un groupe avait le droit d'utiliser les IA, un groupe était limité à l'utilisation de Google, un autre groupe n'avait aucun accès à ces outils numériques. Globalement, les participants du groupe LLM sont de loin ceux qui obtiennent les moins bons résultats, qui comprennent et retiennent le moins bien ce qu'ils ont fait. Ceux qui n'avaient droit à aucun outil sont ceux qui ont le plus appris car ils ont dû s'investir pleinement dans l'exercice. En particulier, les auteurs soulignent ceci :

Les utilisateurs d'un LLM ont eu du mal à citer correctement leurs propres travaux. Si les LLM offrent une facilité immédiate, nos résultats mettent en évidence des coûts cognitifs potentiels. Sur une période de quatre mois, les utilisateurs d'un LLM ont systématiquement enregistré des résultats inférieurs aux niveaux neuronal, linguistique et comportemental.

Bref, ces outils sont puissants, ils sont parfois très utiles (et ils vont continuer de s'améliorer), mais ils doivent être utilisés intelligemment pour éviter des conséquences très contre-productives pour vos apprentissages. Là encore, c'est à vous de faire preuve de maturité pour en tirer le meilleur !

Évaluation(s)

L'évaluation de la partie "Biométrie" de l'EC "Outils pour l'étude et la compréhension du vivant" aura lieu sous la forme

d'un examen terminal sur table. Puisque vous n'aurez pas accès à votre ordinateur, je ne vous demanderai pas d'écrire des lignes de codes. Toutefois, cette évaluation portera sur l'ensemble des notions et concepts abordés dans ce livre et dans les CM et TD de biométrie depuis le début du semestre. Les pré-requis décrits plus haut sont censés être maîtrisés, et vous pourrez donc aussi être interrogés sur des notions abordées lors des semestres précédents.

Pour vous aider à comprendre ce qui est attendu, je vous fournis ci-dessous la grille critériée dont je me servirai pour évaluer le travail que vous devrez produire au semestre 6, dans le cadre de l'évaluation sommative conjointe entre biométrie et stratégie d'échantillonnage. Tous les items de cette grille ne seront évidemment pas évalués lors de l'épreuve terminale de la fin du semestre 5, mais certains le seront. Je ne peux donc que vous encourager à lire attentivement les critères d'évaluation ci-dessous et à tenter de vous les approprier. Les séances de TP qui viennent doivent vous permettre de vous entraîner à produire des scripts de qualité, et à y intégrer des analyses pertinentes et correctement interprétées.

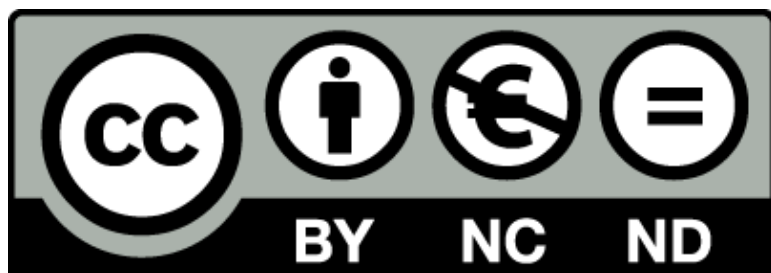
Compétence visée : A l'issue de la licence, face à une question scientifique, vous devriez être capable de :

1. Identifier la ou les méthodes statistiques appropriées pour répondre à cette question
2. Mettre en œuvre ces méthodes dans RStudio
3. Interpréter correctement les résultats obtenus

Résultat d'apprentissage (RA) L'étudiant-e est capable de :	Très insuffisant	Insuffisant	Satisfaisant	Très satisfaisant	n° RA	Évalué par
Produire un script fonctionnel, qui fait ce qu'il est censé faire	Lorsqu'on exécute le script on ne voit pas les messages d'erreurs apparus, tous les résultats attendus ne sont pas produits, ou certains résultats produits sont faux. La plupart des commandes utilisées pour manipuler ou explorer les données ne respectent pas les principes du tidyverse (par exemple, pas d'utilisation ou mauvaise utilisation des fonctions des packages dplyr, tidyr, ggplot2, etc.).	Lorsqu'on exécute le script on voit, quelques messages d'erreurs apparus, tous les résultats attendus ne sont pas produits, ou certains résultats produits sont faux. La plupart des commandes utilisées pour manipuler ou explorer les données respectent les principes du tidyverse (bonne utilisation des fonctions des packages dplyr, tidyr, ggplot2, etc.).	Lorsqu'on exécute le script on voit, aucun message d'erreur n'apparaît, mais certains résultats produits sont faux. La plupart des commandes utilisées pour manipuler ou explorer les données respectent les principes du tidyverse (bonne utilisation des fonctions des packages dplyr, tidyr, ggplot2, etc.).	Lorsqu'on exécute le script on voit, aucun message d'erreur n'apparaît, les résultats attendus sont correctement produits, les commandes utilisées pour manipuler ou explorer les données respectent les principes du tidyverse et les conventions des livres en ligne de biométrie.	1	BSB
Fournir un script d'analyse correctement commenté	(Presque) pas de commentaires ou commentaires inadéquats	Peu de commentaires sont présents ou s'ils sont présents, la plupart ne permettent pas de comprendre clairement ce qui a été fait par (ou l'intention de) l'auteur du script.	La plupart des commandes ou groupes de commandes sont commentés. Certains commentaires manquent parfois de clarté.	Chaque commande ou groupe de commande est bien commenté. Les commentaires sont parlants et permettent de comprendre sans ambiguïté l'intention et les choix de l'auteur du script.	2	BSB
Créer un script facile à lire et à décoder	Le script est difficile à déchiffrer en raison de plusieurs problèmes dans la liste suivante : noms d'objets peu parlants, espaces entre les éléments du code incohérents ou absents, indentations inexistantes, sauts de lignes manquants ou des mauvais endroits, code non structuré, ordre des commandes inadéquat.	Le script est globalement bien structuré, les commandes apparaissent dans un ordre logique, mais il pourrait être plus lisible car quelques problèmes subsistent dans la liste suivante : noms d'objets peu parlants, espaces entre les éléments du code incohérents ou absents, indentations inexistantes, sauts de lignes manquants ou des mauvais endroits.	Script bien structuré et lisible. La plupart des éléments de la liste suivante sont respectés : ordre des commandes adéquat, noms d'objets parlants, espaces, indentations et sauts de lignes cohérents, bon équilibre entre commentaires et commandes.	Script parfaitement structuré et lisible. Tous les éléments de la liste suivante sont respectés (espaces, ponctuation, indentation, sauts de lignes...), les noms d'objets sont courts et parlants, les commandes sont correctement ordonnées et un bon équilibre est respecté entre code et commentaires.	3	BSB
Créer un script générique	Non, si les données changent (ajout ou suppression de lignes dans le tableau de départ, changements de valeurs...), le script ne fonctionne plus (des messages d'erreur apparaissent ou les résultats produits sont faux). Les commandes permettant d'apporter les modifications ne sont pas, ou si elles le sont, elles ne permettent pas de générer un message d'erreur sur un autre ordinateur que celui de l'auteur du script.	Non, si les données changent (ajout ou suppression de lignes dans le tableau de départ, changements de valeurs...), le script ne fonctionne plus (des messages d'erreur apparaissent ou les résultats produits sont faux). Les commandes permettant d'apporter les modifications ne sont pas, ou si elles le sont, elles ne permettent pas de générer un message d'erreur sur un autre ordinateur que celui de l'auteur du script.	Oui, si les données changent (ajout ou suppression de lignes dans le tableau de départ, changements de valeurs...), le script fonctionne toujours : il ne renvoie pas de message d'erreur et les résultats fournis reflètent les modifications des données. Les commandes permettant d'apporter les données sont fournies et permettent à tout utilisateur d'apporter les données sans message d'erreur.	Oui, si les données changent (ajout ou suppression de lignes dans le tableau de départ, changements de valeurs...), le script fonctionne toujours : il ne renvoie pas de message d'erreur et les résultats fournis reflètent les modifications des données. Les commandes permettant d'apporter les données sont fournies et permettent à tout utilisateur d'apporter les données sans message d'erreur.	4	BSB
Mettre en forme les données pour l'analyse (gestion des données manquantes, gestion des données aberrantes, modification d'unités, calcul de nouvelles variables complètes, filtres, etc.)	Les données sont dans un format qui permet les analyses statistiques (format long et/ou tableau rangé), mais il contient encore un problème, par exemple : valeurs aberrantes non prises en compte, unités des variables incorrectes, types incorrects pour certaines variables (numériques, facteurs, etc.), pas de création de nouvelles variables pouvant indiquer la présence de données manquantes...	Les données sont dans un format qui permet les analyses statistiques (format long et/ou tableau rangé), mais il contient encore un problème, par exemple : valeurs aberrantes non prises en compte, unités des variables incorrectes, types incorrects pour certaines variables (numériques, facteurs, etc.), pas de création de nouvelles variables pouvant indiquer la présence de données manquantes...	Les données sont dans un format qui permet les analyses statistiques (format long et/ou tableau rangé), mais il contient encore un problème, par exemple : valeurs aberrantes non prises en compte, unités des variables incorrectes, types incorrects pour certaines variables (numériques, facteurs, etc.), pas de création de nouvelles variables pouvant indiquer la présence de données manquantes...	Les données sont dans un format qui permet les analyses statistiques (format long et/ou tableau rangé), et tous les points suivants sont correctement traités : valeurs aberrantes prises en compte, unités des variables correctes, types des variables corrects (numériques, facteurs, etc.), présence de toutes les nouvelles variables indispensables, bonne gestion des données manquantes...	5	BSB
D'explorer graphiquement les données avec des graphiques clairs et pertinents compte tenu de la nature des variables disponibles et de la question posée	Aucun graphique n'est produit (omission dans le rapport ou message d'erreur dans le script), ou les graphiques obtenus ne sont pas adaptés à la nature des variables disponibles et de la question posée.	Des graphiques sont produits (les commandes permettant de les produire fonctionnent), mais ils ne sont pas adaptés : soit leur mise en forme n'est pas à la hauteur (mauvais choix de couleurs, de symboles, axes non ou incorrectement légendés, catégories impossibles à distinguer...), soit la nature des graphiques produits n'est pas idéale (diagrammes bâtons, diagrammes, nuages de points, boîtes à moustaches, etc.).	Les graphiques produits sont adaptés à la nature des variables disponibles et aux questions posées et leur mise en forme est globalement bonne (p. ex., catégories faciles à distinguer, axes correctement légendés, boîtes à moustaches, symboles adaptés, etc.).	Les graphiques produits sont adaptés à la nature des variables disponibles et aux questions posées et leur mise en forme est globalement bonne (p. ex., catégories faciles à distinguer, axes correctement légendés, boîtes à moustaches, symboles adaptés, etc.).	6	BSB et CL
D'explorer les données à l'aide d'indices de statistiques descriptives pertinentes compte tenu de la nature des variables disponibles et de la question posée	Aucun indice de statistiques descriptives n'est fourni, ou si des indices sont fournis, ils ne sont pas pertinents pour la question posée.	Des indices de statistiques descriptives sont fournis, mais ils ne sont pas pertinents pour la question posée.	Des indices de statistiques descriptives sont fournis, mais ils ne sont pas pertinents pour la question posée.	Tous les indices pertinents de statistiques descriptives de position, dispersion et forme sont fournis, et ils sont correctement présentés (dans des tables correctement formatées ou sur des graphiques adaptés), décrits et commentés.	7	BSB et CL
De formuler clairement les hypothèses statistiques pertinentes dans le contexte de l'étude	Aucune hypothèse n'est posée, ou des hypothèses sont posées, mais ne sont pas en accord avec la problématique de l'étude ou les bases fondamentales des statistiques (par exemple, hypothèses nulles et alternatives inversées, ou hypothèses d'un test utilisées pour un autre test adapté).	Des hypothèses sont posées, mais ne sont pas en accord avec la problématique de l'étude ou les bases fondamentales des statistiques (par exemple, hypothèses nulles et alternatives inversées, ou hypothèses d'un test utilisées pour un autre test adapté).	Des hypothèses sont posées, mais ne sont pas en accord avec la problématique de l'étude ou les bases fondamentales des statistiques (par exemple, hypothèses nulles et alternatives inversées, ou hypothèses d'un test utilisées pour un autre test adapté).	Les hypothèses sont clairement exposées, suivent les règles statistiques, mais ne sont pas pertinentes pour la question posée.	8	BSB et CL
De justifier le choix des méthodes statistiques utilisées pour répondre à la question posée (selon la nature des données, le respect ou non des conditions d'application, etc.)	Aucune justification n'est fournie, ou les justifications avancées ne sont pas fondées.	Des justifications sont fournies, mais ne sont pas fondées.	Des justifications sont fournies, mais ne sont pas fondées.	Chaque test employé est clairement justifié et est adapté à la nature de la donnée. La démarche statistique employée est pertinente, ainsi que la justification de la méthode choisie.	9	BSB et CL
De vérifier les conditions d'application des tests statistiques envisagés	Les conditions d'application ne sont pas prises en compte. Le principe fondamental des tests statistiques n'est pas compris. Si les conditions d'application sont données, elles ne sont pas vérifiées, ou les tests employés pour les vérifier ne sont pas pertinents.	Les conditions d'application ne sont pas prises en compte. Le principe fondamental des tests statistiques n'est pas compris. Si les conditions d'application sont données, elles ne sont pas vérifiées, ou les tests employés pour les vérifier ne sont pas pertinents.	Les conditions d'application ne sont pas prises en compte. Le principe fondamental des tests statistiques n'est pas compris. Si les conditions d'application sont données, elles ne sont pas vérifiées, ou les tests employés pour les vérifier ne sont pas pertinents.	Les conditions d'application sont données et correctes, les tests employés ou les graphiques réalisés pour les vérifier sont adaptés et correctement réalisés, mais ils sont parfois mal interprétés.	10	BSB et CL
De réaliser les tests statistiques choisis dans un ordre logique et cohérent	Les tests statistiques ne sont pas réalisés dans un ordre correct du point de vue statistique (par exemple, test de normalité des données réalisé avant un test de Student) si du point de vue logique (p. ex., réalisation de tests post-hoc après une ANOVA même si l'hypothèse nulle de l'ANOVA n'a pas été rejetée, ou réalisation de tests non-paramétriques alors que les conditions d'application de leurs équivalents paramétriques sont vérifiées).	Les tests statistiques ne sont pas réalisés dans un ordre correct du point de vue statistique (par exemple, test de normalité des données réalisé avant un test de Student) si du point de vue logique (p. ex., réalisation de tests post-hoc après une ANOVA même si l'hypothèse nulle de l'ANOVA n'a pas été rejetée, ou réalisation de tests non-paramétriques alors que les conditions d'application de leurs équivalents paramétriques sont vérifiées).	Les tests statistiques ne sont pas réalisés dans un ordre correct du point de vue statistique (par exemple, test de normalité des données réalisé avant un test de Student) si du point de vue logique (p. ex., réalisation de tests post-hoc après une ANOVA même si l'hypothèse nulle de l'ANOVA n'a pas été rejetée, ou réalisation de tests non-paramétriques alors que les conditions d'application de leurs équivalents paramétriques sont vérifiées).	Les tests statistiques ne sont pas réalisés dans un ordre correct du point de vue statistique (par exemple, test de normalité des données réalisé avant un test de Student) si du point de vue logique (p. ex., réalisation de tests post-hoc après une ANOVA même si l'hypothèse nulle de l'ANOVA n'a pas été rejetée, ou réalisation de tests non-paramétriques alors que les conditions d'application de leurs équivalents paramétriques sont vérifiées).	11	BSB et CL
D'interpréter correctement les résultats obtenus pour répondre à la question scientifique posée	Les p-valeurs ou les valeurs critiques/seuls/théoriques sont incorrectes et les règles de décision ne sont pas correctement appliquées, engendrant systématiquement des conclusions erronées.	Les p-valeurs ou les valeurs critiques/seuls/théoriques sont incorrectes et les règles de décision ne sont pas correctement appliquées, engendrant systématiquement des conclusions erronées.	Les p-valeurs ou les valeurs critiques/seuls/théoriques sont incorrectes et les règles de décision ne sont pas correctement appliquées, engendrant systématiquement des conclusions erronées.	Les p-valeurs ou les valeurs critiques/seuls/théoriques sont incorrectes et les règles de décision ne sont pas correctement appliquées, engendrant systématiquement des conclusions erronées.	12	BSB et CL

Licence

Ce livre en ligne est sous licence Creative Commons ([CC BY-NC-ND 4.0](#))



Vous êtes autorisé à partager, copier, distribuer et communiquer ce matériel par tous moyens et sous tous formats, tant que les conditions suivantes sont respectées :

① Attribution : vous devez créditer ce travail (donc citer son auteur), fournir un lien vers ce livre en ligne, intégrer un lien vers la licence Creative Commons et indiquer si des modifications du contenu original ont été effectuées. Vous devez indiquer ces informations par tous les moyens raisonnables, sans toutefois suggérer que l’auteur vous soutient ou soutient la façon dont vous avez utilisé son travail.

Ⓢ Pas d’Utilisation Commerciale : vous n’êtes pas autorisé à faire un usage commercial de cet ouvrage, ni de tout ou partie du matériel le composant. Cela comprend évidemment la diffusion sur des plateformes de partage telles que studocu.com qui tirent profit d’œuvres dont elles ne sont pas propriétaires, souvent à l’insu des auteurs.

⊖ Pas de modifications : dans le cas où vous effectuez un remix, que vous transformez, ou créez à partir du matériel composant l’ouvrage original, vous n’êtes pas autorisé à distribuer ou mettre à disposition l’ouvrage modifié.

🔒 Pas de restrictions complémentaires : vous n’êtes pas autorisé à appliquer des conditions légales ou des mesures techniques qui restreindraient légalement autrui à utiliser cet ouvrage dans les conditions décrites par la licence.

1 Comparaison de la moyenne d'une population à une valeur théorique

1.1 Pré-requis

Pour travailler dans de bonnes conditions, créez un nouveau dossier sur votre ordinateur, créez un **Rproject** et un script dans ce dossier, et travaillez systématiquement **dans votre script**, et surtout pas directement dans la console. Consultez [le livre en ligne du semestre 3](#) si vous ne savez plus comment faire.

Dans ce chapitre, vous aurez besoin d'utiliser des packages spécifiques et d'importer des données depuis des fichiers externes téléchargeables directement depuis ce document. Les packages dont vous aurez besoin ici et que vous devez donc charger en mémoire, sont :

- le **tidyverse** (Wickham 2023), qui comprend notamment le package **readr** (Wickham, Hester, et Bryan 2026), pour importer facilement des fichiers **.csv** au format **tibble**, le package **dplyr** (Wickham, François, et al. 2026), pour manipuler des tableaux, et le package **ggplot2** (Wickham, Chang, et al. 2026) pour les représentations graphiques.
- **skimr** (Waring et al. 2026), qui permet de calculer des résumés de données très informatifs.

```
library(tidyverse)
library(skimr)
```

⚠ Important

Même si vous avez déjà installé le `tidyverse` ou `dplyr` au semestre précédent, ré-installez `dplyr` avec `install.packages("dplyr")`. Ce package a en effet été mis à jour récemment, et nous aurons besoin d'une version récente ($> v1.1.0$). Chargez-le ensuite en mémoire avec `library(dplyr)`.

! Attention

Pensez à installer tous les packages listés ci-dessous avant de les charger en mémoire si vous ne l'avez pas déjà fait. Si vous ne savez plus comment faire, consultez d'urgence [la section dédiée aux packages dans le livre en ligne de Biométrie du semestre 3](#).

Vous aurez également besoin des jeux de données suivants :

- [Temperature.csv](#)
- [Temperature2.csv](#)

Enfin, je spécifie ici une fois pour toutes le thème que j'utiliserai pour tous les graphiques de ce chapitre. Libre à vous de choisir un thème différent ou de vous contenter du thème proposé par défaut :

```
theme_set(theme_bw())
```

1.2 Contexte

On s'intéresse ici à la température corporelle des adultes en bonne santé. On souhaite examiner la croyance populaire qui veut que cette température vaut en moyenne 37°C . Pour le vérifier, on dispose d'un échantillon de 25 adultes en bonne santé choisis au hasard parmi la population américaine et dont on a mesuré la température. Comme pour toute étude statistique, les étapes que nous allons devoir suivre sont les suivantes (dans l'ordre) :

1. Importer les données dans `RStudio`, les examiner et éventuellement les (re)mettre en forme si besoin.

2. Faire une première exploration des données, grâce au calcul d'indices de statistiques descriptives d'une part, et de représentations graphiques d'autre part.
3. Réaliser un test d'hypothèses en respectant la procédure adéquate (en particulier, la vérification des conditions d'application).

C'est donc ce que nous allons faire dans les sections suivantes.

! À retenir !

Avant de se lancer dans les tests d'hypothèses, il est **toujours indispensable** d'examiner les données dont on dispose à l'aide, d'une part de statistiques descriptives numériques, et d'autre part, de graphiques exploratoires.

Nous avons vu au cours des semestres précédents quels indices statistiques il peut être utile de calculer ([dans le livre en ligne du semestre 4](#)) et quelles représentations graphiques il peut être utile de réaliser ([dans le livre en ligne du semestre 3](#)) afin de pouvoir se lancer dans des tests d'hypothèses sans risquer de grossières erreurs. N'hésitez pas à cliquer sur ces liens pour vous rafraîchir la mémoire !

1.3 Importation et mise en forme des données

Nous allons travailler ici sur les données contenues dans le fichier [Temperature.csv](#). Téléchargez ces données dans votre répertoire de travail (attention : ne les ouvrez pas avec Excel !), puis importez les données dans **RStudio** grâce à l'assistant d'importation. Si vous ne savez plus comment faire, consultez [la section dédiée à l'importation des données dans le livre en ligne de Biométrie du semestre 3](#)

Vous stockerez les données dans un objet que vous nommerez **Temperature**. Après l'importation, tapez son nom dans la console de **RStudio** et vérifiez que vous obtenez bien exactement ce résultat :

Temperature

```
# A tibble: 25 x 2
  individual temperature
      <dbl>         <dbl>
1         1         98.4
2         2         98.6
3         3         97.8
4         4         98.8
5         5         97.9
6         6          99
7         7         98.2
8         8         98.8
9         9         98.8
10        10          99
# i 15 more rows
```

La première chose à faire quand on travaille avec des données inconnues, c'est d'examiner les données brutes. Ici, les données sont importées au format `tibble`, donc seules les premières lignes sont visibles. Pour visualiser l'ensemble du tableau, utilisez la fonction `View()` (avec un `V` majuscule) ou, si vous avez mis en mémoire le `tidyverse`, la fonction `view()` (sans majuscule) :

`View(Temperature)`

Cette commande ouvre un nouvel onglet présentant les données dans un tableur simplifié, en lecture seule. On constate ici 2 choses que nous allons modifier :

1. la première colonne, intitulée `individual`, n'est pas véritablement une variable. Cette colonne ne contient qu'un identifiant sans intérêt pour notre étude et est en fait identique au numéro de ligne. Nous allons donc supprimer cette colonne.
2. les températures sont exprimées en degrés Fahrenheit, ce qui rend leur lecture difficile pour nous qui sommes habitués à utiliser le système métrique et les degrés Celsius. Nous allons donc convertir les températures en degrés Celsius grâce à la formule suivante :

$$C = \frac{F - 32}{1.8}$$

```
Temp_clean <- Temperature |>
  select(-individual) |>      # Suppression de la colonne `individual`
  mutate(                     # Transformation des températures en °Celsius
    temperature = (temperature - 32) / 1.8
  )
```

```
Temp_clean
```

```
# A tibble: 25 x 1
```

```
  temperature
    <dbl>
1      36.9
2      37
3      36.6
4      37.1
5      36.6
6      37.2
7      36.8
8      37.1
9      37.1
10     37.2
```

```
# i 15 more rows
```

Il nous est maintenant possible d'examiner à nouveau les données avec la fonction `View()`. Avec des valeurs de températures comprises entre 36.3°C et 37.8°C, il n'y a visiblement pas de données aberrantes.

Examiner les données brutes est donc la première chose que vous devriez prendre l'habitude de faire, et ce de façon systématique, car cela permet de repérer :

- La nature des variables présentes.
- Les variables inutiles qui pourront être supprimées ou négligées.
- Les unités des variables utiles, afin de pouvoir les convertir si nécessaire.
- Les valeurs manquantes, atypiques ou aberrantes qui demanderont toujours une attention particulière.

Maintenant que l'examen préliminaire des données est réalisé, on peut passer au calcul des statistiques descriptives.

1.4 Exploration statistique des données

1.4.1 Position et dispersion

On s'intéresse ici au calcul de grandeurs statistiques nous apportant des renseignements sur la **position** et la **dispersion** des valeurs de l'échantillon. Les questions auxquelles on tente de répondre à ce stade sont les suivantes :

- Quelle est la tendance centrale (moyenne ou médiane) ?
- Quelle est la dispersion des valeurs autour de la tendance centrale (écart-type, variance, intervalle interquartile...) ?

Pour répondre à ces questions, on peut faire appel à de multiples fonctions déjà présentées [dans le livre en ligne du semestre 4](#). Par exemple la fonction `summarise()`, en conjonction avec les fonctions `mean()`, `median()`, `sd()`, `var()`, `min()`, `max()` ou `quantile()`, ou les fonctions `summary()` ou `skim()` (du package `skimr`).

Je prends ici un exemple simple, mais n'hésitez pas à expérimenter avec les méthodes décrites dans le livre en ligne du semestre 4.

```
summary(Temp_clean)
```

```
temperature
Min.      :36.33
1st Qu.:36.67
Median :37.00
Mean     :36.96
3rd Qu.:37.22
Max.     :37.78
```

On constate ici que la moyenne et la médiane sont très proches. La distribution des températures doit donc être à peu près symétrique, avec à peu près autant de valeurs

au-dessus que de valeurs en dessous de la moyenne. Les premier et troisième quartiles sont à peu près aussi éloignés de la médiane l'un que l'autre, ce qui confirme l'apparente symétrie du jeu de données de part et d'autre de la tendance centrale.

La moyenne observée dans l'échantillon vaut 36.96°C, ce qui est très proche de la moyenne théorique de 37°C.

Une autre fonction utile est la fonction `IQR()`, qui renvoie l'étendue de l'intervalle interquartile (la valeur du troisième quartile moins la valeur du premier quartile) :

```
Temp_clean |>
  summarise(IQ_range = IQR(temperature))

# A tibble: 1 x 1
  IQ_range
    <dbl>
1    0.556
```

On constate ici que l'intervalle interquartile a une largeur de 0.56°C. Cela signifie que les 50% des températures les plus centrales de l'échantillon sont situées dans un intervalle d'environ un demi-degré Celsius autour de la médiane.

Enfin, pour obtenir des informations complémentaires, on peut utiliser la fonction `skim()` du package `skimr` :

```
skim(Temp_clean)
```

```
-- Data Summary -----
Name                               Values
Number of rows                    25
Number of columns                  1
-----
Column type frequency:
  numeric                          1
-----
Group variables                    None
-- Variable type: numeric -----
```

	skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
1	temperature	0		1 37.0	0.377	36.3	36.7	37	37.2	37.8	

Tout comme `summary()`, la fonction `skim()` renvoie les valeurs minimales et maximales, les premiers et troisièmes quartiles ainsi que la moyenne et la médiane. Elle nous indique en outre la valeur de l'écart-type de l'échantillon, ainsi que le nombre d'observations et le nombre de données manquantes. Enfin, elle fournit un histogramme très simplifié et sans échelle. Cet histogramme nous permet de nous faire une première idée de la distribution des données et est particulièrement utile pour comparer rapidement un grand nombre de distributions quand il y a plusieurs catégories dans les données (ce qui n'est pas le cas ici).

Outre ces 3 fonctions (`summary()`, `IQR()`, et `skim()`), il est bien sûr possible de calculer toutes ces valeurs manuellement si besoin :

- `mean()` permet de calculer la moyenne.
- `median()` permet de calculer la médiane.
- `min()` et `max()` permettent de calculer les valeurs minimales et maximales respectivement.
- `quantile()` permet de calculer les quartiles.
- `sd()` permet de calculer l'écart-type.
- `var()` permet de calculer la variance.
- `n()` permet de compter le nombre d'observations.

Toutes ces fonctions prennent seulement un vecteur en guise d'argument. Il faut donc procéder comme avec `IQR()` pour les utiliser, en les intégrant à l'intérieur de la fonction `summarise()`. Par exemple, pour calculer la variance, on peut taper :

```
Temp_clean |>
  summarise(variance = var(temperature))

# A tibble: 1 x 1
  variance
  <dbl>
1    0.142
```

ou :

```
Temp_clean |>
  pull(temperature) |>
  var()
```

```
[1] 0.1417901
```

ou encore :

```
var(Temp_clean$temperature)
```

```
[1] 0.1417901
```

À vous d'utiliser la syntaxe qui vous semble la plus simple.

1.4.2 Incertitude

Outre les informations de position et de dispersion, nous avons vu [au semestre 4](#) qu'il était également important d'avoir une idée de l'**incertitude** associée aux estimations de tendance centrale (erreur standard ou intervalle de confiance de la moyenne ou médiane). Ici, nous allons donc calculer l'intervalle de confiance à 95% de la moyenne. Si vous ne savez plus comment faire, ou que vous ne comprenez pas le code ci-dessous, consultez [le livre en ligne du semestre 4](#) :

```
Temp_clean |>
  reframe(mean_cl_normal(temperature))
```

```
# A tibble: 1 x 3
      y  ymin ymax
<dbl> <dbl> <dbl>
1  37.0  36.8  37.1
```

On constate ici que les bornes inférieure (36.8°C) et supérieure (37.1°C) de l'intervalle de confiance à 95% de la moyenne sont proches de la valeur de moyenne de l'échantillon. Dans la population générale, la moyenne de la température corporelle chez les adultes en bonne santé a

de bonnes chances de se trouver quelque part entre 36.8°C et 37.1°C. Autrement dit, si la température corporelle des adultes en bonne santé n'est pas exactement de 37°C, l'écart à cette valeur théorique ne doit pas être très important.

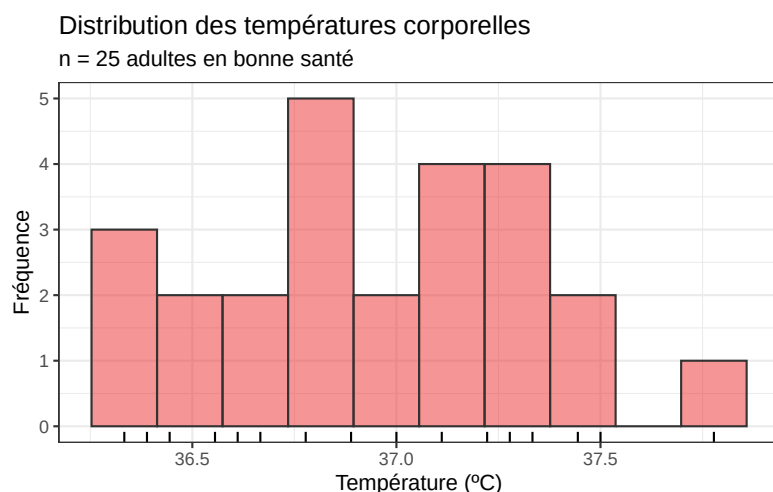
1.5 Exploration graphique des données

Ici, puisque nous ne disposons que d'une unique variable numérique et que nous n'avons donc qu'un unique groupe, les représentations graphiques que nous allons réaliser doivent nous permettre d'examiner la **distribution des données**. Pour cela, nous pouvons réaliser soit un histogramme, soit un diagramme de densité.

1.5.1 Histogramme

Voilà comment produire un histogramme de qualité pour les données de températures :

```
Temp_clean |>
  ggplot(aes(x = temperature)) +
  geom_histogram(bins = 10, fill = "firebrick2", color = "grey20",
                 alpha = 0.5) +
  geom_rug() +
  labs(x = "Température (°C)",
       y = "Fréquence",
       title = "Distribution des températures corporelles",
       subtitle = "n = 25 adultes en bonne santé")
```



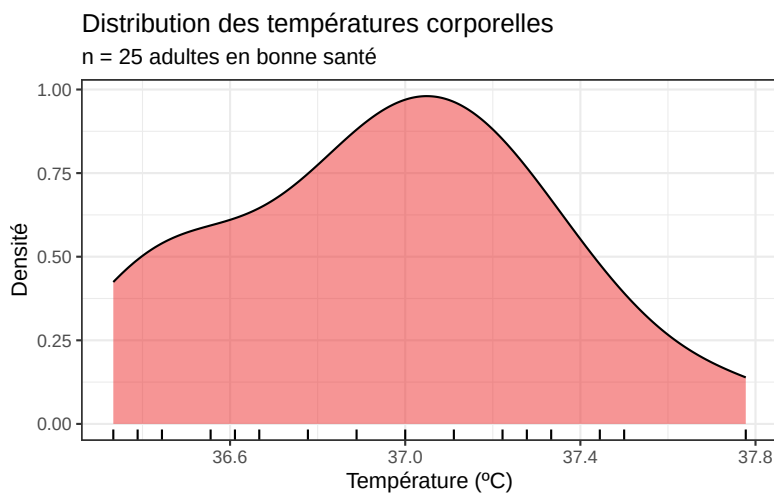
Si vous ne vous rappelez plus ce qu’est un histogramme ou comment le faire, ou la signification de l’argument `bins`, relisez [la section consacrée aux histogrammes](#) du livre en ligne de Biométrie du semestre 3. Notez que j’ai ajouté une couleur de remplissage et de la transparence pour rendre le graphique plus facile à lire. J’ai également spécifié des titres pour les axes (en précisant l’unité de la variable numérique dont on représente la distribution) ainsi que le titre (et sous-titre) du graphique, qui précise ce qu’on a sous les yeux et la taille de l’échantillon. Il n’est pas toujours nécessaire de spécifier le titre (et le sous-titre) de cette façon : lorsque vous intégrez des graphiques dans un compte rendu ou un rapport, le titre est en général précisé sous la figure, au début d’une légende qui la décrit. Enfin, j’ai ajouté `geom_rug()` pour faire apparaître sous le graphique, le long de l’axe des x , la position des données observées. Cela permet de visualiser les données brutes, et peut donc permettre de mieux comprendre pourquoi un histogramme présente telle ou telle forme.

Ici, la forme de cet histogramme est assez proche de celle présentée plus tôt par l’histogramme très simplifié produit par la fonction `skim()`. Cet histogramme nous apprend qu’en dehors d’un “trou” autour de la température 36.75°C, la distribution des données est proche d’une courbe en cloche. Il y a fort à parier qu’un test de normalité conclurait à la normalité des données de cet échantillon. C’est ce que nous verrons dans la Section [1.6.1](#).

1.5.2 Diagramme de densité

Une autre façon de visualiser la distribution d'une variable numérique est de produire un graphique de densité. Il a l'avantage d'éviter à l'utilisateur d'avoir à choisir une valeur pour l'argument `bin` de la fonction `geom_histogram()`, mais il a l'inconvénient de présenter une échelle plus difficile à comprendre pour l'axe des ordonnées :

```
Temp_clean |>
  ggplot(aes(x = temperature)) +
  geom_density(fill = "firebrick2", alpha = 0.5) +
  geom_rug() +
  labs(x = "Température (°C)",
       y = "Densité",
       title = "Distribution des températures corporelles",
       subtitle = "n = 25 adultes en bonne santé")
```



Les informations apportées par ce graphique sont cohérentes avec celles de l'histogramme :

- les températures les plus fréquemment observées dans notre échantillon de 25 adultes en bonne santé se situent légèrement au-dessus de 37°C. Il s'agit d'une information concernant la **position** des données (c'est-à-dire où se trouve le pic de la distribution sur l'axe des x)
- les températures observées ont une distribution qui ressemble à peu près à une courbe en cloche, avec des valeurs comprises entre 36.4°C et 37.8°C environ. La

symétrie de part et d'autre du pic n'est pas parfaite, mais elle reste bonne. Il s'agit d'informations concernant la forme de la distribution et la **dispersion** des données.

i Bilan des analyses préliminaires

Suite à l'exploration statistique et graphique des données de températures, voilà ce qu'on retient :

1. Il n'y a visiblement pas de données aberrantes.
2. La distribution des données semble suivre à peu près la loi Normale.
3. La médiane et la moyenne sont très proches de 37°C. Un test devrait donc arriver à la conclusion que la température corporelle des adultes n'est pas significativement différente de 37°C.
4. La largeur de l'intervalle de confiance à 95% semble faible, ce qui indique une incertitude relativement faible. Si la température réelle des adultes en bonne santé n'est pas exactement de 37°C, elle ne devrait pas en être très éloignée (quelques dixièmes de degrés Celsius au plus).

1.6 Le test paramétrique

Le test permettant de comparer la moyenne μ d'une population à une valeur théorique, fixée par l'utilisateur, est le **test de Student à un échantillon**. Il permet de répondre à la question suivante :

Les données observées dans l'échantillon dont je dispose sont-elles compatibles avec l'hypothèse que la moyenne μ de la population dont est issu mon échantillon vaut **XXX** ?

avec **XXX**, une valeur d'intérêt spécifiée par l'utilisateur. Il s'agit d'un test paramétrique très puissant. Comme tous les tests paramétriques, certaines conditions d'application doivent être vérifiées avant de pouvoir l'appliquer.

! Important

Comme pour tous les tests statistiques que nous allons réaliser lors de ces séances de TP et TEA, nous devons commencer par **spécifier les hypothèses** nulles et alternatives de chaque test, ainsi que la **valeur du seuil** α que nous allons utiliser. À moins d'avoir une bonne raison de faire autrement, on utilise presque toujours le seuil $\alpha = 0.05$ dans le domaine des sciences du vivant. C'est donc ce seuil que nous utiliserons dans ce livre en ligne.

1.6.1 Conditions d'application

Les conditions d'application du test de Student à un échantillon sont les suivantes :

1. Les données de l'échantillon sont issues d'un **échantillonnage aléatoire** au sein de la population générale. Cette condition est partagée par toutes les méthodes que nous verrons dans ces TP. En l'absence d'informations sur la façon dont l'échantillonnage a été réalisé, on considère que cette condition est remplie. Il n'y a pas de moyen statistique de le vérifier, cela fait uniquement référence à la stratégie d'échantillonnage déployée et à la rigueur de la procédure mise en œuvre lors de l'acquisition des données.
2. La variable étudiée doit suivre une **distribution Normale** dans la population générale. Nous allons vérifier cette condition d'application avec un **test de normalité de Shapiro-Wilk**.

Pour un test de normalité, les hypothèses seront toujours les suivantes :

- H_0 : la variable étudiée suit une distribution Normale dans la population générale.
- H_1 : la variable étudiée ne suit pas une distribution Normale dans la population générale.

Le test de Shapiro-Wilk se réalise de la façon suivante :

```
shapiro.test(Temp_clean$temperature)
```

ou

```
Temp_clean |>  
  pull(temperature) |>  
  shapiro.test()
```

Shapiro-Wilk normality test

```
data: pull(Temp_clean, temperature)  
W = 0.97216, p-value = 0.7001
```

la fonction `pull()` permet d'extraire une colonne (ici `temperature`) d'un tibble (ici `Temp_clean`) et de la transformer en vecteur.

W est la statistique du test. Elle permet à RStudio de calculer la p -value. Ici, $p > \alpha$. On ne peut donc pas rejeter l'hypothèse nulle de normalité : on ne peut pas exclure que dans la population générale, la température suive bel et bien une distribution Normale. Les conditions d'application du test de Student sont bien vérifiées.

⚠ Tests et décision : rappel de cours

À l'issue d'un test statistique, la décision finale est toujours prise par rapport à l'hypothèse nulle (H_0) :

- Si la p -value du test est supérieure ou égale à α , on dit qu'on ne peut pas rejeter l'hypothèse nulle H_0 . Attention, on ne dit jamais que " H_0 est vraie", car il est impossible de le vérifier avec une certitude absolue. Toutefois, les données observées (celles de notre échantillon), sont compatibles avec l'hypothèse nulle que nous avons formulée, jusqu'à preuve du contraire.
- Si la p -value du test est inférieure à α , on dit qu'on rejette l'hypothèse nulle au seuil α . Autrement dit, les données observées ne sont pas compatibles avec l'hypothèse nulle. On accepte alors l'hypothèse alternative (H_A).

L'hypothèse nulle est toujours l'hypothèse la moins “intéressante”, celle pour laquelle “il ne se passe rien de notable” (par exemple : “les données suivent la distribution Normale”, ou “les moyennes sont égales”).

1.6.2 Signification de la p -value

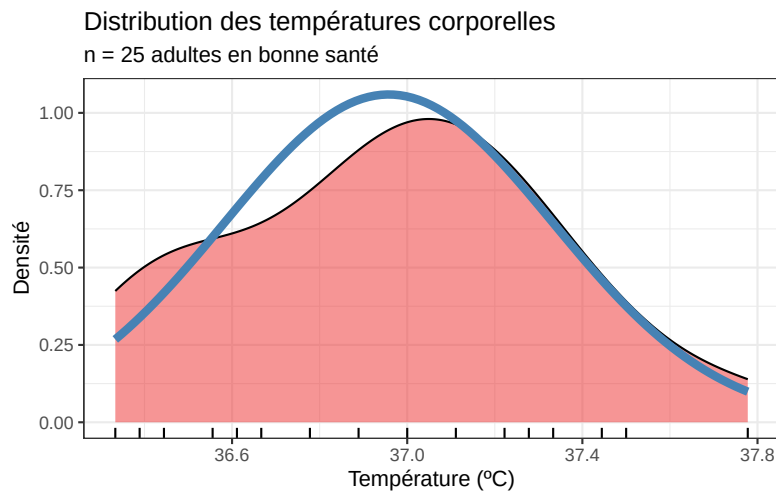
La p -value est une grandeur centrale en statistiques et elle est souvent mal comprise et donc mal interprétée. Je prends donc le temps ici d'expliquer ce qu'est la p -value et comment il faut la comprendre.

Définition : la p -value

La p -value d'un test statistique, c'est la probabilité, si H_0 est vraie, d'obtenir un effet au moins aussi extrême que celui qu'on a observé dans l'échantillon, sous le seul effet du hasard.

Ici, la p -value de notre test de Normalité de Shapiro-Wilk vaut 0.7101. Cela signifie que si les données suivent réellement la loi Normale dans la population générale (donc si H_0 est vraie), l'écart à la Normalité que nous avons observé (ou un écart encore plus important), peut être observé dans 70.1% des cas. Autrement dit, si on prélève un grand nombre d'échantillons de 25 adultes dans la population générale et qu'on regarde à quoi ressemble la distribution des températures dans chacun de ces échantillons, pour 70.1% d'entre eux, la distribution obtenue sera au moins aussi éloignée de la distribution Normale que celle que nous avons observée ici.

Dans notre cas, l'écart entre la loi Normale et les données de notre échantillon peut être visualisé de la façon suivante :

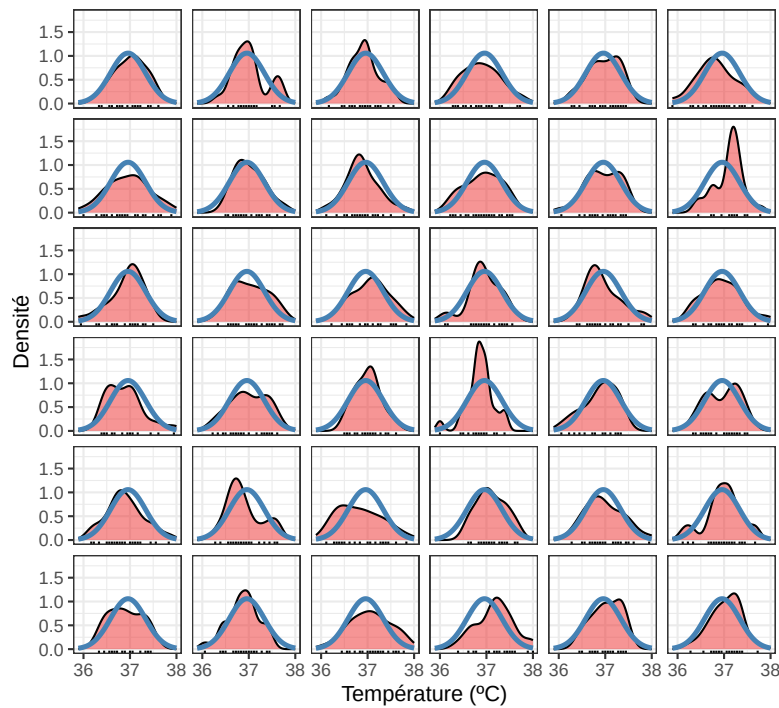


La courbe de densité des données observées est en rouge, et la distribution Normale théorique correspond à la courbe en bleu. Il y a donc un écart entre la courbe en cloche parfaite de la loi Normale et les données observées. La p -value du test de Shapiro-Wilk nous dit que si la température des adultes en bonne santé suit réellement la loi Normale dans la population générale, alors, l'écart que nous avons observé, ou un écart encore plus important, peut être observé simplement par hasard dans 70.1% des cas. Autrement dit, c'est très probable, et on peut donc considérer que l'écart à la loi Normale que nous avons observé est le fruit du hasard et que notre variable suit donc bien la Loi Normale.

Pour bien comprendre cette notion importante, je simule ci-dessous 36 échantillons de 25 adultes dont les températures suivent parfaitement la loi Normale dans la population générale. Je me place donc dans la situation où je sais que H_0 est vraie, pour illustrer la notion de *fluctuation d'échantillonnage*. En raison du seul hasard de l'échantillonnage, et alors même que les échantillons que je génère sont issus d'une population qui suit parfaitement la Normale, la distribution dans chaque échantillon s'écarte parfois fortement de la courbe en cloche théorique :

Distribution des températures corporelles

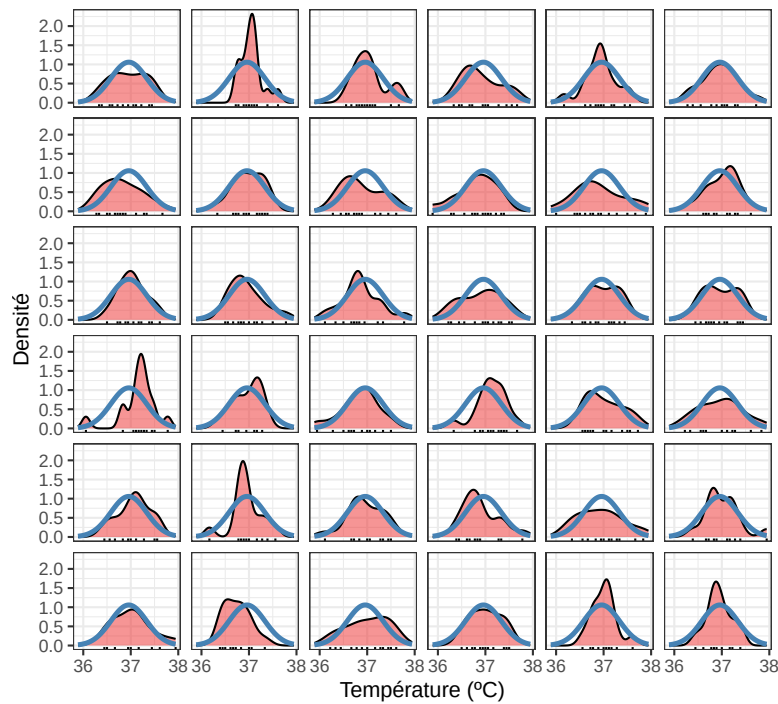
36 échantillons de $n = 25$ adultes en bonne santé



On voit bien ici que certains échantillons s'écartent fortement de la distribution théorique alors même que tous les échantillons sont issus d'une population Normale. Et plus l'échantillon sera de taille réduite, plus les écarts à la courbe en cloche parfaite seront grands. La preuve ci-dessous avec des échantillons de $n = 15$ adultes au lieu de 25 :

Distribution des températures corporelles

36 échantillons de $n = 15$ adultes en bonne santé



Au final, la p -value de 0.701 de notre test de Shapiro-Wilk nous indique que l'hypothèse de la Normalité n'est pas incompatible avec les données que nous avons observées.

Imaginons qu'à l'inverse, nous ayons obtenu une p -value très faible, égale à 0.01 par exemple (donc inférieure à notre seuil α de 0.05). Nous aurions alors rejeté l'hypothèse nulle. En effet, obtenir une p -value de 0.01, signifie que si H_0 est vraie, obtenir un écart à la courbe en cloche théorique aussi important que celui que nous observons est très peu probable (une chance sur 100). Puisqu'il est très improbable d'observer un tel écart si H_0 est vraie, on en conclut que H_0 n'est pas vraie : les données sont incompatibles avec l'hypothèse nulle et on la rejette donc logiquement.

Cette logique sera valable pour tous les autres tests statistiques que nous aborderons dans cet ouvrage. Pour un test de Normalité, on regarde l'écart entre la distribution Normale et les données observées. Pour un test de comparaison de moyennes, on regarde l'écart entre la moyenne théorique et la moyenne observée, ou entre les 2 moyennes qu'on essaie de comparer. Mais la philosophie reste la même.

1.6.3 Réalisation du test de Student et interprétation

Puisque les conditions d'application du test de Student à un échantillon sont vérifiées, nous avons le droit de faire ce test, et nous devons donc maintenant spécifier les hypothèses nulles et alternatives que nous allons utiliser pour le réaliser :

- H_0 : dans la population générale, la température corporelle moyenne des adultes en bonne santé vaut 37°C ($\mu = 37$).
- H_1 : dans la population générale, la température corporelle moyenne des adultes en bonne santé est différente de 37°C ($\mu \neq 37$).

Hypothèses et paramètres

Notez que les hypothèses des tests statistiques concernent toujours la valeur d'un paramètre de la population générale, et non la valeur des estimateurs calculés dans un échantillon.

On réalise ensuite le test de la façon suivante :

```
t.test(Temp_clean$temperature, mu = 37)
```

ou

```
t.test(temperature ~ 1, mu = 37, data = Temp_clean)
```

ou encore,

```
Temp_clean |>
  pull(temperature) |>
  t.test(mu = 37)
```

One Sample t-test

```
data: pull(Temp_clean, temperature)
t = -0.56065, df = 24, p-value = 0.5802
alternative hypothesis: true mean is not equal to 37
```

```
95 percent confidence interval:
 36.80235 37.11321
sample estimates:
mean of x
 36.95778
```

Les résultats fournis ont une forme particulière qui est utilisée par de nombreuses fonctions de tests statistiques dans R. Ils méritent donc qu'on s'y attarde un peu.

Sur la première ligne, R nous confirme que nous avons bien réalisé un test de Student à un échantillon. La première ligne de résultats fournit la valeur du t calculé (ici, -0.56), le nombre de degrés de liberté (ici, $df = 24$), et la p -value (ici, 0.58 , soit une valeur supérieure à α). Cette première ligne contient donc tous les résultats du test qu'il conviendrait de rappeler dans un rapport. On devrait ainsi dire :

Au seuil α de 5%, le test de Student ne permet pas de rejeter l'hypothèse nulle $\mu = 37$ ($t = -0.56$, $ddl = 24$, $p = 0.58$). Les données observées sont donc compatibles avec l'hypothèse selon laquelle la température corporelle moyenne des adultes en bonne santé vaut 37°C .

C'est de cette manière que vous devriez rapporter les résultats de ce test dans un compte rendu ou un rapport à partir de maintenant.

Dans les résultats du test, la ligne suivante (**alternative hypothesis: ...**) **ne donne pas la conclusion du test**. Il s'agit simplement d'un rappel concernant l'hypothèse alternative qui a été utilisée pour réaliser le test. Ici, l'hypothèse alternative utilisée est une hypothèse bilatérale ($\mu \neq 37$). Nous verrons plus tard comment spécifier des hypothèses alternatives unilatérales, même si la plupart du temps, mieux vaut s'abstenir de réaliser de tels tests (à moins bien sûr d'avoir une bonne raison de le faire).

Les résultats fournis ensuite concernent, non plus le test statistique à proprement parler, mais l'estimation. Ici, la moyenne de l'échantillon est fournie. Il s'agit de la meilleure estimation possible de la moyenne de la population : $\bar{x} = \hat{\mu} = 36.96$. Comme pour toutes les estimations, cette valeur est entachée d'incertitude liée à la fluctuation d'échantillonnage. L'intervalle de confiance à 95% de

cette estimation de moyenne est donc également fourni : [36.80;37.11]. Vous notez qu'il s'agit des mêmes valeurs que celles que nous avons calculées dans la Section 1.4.2. Autrement dit, cet intervalle contient les valeurs les plus vraisemblables pour la véritable valeur de moyenne dans la population générale. Cela confirme bien que nous n'avons pas prouvé au sens strict que la moyenne de la population vaut 37°C. Nous avons en réalité montré que nous ne pouvions pas exclure que la moyenne de la population générale soit de 37°C. Puisque cette valeur est comprise dans l'intervalle de confiance, on ne peut donc pas l'exclure : nos données sont compatibles avec cette hypothèse. Mais beaucoup d'autres valeurs figurent aussi dans cet intervalle. Il est donc tout à fait possible que la moyenne soit en réalité différente de 37°C (par exemple, 36.9°C). Pour en être sûr, il faudrait probablement un échantillon de plus grande taille afin de limiter l'incertitude, d'augmenter la puissance statistique de notre test, et ainsi d'être en mesure de détecter des différences subtiles.

1.7 L'alternative non paramétrique

Si jamais les conditions d'application du test de Student à un échantillon n'étaient pas remplies, il faudrait alors réaliser son équivalent non paramétrique : le **test de Wilcoxon des rangs signés**. Ce test est moins puissant que son homologue paramétrique. On ne l'effectue donc que lorsque l'on n'a pas le choix :

```
wilcox.test(Temp_clean$temperature, mu = 37, conf.int = TRUE)
```

ou

```
wilcox.test(temperature ~ 1, mu = 37, conf.int = TRUE, data = Temp_clean)
```

ou encore

```
Temp_clean |>
  pull(temperature) |>
  wilcox.test(mu = 37, conf.int = TRUE)
```

Wilcoxon signed rank exact test

```
data: pull(Temp_clean, temperature)
V = 143, p-value = 0.6084
alternative hypothesis: true location is not equal to 37
95.9 percent confidence interval:
 36.77778 37.11111
sample estimates:
(pseudo)median
 36.94444
```

La syntaxe est identique à celle du test de Student à un échantillon à une exception près : l'ajout de l'argument `conf.int = TRUE` qui permet d'afficher la (pseudo)médiane de l'échantillon et son intervalle de confiance à 95%.

Les hypothèses nulles et alternatives de ce test sont les mêmes que celles du test de Student à un échantillon. En toute rigueur, on compare la médiane à une valeur théorique, et non la moyenne. Mais dans la pratique, la grande majorité des utilisateurs de ce test font l'amalgame entre moyenne et médiane. Ici, la conclusion correcte devrait donc être :

Au seuil α de 5%, on ne peut pas rejeter l'hypothèse nulle (test de Wilcoxon des rangs signés, $V = 143$, $p = 0.6077$). La médiane de la population ($\widehat{med} = 36.94$) n'est pas significativement différente de 37°C (IC 95% : [36.78; 37.11]).

Si les données ne suivent pas la loi Normale, la médiane est bien la métrique la plus intéressante puisque c'est elle qui nous renseigne sur la tendance centrale des données.

Enfin, les tests de Wilcoxon renvoient souvent des messages d'avertissement. Il ne s'agit que de ça : des avertissements. Tant que la p -value d'un test est éloignée de la valeur seuil α , cela n'a pas d'importance. Quand en revanche la p -value est très proche de α , les messages d'avertissement doivent vous alerter : il faut être très prudent face aux conclusions du test qui peuvent alors être assez "fragiles".

1.8 Les notions d'erreur et de puissance statistique

Pour avoir le droit de réaliser un test paramétrique, il faut au préalable vérifier qu'un certain nombre de conditions sont vérifiées. Si ce n'est pas le cas, on réalise un équivalent non paramétrique. On peut alors se demander pourquoi ne pas se contenter de faire des tests non paramétriques systématiquement, sans s'embêter à faire des tests supplémentaires ou des tests paramétriques.

La raison est simple et elle est liée aux notions d'erreur et de puissance statistique.

Définitions

- **Erreur de type I** : notée α , c'est la probabilité de rejeter à tort l'hypothèse nulle. C'est donc la probabilité de rejeter H_0 alors qu'elle est vraie.
- **Erreur de type II** : notée β , c'est la probabilité d'accepter à tort l'hypothèse nulle. C'est donc la probabilité d'accepter H_0 alors qu'elle est fausse.
- **Puissance statistique** : notée $1 - \beta$, c'est la probabilité de rejeter l'hypothèse nulle à raison. C'est donc la probabilité de rejeter H_0 quand elle est réellement fausse.

À chaque fois que l'on réalise un test statistique, on commet nécessairement les 2 types d'erreurs α et β . On souhaite évidemment minimiser les erreurs, mais on ne peut malheureusement pas faire baisser les 2 en même temps. Faire baisser α (pour diminuer les faux positifs) conduit toujours à augmenter β (les faux négatifs). Faire baisser α revient en effet à accepter plus souvent l'hypothèse nulle quand elle est vraie. Cela conduit inévitablement à accepter aussi plus souvent l'hypothèse nulle quand elle est fausse (et donc, à augmenter les faux négatifs).

Pour bien comprendre l'enjeu associé à ces erreurs, prenons l'exemple de notre système judiciaire. Lorsqu'un accusé est jugé, il est présumé innocent jusqu'à preuve du contraire. Le procès est l'équivalent d'un test statistique, avec :

- H_0 : l'accusé est innocent
- H_1 : l'accusé est coupable

Commettre une erreur de type I revient à condamner à tort l'accusé (on rejette à tort H_0), donc on condamne un innocent. À l'inverse, commettre une erreur de type II revient à libérer un coupable (accepter à tort H_0). Un système de justice plus strict condamnera un plus grand nombre d'accusés, qu'ils soient coupables ou non. Un système plus strict fera donc augmenter l'erreur de type I et baisser l'erreur de type II. À vous de voir ce que vous préférez : libérer plus de coupables, ou condamner plus d'innocents ?

En statistiques, la question est tranchée puisqu'on préfère maintenir l'erreur de type I à un niveau assez faible (à 5% ou moins), quitte à laisser augmenter l'erreur de type II (qui est considérée comme acceptable jusqu'à 20% environ). Toutefois, seule l'erreur de type I est sous notre contrôle. En effet, c'est nous qui la choisissons lorsque l'on fixe le seuil α de nos tests statistiques.

! À retenir

C'est vous qui fixez l'erreur de type I lorsque vous faites un test statistique. L'erreur de type I est le seuil α du test, que l'on fixe en général à 0,05 (soit 5%) dans le domaine des sciences du vivant.

Une fois que le seuil α est fixé, l'erreur β l'est aussi dans une certaine mesure. Mais on ne peut la connaître avec précision car elle dépend de beaucoup de choses, notamment la taille des échantillons dont on dispose, la variabilité des données, le type de test réalisé, etc. En général, **plus la taille de l'échantillon sera grande, plus l'erreur β sera faible, et donc plus la puissance sera élevée**. De même, par rapport aux tests non paramétriques, les tests paramétriques permettent de minimiser l'erreur β et donc d'augmenter la puissance.

Puisque la puissance statistique vaut $1 - \beta$, cela revient à dire que les tests paramétriques sont plus puissants que les tests non paramétriques (parfois, beaucoup plus). Au contraire des erreurs de type I et II, la puissance est une grandeur que l'on souhaite maximiser. On aimerait en effet être capables de systématiquement rejeter H_0 quand elle est fausse. Nous avons

vu plus haut que c'est hélas impossible. Mais choisir le bon test et la bonne procédure statistique permettent néanmoins d'augmenter la puissance, jusqu'à un certain point. C'est la raison pour laquelle on réalisera toujours un test paramétrique si les données dont on dispose le permettent (donc si les conditions d'application des tests paramétriques sont respectées). Et ce n'est qu'en dernier recours qu'on se tournera vers les tests non paramétriques, toujours moins puissants.

! Important

Un test paramétrique est toujours plus puissant que ses homologues non paramétriques. Avec un test paramétrique, il est donc plus probable de rejeter H_0 à raison qu'avec un test non paramétrique.

1.9 Bilan

Nous avons vu dans ce chapitre quelle est la procédure à suivre pour réaliser un test de comparaison de la moyenne d'une population à une valeur théorique :

1. examen préliminaire des données
2. calcul de statistiques descriptives
3. création de graphiques exploratoires
4. vérification des conditions d'application du test paramétrique
5. réalisation du test paramétrique ou non paramétrique selon l'issue de l'étape 4

Mais nous avons aussi abordé des notions statistiques essentielles pour la suite :

1. Les ingrédients indispensables pour réaliser un test statistique (les hypothèses nulle et alternative, la statistique du test et le seuil α).
2. La p -value et la décision du test.
3. Les erreurs de type I (α) et II (β).
4. La puissance statistique ($1 - \beta$) qui n'a rien à voir avec la notion de précision.
5. La notion de test paramétrique ou non paramétrique.

Assurez-vous d'avoir les idées claires sur toutes ces notions car elles sont absolument centrales pour ne pas faire/dire de bêtises lorsque l'on analyse des données.

1.10 Exercice d'application

Le fichier [Temperature2.csv](#) contient les données brutes d'une seconde étude similaire, réalisée à plus grande échelle. Importez ces données et analysez-les afin de vérifier si la température corporelle moyenne des adultes en bonne santé vaut bien 37°C . Comme toujours, avant de vous lancer dans la réalisation des tests statistiques, prenez le temps d'examiner vos données comme nous l'avons décrit dans la Section 1.4 et la Section 1.5, afin de savoir où vous allez, et de repérer les éventuelles données manquantes ou aberrantes. Enfin, interprétez les résultats à la lumière des notions que nous avons abordées ici (en particulier la notion de puissance statistique).

2 Comparaison de moyennes : deux échantillons appariés

2.1 Pré-requis

Pour ce nouveau chapitre, je vous conseille de travailler dans un nouveau script que vous placerez dans votre répertoire de travail, et dans une nouvelle session de travail (Menu **Session > Restart R**). Inutile en revanche de créer un nouveau **Rproject** : vous pouvez tout à fait avoir plusieurs scripts dans le même répertoire de travail et pour un même **Rproject**. Comme toujours, consultez [le livre en ligne du semestre 3](#) si vous ne savez plus comment faire.

Si vous êtes dans une nouvelle session de travail (ou que vous avez quitté puis relancé **RStudio**), vous devrez penser à recharger en mémoire les packages utiles. Dans ce chapitre, vous aurez besoin d'utiliser les mêmes packages que précédemment :

- le **tidyverse** (Wickham 2023), qui comprend notamment le package **readr** (Wickham, Hester, et Bryan 2026), pour importer facilement des fichiers **.csv** au format **tibble**, le package **dplyr** (Wickham, François, et al. 2026), pour manipuler des tableaux, et le package **ggplot2** (Wickham, Chang, et al. 2026) pour les représentations graphiques.
- **skimr** (Waring et al. 2026), qui permet de calculer des résumés de données très informatifs.

```
library(tidyverse)
library(skimr)
```

Vous aurez également besoin des jeux de données suivants que vous pouvez dès maintenant télécharger dans votre répertoire de travail :

- [Autruches.csv](#)
- [Testosterone.csv](#)

Enfin, je spécifie ici une fois pour toutes le thème que j'utiliserai pour tous les graphiques de ce chapitre. Libre à vous de choisir un thème différent ou de vous contenter du thème proposé par défaut :

```
theme_set(theme_bw())
```

2.2 Contexte

On s'intéresse ici à la comparaison de 2 séries de données dont les observations sont liées 2 à 2. C'est par exemple le cas lorsque l'on fait subir un traitement à différents sujets et que l'on souhaite comparer les mesures obtenues avant et après le traitement.

Autrement dit, dans les plans d'expériences appariés, **les deux traitements** ou modalités **sont appliqués à chaque unité d'échantillonnage** : chaque sujet ou unité d'échantillonnage fournit plusieurs valeurs. Ça n'était pas le cas du chapitre précédent (Chapitre 1) où chaque adulte n'avait fourni qu'une unique valeur de température.

Voici quelques exemples de situations qui devraient être traitées avec des tests sur données appariées :

- Comparaison de la masse de patients avant et après une hospitalisation.
- Comparaison de la diversité de peuplements de poissons dans des lacs avant et après contamination par des métaux lourds.
- Test des effets d'une crème solaire appliquée sur un bras de chaque volontaire alors que l'autre bras ne reçoit qu'un placebo.
- Test des effets du tabagisme dans un échantillon de fumeurs, dont chaque membre est comparé à un non fumeur choisi pour qu'il lui ressemble le plus possible en termes d'âge, de masse, d'origine ethnique et sociale, etc.

- Test des effets que les conditions socio-économiques ont sur les préférences alimentaires en comparant des vrais jumeaux élevés dans des familles adoptives séparées qui diffèrent en termes de conditions socio-économiques.

Les 2 derniers exemples montrent que même des individus séparés peuvent constituer une “paire statistique” s’ils partagent un certain nombre de caractéristiques (physiques, environnementales, génétiques, comportementales, etc.) pertinentes pour l’étude.

Ici, nous allons nous intéresser au lien qui pourrait exister entre la production de testostérone et l’immunité chez une espèce d’oiseau vivant en Amérique du Nord, [le carouge à épaulettes](#).



Figure 2.1: Le carouge à épaulettes

Chez de nombreuses espèces, les mâles ont plus de chances d’attirer des femelles s’ils produisent des niveaux de testostérone élevés. Est-ce que la forte production de testostérone de certains mâles a un coût, notamment en termes d’immunocompétence ? Autrement dit, est-ce que produire beaucoup de testostérone au moment de la reproduction (ce qui fournit un avantage sélectif) se traduit par une immunité plus faible par la suite, et donc une plus forte susceptibilité de contracter des maladies (ce qui constitue donc un désavantage sélectif) ? Ce type de question est

central pour comprendre comment l'allocation des ressources affecte à la fois la survie et la fécondité des individus.

Pour étudier cette question, une équipe de chercheurs (Hasselquist et al. 1999) a mis en place le dispositif expérimental suivant. Les niveaux de testostérone de 13 carouges à épaulettes mâles ont été artificiellement augmentés par l'implantation chirurgicale d'un microtube perméable contenant de la testostérone. L'immunocompétence a été mesurée pour chaque oiseau avant et après l'opération chirurgicale. La variable mesurée est la production d'anticorps suite à l'exposition des oiseaux avec un antigène non pathogène mais censé déclencher une réponse immunitaire. Les taux de production d'anticorps sont exprimés en logarithmes de densité optique par minute ($\ln \frac{mOD}{min}$). Si la production de testostérone influence l'immunocompétence, on s'attend à observer des différences de production d'anticorps avant et après l'intervention chirurgicale.

2.3 Importation et mise en forme des données

Les données se trouvent dans le fichier [Testosterone.csv](#). Importez ces données dans un objet nommé `Testo` et affichez son contenu.

```

Testo

# A tibble: 13 x 5
  blackbird beforeImplant afterImplant logBeforeImplant logAfterImplant
    <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
1         1         105           85          4.65          4.44
2         2          50           74          3.91          4.3
3         3         136          145          4.91          4.98
4         4          90           86          4.5          4.45
5         5         122          148          4.8           5
6         6         132          148          4.88           5
7         7         131          150          4.88          5.01
8         8         119          142          4.78          4.96
9         9         145          151          4.98          5.02
10        10         130          113          4.87          4.73
11        11         116          118          4.75          4.77

```

12	12	110	99	4.7	4.6
13	13	138	150	4.93	5.01

Visiblement, il n'y a pas de données manquantes mais certaines variables sont inutiles. En effet, nous aurons besoin des variables transformées en logarithmes, mais pas des 2 colonnes `beforeImplant` et `afterImplant`. Nous allons donc les retirer avec la fonction `select()`. Par ailleurs, la variable `blackbird` est importante puisque chaque individu a fourni 2 valeurs de production d'anticorps : 1 avant et 1 après l'opération chirurgicale. Il sera donc important de conserver cet identifiant individuel. Toutefois, il apparaît ici sous la forme d'une variable numérique alors qu'il s'agit d'un identifiant, d'un code. Il faut donc le transformer en facteur car cela n'aurait pas de sens de calculer une moyenne des identifiants par exemple. Pour cela, nous utiliserons la fonction `factor()` à l'intérieur de `mutate()`. Enfin, nous renommerons les colonnes avec `rename()` pour avoir des noms plus courts et plus faciles à utiliser. Si vous ne vous rappelez plus comment utiliser ces fonctions, consultez ces chapitres du livre en ligne de biométrie du semestre 3 : [select\(\) et rename\(\)](#), [mutate\(\) et factor\(\)](#). Enfin, nous donnerons le nom `Testo_large` au tableau modifié :

```
Testo_large <- Testo |>
  select(-beforeImplant, -afterImplant) |> # Suppression des colonnes inutiles
  mutate(blackbird = factor(blackbird)) |> # Transformation en facteur
  rename(ID = blackbird,                    # Changement des noms de variables
         Before = logBeforeImplant,
         After = logAfterImplant)
```

```
Testo_large
```

```
# A tibble: 13 x 3
  ID    Before After
<fct> <dbl> <dbl>
1 1      4.65  4.44
2 2      3.91  4.3
3 3      4.91  4.98
4 4      4.5   4.45
5 5      4.8   5
6 6      4.88  5
```

7	7	4.88	5.01
8	8	4.78	4.96
9	9	4.98	5.02
10	10	4.87	4.73
11	11	4.75	4.77
12	12	4.7	4.6
13	13	4.93	5.01

Le tableau `Testo_large` dont nous disposons maintenant n'est pas dans un format qui nous permettra de réaliser toutes les opérations dont nous aurons besoin. En réalité, il ne s'agit pas d'un "tableau rangé" au sens du tidyverse. Un tableau rangé est un tableau dans lequel chaque ligne correspond à une unique observation et chaque colonne correspond à une unique variable. Ici, nous devrions avoir les 3 variables suivantes :

1. L'identifiant des individus. La colonne ID correspond à cette variable.
2. Le moment auquel chaque mesure a été effectuée, avant ou après l'opération chirurgicale. Cette information est pour l'instant stockée dans l'en-tête des colonnes 2 et 3 du tableau `Testo_large`
3. La mesure de réponse immunitaire (en logarithme de la densité optique par minute). Cette information est pour l'instant stockée sous forme de valeurs numériques dans les colonnes 2 et 3 du tableau `Testo_large`

Pour obtenir un tableau rangé, il nous faut donc réorganiser les colonnes 2 et 3 du tableau `Testo_large` :

- l'en-tête de ces 2 colonnes devrait constituer une nouvelle variable que nous nommerons `Moment`
- le contenu de ces 2 colonnes (les valeurs numériques) devrait constituer une nouvelle variable que nous nommerons `DO` (pour densité optique).

Pour effectuer cette transformation, nous utiliserons la fonction `pivot_longer()` du package `tidyr` (il est déjà chargé en mémoire si vous avez chargé le `tidyverse`). Comme son nom l'indique, cette fonction produira un tableau plus "long" (qui aura plus de lignes) que le tableau de départ. Nous l'appellerons donc `Testo_long` :

```
Testo_long <- Testo_large |>
  pivot_longer(cols = c(Before, After), # Les colonnes qu'on veut réorganiser
               names_to = "Moment", # Quel nom donner à la variable qui contiendra les
               values_to = "D0") |> # Quel nom donner à la variable qui contiendra l
  mutate(Moment = factor(Moment, levels = c("Before", "After")))
```

```
Testo_long
```

```
# A tibble: 26 x 3
   ID      Moment    D0
  <fct> <fct>   <dbl>
1 1      Before  4.65
2 1      After  4.44
3 2      Before  3.91
4 2      After  4.3
5 3      Before  4.91
6 3      After  4.98
7 4      Before  4.5
8 4      After  4.45
9 5      Before  4.8
10 5     After  5
# i 16 more rows
```

Ce nouvel objet contient les mêmes données que précédemment, mais sous un format différent (il contient maintenant 26 lignes et non plus 13) : il s'agit d'un tableau rangé.

La plupart du temps, on a besoin de ces 2 formats de tableaux quand nous traitons des données. Le tableau au format long est à privilégier pour les représentations graphiques et les tests statistiques, et le format court sert souvent à présenter des résultats sous une forme synthétique. Mais parfois (et c'est justement le cas quand on dispose de données appariées comme pour notre exemple de lien entre testostérone et immunocompétence), le tableau au format large permettra de faire certains graphiques, certains tests ou certaines manipulations plus facilement que le tableau rangé au format long. Si on ne dispose que d'un tableau au format large, on peut passer au format long, comme nous venons de le faire, grâce à la fonction `pivot_longer()`. Et si on ne dispose que d'un tableau au format long, on peut passer au format large grâce à la fonction `pivot_wider()`. Nous avons déjà évoqué cette

fonction dans le livre en ligne de biométrie du semestre 4 pour mettre en forme des résultats obtenus avec `summarise()` ([par exemple ici](#)) ou `reframe()` ([ou là](#)), et je vous encourage à y jeter un œil à nouveau pour vous remémorer la syntaxe. Car il est important que vous maîtrisiez ces 2 fonctions dont vous aurez très souvent besoin.

Maintenant que nous disposons de ces 2 tableaux, `Testo_large` et `Testo_long`, nous pouvons commencer à décrire nos données.

2.4 Exploration statistique des données

Pour décrire simplement les données, nous nous en tiendrons ici à l'utilisation des fonctions `summary()` et `skim()`.

Pour la fonction `summary()`, le plus simple est toujours d'utiliser le tableau au format large :

```
summary(Testo_large)
```

	ID	Before	After
1	:1	Min. :3.910	Min. :4.30
2	:1	1st Qu.:4.700	1st Qu.:4.60
3	:1	Median :4.800	Median :4.96
4	:1	Mean :4.734	Mean :4.79
5	:1	3rd Qu.:4.880	3rd Qu.:5.00
6	:1	Max. :4.980	Max. :5.02
(Other):	7		

On constate ici que pour les 2 traitements, les valeurs des différents indices sont très proches entre les 2 séries de données, avec des valeurs de densité optique (DO) légèrement supérieures après l'opération chirurgicale (sauf pour le premier quartile).

Pour la fonction `skim()` le plus simple est là aussi d'utiliser le tableau large :

```
skim(Testo_large)
```

```
-- Data Summary -----
Name                               Values
Number of rows                     13
Number of columns                   3

-----
Column type frequency:
  factor                            1
  numeric                          2

-----
Group variables                     None

-- Variable type: factor -----
skim_variable n_missing complete_rate ordered n_unique top_counts
1 ID          0          1 FALSE          13 1: 1, 2: 1, 3: 1, 4: 1

-- Variable type: numeric -----
skim_variable n_missing complete_rate mean    sd    p0 p25  p50  p75 p100 hist
1 Before      0          1 4.73 0.280 3.91 4.7 4.8  4.88 4.98
2 After       0          1 4.79 0.262 4.3  4.6 4.96 5   5.02
```

On arrive toutefois aux mêmes résultats avec le tableau long,
à condition de grouper les données par traitement (variable
Traitement) avec `group_by()` :

```
Testo_long |>
  group_by(Moment) |>
  skim(DO)
```

```
-- Data Summary -----
Name                               Values
Number of rows                     26
Number of columns                   3

-----
Column type frequency:
  numeric                          1

-----
Group variables                     Moment

-- Variable type: numeric -----
skim_variable Moment n_missing complete_rate mean    sd    p0 p25  p50  p75
```

```

1 DO          Before          0          1 4.73 0.280 3.91 4.7 4.8 4.88
2 DO          After           0          1 4.79 0.262 4.3 4.6 4.96 5
  p100 hist
1 4.98
2 5.02

```

Cela revient à demander à la fonction `skim()` de produire un résumé des données de densité optique (variable `DO`), pour chaque catégorie de la variable `Moment`, soit un résumé pour la catégorie `Before` (avant l'intervention chirurgicale), et un résumé pour la catégorie `After` (après l'intervention chirurgicale).

Par rapport aux résultats fournis par la fonction `summary()`, la fonction `skim()` nous permet de confirmer que les valeurs de `DO` sont très légèrement supérieures après l'opération (sauf pour le premier quartile). Elle nous permet également de constater que l'écart-type est du même ordre de grandeur pour les 2 catégories, bien qu'il soit légèrement plus faible après l'opération. Enfin, les petits histogrammes laissent entrevoir une distribution très asymétrique des données dans chacun des 2 groupes de mesures.

2.5 Exploration graphique des données

Ici, c'est le tableau rangé au format long qui sera le plus adapté. Lorsque nous avons une unique série de données, nous avons utilisé 2 types de représentations graphiques très similaires pour visualiser les données (les histogrammes et les graphiques de densités). Ici, nous allons utiliser ces mêmes types de graphiques mais "facettés". Les graphiques facettés ont été abordés dans [le livre en ligne de biométrie du semestre 3](#). Ils permettent de faire des sous-graphiques pour chaque catégorie d'un facteur. Ici, le facteur `Moment` contient 2 catégories. Les facets nous permettront donc de comparer les 2 distributions de densités optiques.

Outre ces graphiques, nous utiliserons aussi les stripcharts et les boîtes à moustaches pour comparer les 2 catégories. Ces 2 types de graphiques sont particulièrement adaptés pour ce genre de tâche, et seront aussi très utiles pour l'ANOVA lorsque nous aurons plus de 2 catégories à comparer.

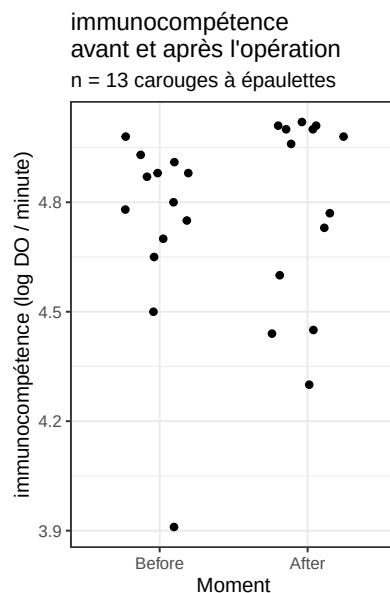
D'une façon générale, nous disposons :

- d'une variable numérique, DO : la mesure de densité optique qui rend compte de l'immunocompétence des carouges à épaulettes
- d'une variable catégorielle, le facteur **Moment** : indique si les valeurs d'immunocompétences ont été mesurées avant ou après l'opération chirurgicale d'implantation de la capsule de testostérone.

Tous les graphiques présentés dans [le chapitre consacré à cette situation précise](#) dans le livre en ligne de biométrie du semestre 3, peuvent être réalisés. N'hésitez pas à le relire, en particulier la section expliquant comment faire apparaître et interpréter les encoches d'incertitudes sur des boîtes à moustaches.

2.5.1 Avec un stripchart

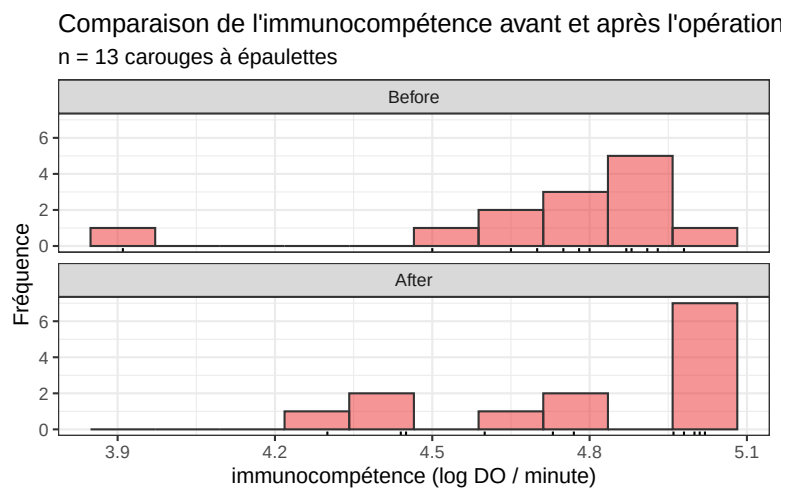
```
Testo_long |>
  ggplot(aes(x = Moment, y = DO)) +
  geom_jitter(height = 0, width = 0.25) +
  labs(y = "immunocompétence (log DO / minute)",
       title = "immunocompétence\navant et après l'opération",
       subtitle = "n = 13 carouges à épaulettes")
```



2.5.2 Avec des histogrammes facettés

Nous allons faire un histogramme pour chaque série de données en utilisant des facettes :

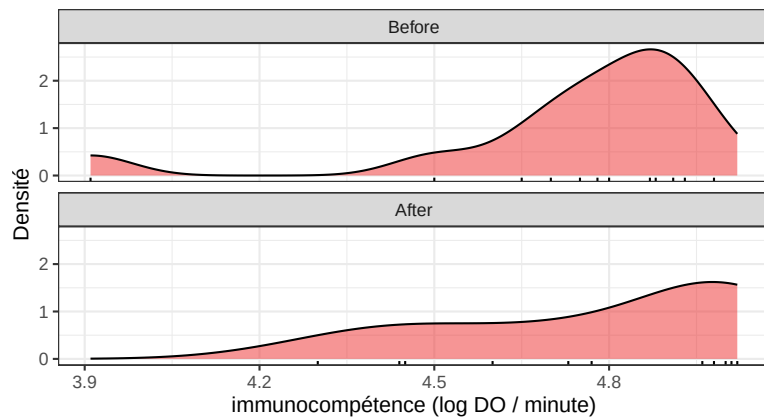
```
Testo_long |>
  ggplot(aes(x = DO)) +
  geom_histogram(bins = 10, fill = "firebrick2", color = "grey20", alpha = 0.5)+
  geom_rug() +
  facet_wrap(~Moment, ncol = 1) +
  labs(x = "immunocompétence (log DO / minute)",
       y = "Fréquence",
       title = "Comparaison de l'immunocompétence avant et après l'opération chirurgicale",
       subtitle = "n = 13 carouges à épaulettes")
```



2.5.3 Avec des diagrammes de densité facettés

```
Testo_long |>
  ggplot(aes(x = DO)) +
  geom_density(fill = "firebrick2", alpha = 0.5) +
  geom_rug() +
  facet_wrap(~Moment, ncol = 1) +
  labs(x = "immunocompétence (log DO / minute)",
       y = "Densité",
       title = "Comparaison de l'immunocompétence avant et après l'opération chirurgicale",
       subtitle = "n = 13 carouges à épaulettes")
```

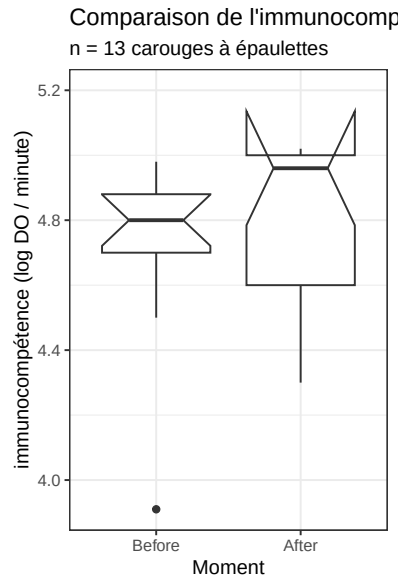
Comparaison de l'immunocompétence avant et après l'opération
n = 13 carouges à épaulettes



2.5.4 Avec des boîtes à moustaches

```
Testo_long |>
  ggplot(aes(x = Moment, y = DO)) +
  geom_boxplot(notch = TRUE) +
  expand_limits(y = 5.2) +
  labs(y = "immunocompétence (log DO / minute)",
       title = "Comparaison de l'immunocompétence avant et après opération chirurgicale",
       subtitle = "n = 13 carouges à épaulettes")
```

Notch went outside hinges
i Do you want `notch = FALSE`?

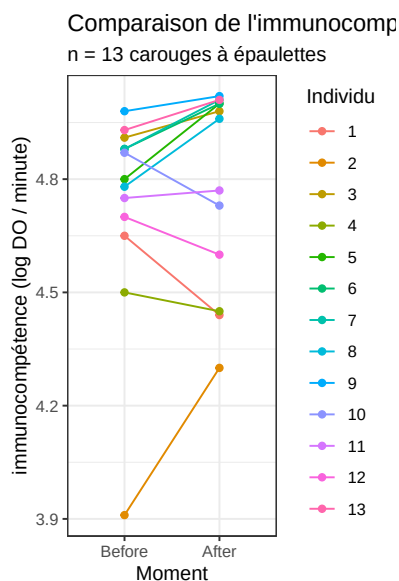


Du point de vue de la **position** des données, ces différents graphiques montrent tous que la seconde série de données (catégorie **After** : après l'opération chirurgicale) présente en moyenne des valeurs très légèrement plus élevées que la première (catégorie **Before** avant l'opération). En termes de **dispersion**, si l'on met de côté la valeur minimale de la série **Before** qui semble atypique (un individu outlier qui présente une immunocompétence très faible avant l'opération), la dispersion des données autour de la tendance centrale semble globalement plus importante pour la série **After**. Enfin, pour ce qui concerne l'**incertitude**, les intervalles de confiance à 95% des médianes (qui apparaissent sous la forme d'encoches sur les boîtes à moustaches) se chevauchent assez largement, ce qui nous permet d'anticiper les résultats des tests que nous ferons ensuite : puisque les encoches se chevauchent, il y a fort à parier que le test de comparaison de moyenne ne montrera aucune différence significative. On note également que l'encoche de la série **After** est particulièrement large : la limite supérieure de l'intervalle de confiance à 95% de la médiane est supérieure à la valeur maximale observée dans l'échantillon. Cela traduit le fait que compte tenu de la grande variabilité des données dans cette série, un échantillon de taille $n = 13$ n'est probablement pas suffisant pour avoir une estimation précise de la médiane.

2.5.5 Avec un nuage de points appariés

Toutes ces représentations graphiques sont certes utiles, mais elles masquent un élément crucial : ce sont les mêmes individus qui sont étudiés avant et après l'opération. Il s'agit de données appariées ! Les graphiques que nous avons faits jusque-là ne permettent pas de visualiser ce lien entre les deux séries de données. Pour avoir une bonne vision de ce qui se passe, il nous faut faire apparaître ce lien entre les 2 séries de données :

```
Testo_long |>
  ggplot(aes(x = Moment, y = DO, group = ID, color = ID)) +
  geom_line() +
  geom_point() +
  labs(y = "immunocompétence (log DO / minute)",
       title = "Comparaison de l'immunocompétence avant et après opération chirurgicale",
       subtitle = "n = 13 carouges à épaulettes",
       color = "Individu")
```



Ce graphique nous donne une image très différente de la réalité des données. On constate ici que l'immunocompétence de certains individus augmente après l'opération (parfois fortement), alors que pour d'autres, elle diminue.

Une façon d'estimer si les changements d'immunocompétence sont majoritairement orientés dans un sens ou non est de calculer l'intervalle de confiance à 95% de la différence d'immunocompétence entre avant et après l'opération. Pour cela, on peut calculer, grâce au tableau large `Testo_large`, les différences d'immunocompétences (DO après opération moins DO avant opération), pour chacun des 13 individus. Puis, grâce à la fonction `mean_cl_normal()` déjà utilisée à plusieurs reprises, on calcule l'intervalle de confiance à 95% de la moyenne de cette différence :

```
# Calcul de la différence de DO (After - Before)
Testo_large <- Testo_large |>
  mutate(Difference = After - Before)

# Affichage du tableau
Testo_large
```

```
# A tibble: 13 x 4
  ID      Before After Difference
  <fct>   <dbl> <dbl>      <dbl>
1 1        4.65  4.44     -0.21
2 2        3.91  4.3      0.390
3 3        4.91  4.98     0.0700
4 4        4.5   4.45    -0.0500
5 5        4.8   5        0.200
6 6        4.88  5        0.120
7 7        4.88  5.01     0.130
8 8        4.78  4.96     0.180
9 9        4.98  5.02     0.0400
10 10       4.87  4.73    -0.140
11 11       4.75  4.77     0.0200
12 12       4.7   4.6     -0.100
13 13       4.93  5.01     0.0800
```

```
# Calcul de la moyenne des différences et de son IC95%
Testo_large |>
  reframe(mean_cl_normal(Difference))
```

```
# A tibble: 1 x 3
  y      ymin ymax
```

```
<dbl>    <dbl> <dbl>
1 0.0562 -0.0401 0.152
```

On constate ici que la moyenne des différences de densité optique vaut 0.06, soit une valeur positive, qui montre que l'immunocompétence augmente après l'opération (ce qui semble aller à l'opposé de l'hypothèse des chercheurs). Cette moyenne reste néanmoins très proche de 0. D'ailleurs, l'intervalle de confiance de cette moyenne comprend les valeurs situées entre -0.04 et +0.15. La valeur 0 est donc comprise dans cet intervalle. Le zéro fait donc partie des valeurs les plus probables pour la moyenne de ces différences dans la population générale. C'est là encore un résultat qui nous permet d'anticiper sur les résultats du test statistique que nous ferons ensuite.

2.6 Le test paramétrique

2.6.1 Procédure

Le test paramétrique permettant de comparer la moyenne sur des séries appariées est là encore un test de Student : le **test de Student sur données appariées** (étonnant non ?...). En réalité, ce test de Student n'est pas un test de comparaison de moyennes entre 2 séries de données. La procédure est la suivante :

1. Pour chaque individu, calculer la différence d'immunocompétence entre les deux temps de l'expérience (DO après - DO avant opération). C'est ce que nous avons fait plus haut en ajoutant la colonne **Difference** au tableau **Testo_large**.
2. Puisque nous avons 13 individus, nous aurons 13 valeurs de différences. La moyenne de cette différence sera comparée à la valeur théorique 0. Autrement dit, si cette moyenne vaut 0, l'immunocompétence sera la même avant et après l'opération. Si la moyenne des différences n'est pas égale à 0, alors nous aurons prouvé qu'il existe une différence d'immunocompétence entre les 2 groupes, nous aurons prouvé que la procédure chirurgicale d'implantation de la capsule de testostérone

a un impact sur l'immunocompétence des carouges à épaulettes

Attention

Dans un test sur données appariées, on s'intéresse à **la moyenne des différences** entre les données des 2 séries. Cette moyenne est alors comparée à la valeur théorique $\mu = 0$. Ce test est donc équivalent au test vu dans le Chapitre 1 sur la comparaison de la moyenne d'une population à une valeur théorique.

Notez également que **la moyenne des différences** n'est pas équivalente à **la différence des moyennes**. La différence des moyennes est une grandeur qui nous sera utile dans le chapitre suivant (Chapitre 3) sur la comparaison de la moyenne de deux populations lorsque les données sont indépendantes.

2.6.2 Conditions d'application

Les conditions d'application de ce test paramétrique sont presque les mêmes que pour le test de Student à un échantillon :

1. Les individus sur lesquels porte la comparaison doivent être issus d'un échantillonnage aléatoire. Comme toujours, en l'absence d'indication contraire, on considère que cette condition est vérifiée.
2. Les différences par paires entre les 2 modalités du traitement doivent suivre une distribution Normale. Attention, ce n'est donc pas les données brutes de chaque série qui doivent suivre une loi Normale, mais bien la différence "après" - "avant" calculée pour chaque individu. Nous avons déjà calculé ces différences plus haut :

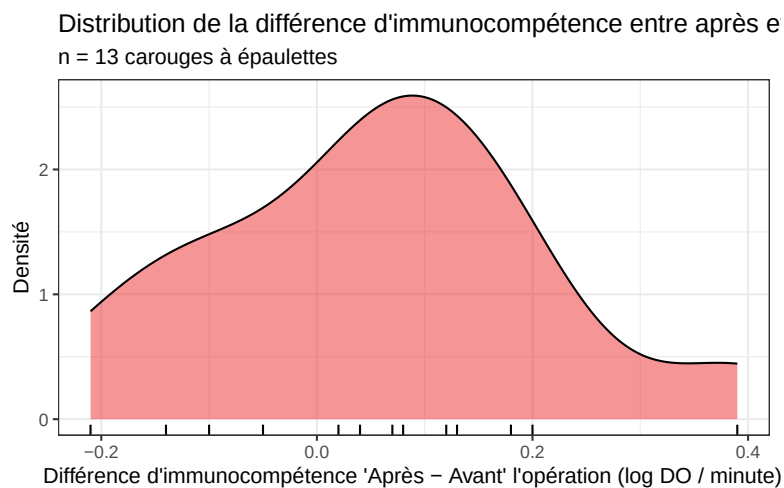
```
# On s'intéresse aux 13 différences calculées sur les 13 individus
Testo_large
```

```
# A tibble: 13 x 4
  ID   Before After Difference
<fct> <dbl> <dbl>     <dbl>
```

1	1	4.65	4.44	-0.21
2	2	3.91	4.3	0.390
3	3	4.91	4.98	0.0700
4	4	4.5	4.45	-0.0500
5	5	4.8	5	0.200
6	6	4.88	5	0.120
7	7	4.88	5.01	0.130
8	8	4.78	4.96	0.180
9	9	4.98	5.02	0.0400
10	10	4.87	4.73	-0.140
11	11	4.75	4.77	0.0200
12	12	4.7	4.6	-0.100
13	13	4.93	5.01	0.0800

Il nous faut donc tester la Normalité de la nouvelle variable **Difference**. Commençons par en faire un graphique :

```
Testo_large |>
  ggplot(aes(x = Difference)) +
  geom_density(fill = "firebrick2", alpha = 0.5) +
  geom_rug() +
  labs(x = "Différence d'immunocompétence 'Après - Avant' l'opération (log DO / minute)",
       y = "Densité",
       title = "Distribution de la différence d'immunocompétence entre après et avant l'o",
       subtitle = "n = 13 carouges à épaulettes")
```



Compte tenu du faible nombre d'individus ($n = 13$), la forme de cette courbe de densité n'est pas si éloignée que ça d'une courbe en cloche (notez que ce n'était pas du tout le cas pour

les données brutes de chaque série de départ qui ont toutes les deux des distributions très éloignées de la distribution Normale). On le vérifie avec un test de normalité de Shapiro-Wilk :

- H_0 : la différence d'immunocompétence des individus suit une distribution Normale.
- H_1 : la différence d'immunocompétence des individus ne suit pas une distribution Normale.

```
Testo_large |>  
  pull(Difference) |>  
  shapiro.test()
```

Shapiro-Wilk normality test

```
data: pull(Testo_large, Difference)  
W = 0.97949, p-value = 0.977
```

Au seuil $\alpha = 0.05$, on ne peut pas rejeter l'hypothèse nulle de normalité pour la différence d'immunocompétence entre après et avant l'intervention chirurgicale (test de Shapiro-Wilk, $W = 0.98$, $p = 0.977$).

Les conditions d'application du test paramétrique sont donc réunies.

⚠ Attention !

Pour ce test, la Normalité doit bien être vérifiée sur la différence entre les 2 groupes de valeurs, et non sur chaque groupe de valeurs pris séparément. C'est une source d'erreur fréquente. Ici, les données de départ (DO avant et DO après) ne suivaient pas du tout une distribution Normale. Pourtant, la différence de DO suit bel et bien la distribution Normale, nous permettant de faire le test paramétrique.

2.6.3 Réalisation du test et interprétation

Le test de Student sur données appariées peut se faire de 2 façons distinctes. Les 2 méthodes fournissent exactement les mêmes résultats, seule la syntaxe utilisée change. Quelle que soit la méthode utilisée, les hypothèses nulles et alternatives sont toujours les mêmes :

- H_0 : le changement moyen de production d'anticorps après la pose chirurgicale de l'implant de testostérone est nul ($\mu_{Diff} = 0$). La procédure chirurgicale n'a pas d'effet sur l'immunocompétence. Les variations observées ne sont que le fruit du hasard de l'échantillonnage.
- H_1 : le changement moyen de production d'anticorps après la pose chirurgicale de l'implant de testostérone n'est pas nul ($\mu_{Diff} \neq 0$). La procédure chirurgicale a un effet significatif sur l'immunocompétence. Les variations observées ne sont pas uniquement dues à la fluctuation d'échantillonnage.

2.6.3.1 Première syntaxe

```
# Méthode n°2 : avec les 2 séries de données et le tableau au format large  
t.test(Testo_large$Before, Testo_large$After, paired = TRUE)
```

Paired t-test

```
data: Testo_large$Before and Testo_large$After  
t = -1.2714, df = 12, p-value = 0.2277  
alternative hypothesis: true mean difference is not equal to 0  
95 percent confidence interval:  
 -0.15238464  0.04007695  
sample estimates:  
mean difference  
 -0.05615385
```

Avec cette première syntaxe, on indique le nom des 2 colonnes du tableau `Testo_large` qui contiennent les 2 séries de données. Il nous faut taper spécifiquement “`Testo_large$Before`” et “`Testo_large$After`”, et non

pas simplement “Before” et “After” car la fonction `t.test` n’a aucun moyen de savoir que “Before” et “After” sont des colonnes du tableau `Testo_large`. Nous devons donc être explicites. Il est en outre indispensable d’indiquer “`paired = TRUE`” pour faire un test de Student sur données appariées. Omettre cet argument reviendrait à faire un test de Student sur données indépendantes, qui est moins puissant que le test sur données appariées. Ça n’est donc pas souhaitable avec les données dont nous disposons et ça pourrait même conduire à des conclusions fausses.

Ici, voilà la conclusion de ce test :

Le test de Student sur données appariées ne permet pas de montrer de changement d’immunocompétence suite à l’intégration de l’implant chirurgical de testostérone. On ne peut pas rejeter l’hypothèse nulle au seuil $\alpha = 0.05$ ($t = -1.27$, $ddl = 12$, $p = 0.223$). La moyenne des différences de densités optiques observées entre avant et après l’intervention chirurgicale vaut -0.056 (intervalle de confiance à 95% de cette différence : $[-0.152 ; 0.040]$)

Donc visiblement, une forte production de testostérone n’est pas significativement associée à une baisse de l’immunocompétence.

Notez bien que les résultats et leurs interprétations dépendent de l’ordre dans lequel les 2 séries de données sont indiquées dans la fonction `t.test()`. Pour vous en convaincre, regardez ce que donne cette commande :

```
t.test(Testo_large$After, Testo_large$Before, paired = TRUE)
```

Qu’est-ce qui change ? Et qu’est-ce qui reste inchangé ?

2.6.3.2 Deuxième syntaxe

```
# Méthode n°3 : avec la variable Diff, mu = 0, et le tableau au format large
t.test(Testo_large$Difference, mu = 0)
```

One Sample t-test

```
data: Testo_large$Difference
t = 1.2714, df = 12, p-value = 0.2277
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 -0.04007695  0.15238464
sample estimates:
mean of x
0.05615385
```

Comme expliqué plus haut, le test de Student sur données appariées est strictement équivalent à un test de Student à un échantillon pour lequel on compare la moyenne des différences individuelles à 0. Là encore, les résultats produits et leur interprétation sont identiques aux deux tests précédents. La seule différence concerne les signes puisque le premier test regardait la différence “Before - After” alors que ce test regarde la différence “After - Before” (que nous avons calculée manuellement).

À vous donc de choisir la syntaxe qui vous paraît la plus parlante ou celle que vous avez le plus de facilité à retenir et à interpréter.

2.7 L’alternative non paramétrique

Comme pour le test de Student à un échantillon, lorsque les conditions d’application du test de Student sur données appariées ne sont pas vérifiées (c’est-à-dire lorsque la différence entre les données appariées des deux séries ne suit pas une loi Normale), il faut utiliser un test non paramétrique équivalent.

Il s’agit là encore du **test de Wilcoxon des rangs signés** qui s’intéresse aux médianes. Les hypothèses nulles et alternatives sont les suivantes :

- H_0 : le changement **médian** de production d’anticorps après la pose chirurgicale de l’implant de testostérone est nul ($med_{Diff} = 0$).

- H_1 : le changement **médian** de production d'anticorps après la pose chirurgicale de l'implant de testostérone n'est pas nul ($med_{Diff} \neq 0$).

Comme pour le test de Student, 2 syntaxes sont possibles et strictement équivalentes. Il est important de ne pas oublier l'argument `paired = TRUE` afin de s'assurer que l'on réalise bien un test sur données appariées. Enfin, l'argument `conf.int = TRUE` doit être ajouté afin que la (pseudo-) médiane et son intervalle de confiance à 95% soient calculés et affichés.

```
wilcox.test(Testo_large$Before, Testo_large$After, paired = TRUE,
            conf.int = TRUE)
```

Wilcoxon signed rank exact test

```
data: Testo_large$Before and Testo_large$After
V = 30, p-value = 0.3054
alternative hypothesis: true location shift is not equal to 0
95.2 percent confidence interval:
 -0.145  0.040
sample estimates:
(pseudo)median
 -0.055
```

```
wilcox.test(Testo_large$Difference, mu = 0, conf.int = TRUE)
```

Wilcoxon signed rank exact test

```
data: Testo_large$Difference
V = 61, p-value = 0.3054
alternative hypothesis: true location is not equal to 0
95.2 percent confidence interval:
 -0.040  0.145
sample estimates:
(pseudo)median
  0.055
```

Ici, la conclusion de ce test est :

Le test de Wilcoxon des rangs signés n'a pas permis de montrer de changement d'immunocompétence suite à l'intégration de l'implant chirurgical de testostérone. On ne peut pas rejeter l'hypothèse nulle au seuil $\alpha = 0.05$ ($V = 61$, $p = 0.305$). La médiane des différences de densités optiques observées entre après et avant l'intervention chirurgicale vaut 0.055 (intervalle de confiance à 95% de cette différence : $[-0.040 ; 0.145]$).

2.8 Exercice d'application

Les autruches vivent dans des environnements chauds et elles sont donc fréquemment exposées au soleil durant de longues périodes. Dans des environnements similaires, les mammifères ont des mécanismes physiologiques leur permettant de réduire la température de leur cerveau par rapport à celle de leur corps. Une équipe de chercheurs (Fuller et al. 2003) a testé si les autruches pouvaient faire de même. La température du corps et du cerveau de 37 autruches a été enregistrée par une journée chaude typique. Les résultats, exprimés en degrés Celsius, figurent dans [le fichier Autruches.csv](#).

Importez ces données et faites-en l'analyse pour savoir s'il existe une différence de température moyenne entre le corps et le cerveau des autruches. Vos observations chez les autruches sont-elles conformes à ce qui est observé chez les mammifères dans un environnement similaire ? Comme toujours, vous commencerez par faire une analyse descriptive des données, sous forme numérique et graphique, avant de vous lancer dans les tests d'hypothèses.

3 Comparaison de moyennes : deux échantillons indépendants

3.1 Pré-requis

Comme pour chaque nouveau chapitre, je vous conseille de travailler dans un nouveau script que vous placerez dans votre répertoire de travail, et dans une nouvelle session de travail (Menu **Session** > **Restart R**). Inutile en revanche de créer un nouveau **Rproject** : vous pouvez tout à fait avoir plusieurs scripts dans le même répertoire de travail et pour un même **Rproject**. Comme toujours, consultez [le livre en ligne du semestre 3](#) si vous ne savez plus comment faire.

Si vous êtes dans une nouvelle session de travail (ou que vous avez quitté puis relancé **RStudio**), vous devrez penser à recharger en mémoire les packages utiles. Dans ce chapitre, vous aurez besoin d'utiliser :

- le **tidyverse** (Wickham 2023), qui comprend notamment le package **readr** (Wickham, Hester, et Bryan 2026), pour importer facilement des fichiers **.csv** au format **tibble**, le package **dplyr** (Wickham, François, et al. 2026), pour manipuler des tableaux, et le package **ggplot2** (Wickham, Chang, et al. 2026) pour les représentations graphiques.
- **readxl** (Wickham et Bryan 2025), pour importer facilement des fichiers Excel au format **tibble**.
- **skimr** (Waring et al. 2026), qui permet de calculer des résumés de données très informatifs.
- **car** (Fox, Weisberg, et Price 2026), qui permet d'effectuer le test de comparaison des variances de Levene.

- le package `palmerpenguins` (Horst, Hill, et Gorman 2022) pour accéder au jeu de données `penguins` que nous utiliserons pour les exercices d'application.

```
library(tidyverse)
library(readxl)
library(skimr)
library(car)
library(palmerpenguins)
```

Vous aurez également besoin des jeux de données suivants que vous pouvez dès maintenant télécharger dans votre répertoire de travail :

- [HommesFemmes.xls](#)
- [HornedLizards.csv](#)

Enfin, je spécifie ici une fois pour toutes le thème que j'utiliserai pour tous les graphiques de ce chapitre. Libre à vous de choisir un thème différent ou de vous contenter du thème proposé par défaut :

```
theme_set(theme_bw())
```

3.2 Contexte

On s'intéresse maintenant aux méthodes permettant de comparer la moyenne de deux groupes ou de deux traitements dans le cas d'échantillons indépendants. Au contraire de la situation décrite dans le Chapitre 2, dans ce type de design expérimental, les deux traitements sont appliqués à des échantillons indépendants issus de 2 groupes ou populations distincts. Chaque individu collecté, ou chaque unité expérimentale observée, ne fournit qu'une seule valeur, indépendante de toutes les autres.

Cette situation est extrêmement classique dans le domaine de l'écologie au sens large. Ainsi, par exemple, lorsque l'on souhaite comparer 2 sites, on réalise des prélèvements dans chacun des 2 sites. Chaque prélèvement ne fournit qu'une valeur pour l'un des 2 sites.

Ici, nous allons examiner une espèce intéressante, le **lézard cornu** *Phrynosoma mcallii*, qui possède une frange de piquants autour de la tête. Une équipe d'herpétologues (Young, Brodie, et Brodie 2004) a étudié la question suivante : des piquants plus longs autour de la tête protègent-ils le lézard cornu de son prédateur naturel, la **pie-grièche migratrice** *Lanius ludovicianus* ? Ce prédateur a en effet une particularité : il accroche ses proies mortes à des barbelés ou des branches pour les consommer plus tard. Les chercheurs ont donc mesuré la longueur des cornes de 30 lézards retrouvés morts et accrochés dans des arbres par la pie-grièche migratrice. Et en parallèle, ils ont mesuré les cornes de 154 individus vivants et en bonne santé choisis au hasard dans la population.

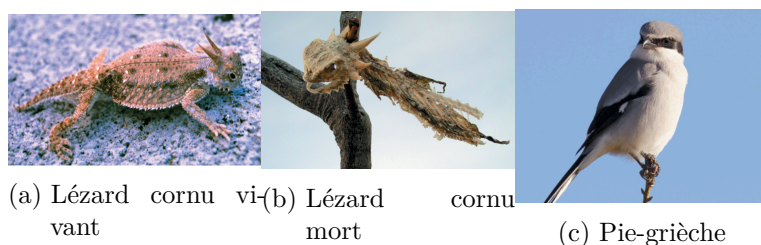


Figure 3.1: Le lézard cornu et son prédateur

Nous disposons donc de 2 groupes indépendants : chaque lézard n'a fourni qu'une valeur de longueur de cornes, et chaque lézard n'appartient qu'à un groupe, vivant ou mort. Nous sommes donc dans la situation typique de la comparaison de moyennes de 2 populations avec des données indépendantes. Avant de procéder aux tests, et comme toujours, nous allons commencer par importer et mettre en forme les données (si besoin), puis nous devrons explorer les données, à l'aide d'une part d'indices statistiques de position, de dispersion et d'incertitude et d'autre part de représentations graphiques pertinentes. Enfin, nous vérifierons les conditions d'application du test paramétrique de Student avant de réaliser ce test si les conditions d'application sont remplies, ou son équivalent non paramétrique si elles ne le sont pas.

3.3 Importation et mise en forme des données

Les données de cette étude sont stockées dans [le fichier HornedLizards.csv](#). Importez ces données dans un objet nommé `Lizard_raw` et examinez le tableau obtenu.

```
Lizard_raw

# A tibble: 185 x 2
  squamosalHornLength Survival
      <dbl> <chr>
1      25.2 living
2      26.9 living
3      26.6 living
4      25.6 living
5      25.7 living
6      25.9 living
7      27.3 living
8      25.1 living
9      30.3 living
10     25.6 living
# i 175 more rows
```

```
View(Lizard_raw)
```

On constate ici 3 choses :

1. la variable `Survival` devrait être un facteur.
2. le nom de la première colonne (`squamosalHornLength`), qui contient les mesures des longueurs de cornes, est bien trop long.
3. pour l'un des lézards vivants, la mesure de longueur des cornes est manquante. Nous allons donc retirer cet individu pour éviter les messages d'erreurs par la suite.

Nous pouvons facilement réaliser les 3 modifications d'un coup, et stocker le résultat dans un nouveau tableau `Lizard` :

```
Lizard <- Lizard_raw |>
  mutate(Survival = factor(Survival)) |>
```

```
rename(Horn_len = squamosalHornLength) |>
filter(!is.na(Horn_len))
```

Lizard

```
# A tibble: 184 x 2
  Horn_len Survival
  <dbl> <fct>
1    25.2 living
2    26.9 living
3    26.6 living
4    25.6 living
5    25.7 living
6    25.9 living
7    27.3 living
8    25.1 living
9    30.3 living
10   25.6 living
# i 174 more rows
```

Ce tableau est bien un tableau rangé, au format long : chaque colonne contient une unique variable (**Horn_len** : longueur des cornes, **Survival** : groupe de l'individu mesuré, vivant ou mort), et chaque ligne contient les informations d'un unique individu.

Ici, il ne serait pas correct de présenter les données au format large. Il nous faudrait en effet une colonne pour chaque groupe, lézard vivant et lézard mort, mais puisque les données de ces 2 groupes sont indépendantes, nous aurions 2 problèmes :

1. si le nombre d'individus n'est pas le même dans les 2 groupes, les deux colonnes n'auraient pas la même longueur. C'est impossible dans **RStudio**, et le logiciel remplirait donc la colonne la plus courte de NAs pour y remédier.
2. les lignes de cet hypothétique tableau large ne correspondraient plus à des observations uniques. Chaque ligne renseignerait en effet sur les mesures de 2 individus distincts, un vivant et un mort.

Lorsque vous disposez de données appartenant à des groupes indépendants, il faut donc toujours travailler avec un tableau rangé, nécessairement au format long.

3.4 Exploration statistique des données

Comme dans le Chapitre 2 sur les données appariées, les statistiques descriptives doivent ici être réalisées pour chaque groupe d'individus, et non tous groupes confondus. Ici, le plus simple est d'utiliser la fonction `skim()` sur les données groupées par niveau du facteur `Survival` (avec la fonction `group_by()`) :

```
Lizard |>
  group_by(Survival) |>
  skim()
```

```
-- Data Summary -----
                                Values
Name                           group_by(Lizard, Survival...)
Number of rows                  184
Number of columns                2

-----
Column type frequency:
  numeric                        1

-----
Group variables                Survival

-- Variable type: numeric -----
  skim_variable Survival n_missing complete_rate mean  sd  p0  p25  p50  p75
1 Horn_len      killed         0             1 22.0 2.71 15.2 21.1 22.2 23.8
2 Horn_len      living         0             1 24.3 2.63 13.1 23   24.6 26
  p100 hist
1 26.7
2 30.3
```

On constate ici qu'il n'y a pas de données manquantes (`n_missing` = 0 dans les deux groupes). La moyenne des longueurs de cornes est plus grande chez les lézards vivants ($\bar{x}_{living} = 24.3$ mm) que chez les lézards retrouvés morts ($\bar{x}_{killed} = 22.0$), de plus de 2 millimètres. On retrouve cette

tendance pour les médianes, ainsi que pour les premiers et troisièmes quartiles. En revanche, les écarts-types des 2 groupes sont proches, et celui du groupe `living` est très légèrement plus faible (0.08 mm) que celui du groupe `killed`.

Enfin, les histogrammes très simplifiés fournis laissent penser que les données de chaque groupe ne s'écartent pas trop fortement d'une courbe en cloche.

Outre ces informations sur les ordres de grandeurs observés dans chaque groupe de lézards pour les indices de **position** (moyennes, médianes et quartiles), et de **dispersion** (écarts-types et histogrammes), la fonction `skim()` ne fournit pas les effectifs observés dans chaque groupe. On sait qu'il y a en tout 184 individus, mais on ne sait pas comment ils se répartissent dans les 2 groupes de lézards. Pour le déterminer, on peut utiliser une fonction décrite plus tôt, la fonction `count()` :

```
Lizard |>
  count(Survival)
```

```
# A tibble: 2 x 2
  Survival      n
  <fct>    <int>
1 killed      30
2 living     154
```

On peut obtenir la même information avec la fonction `summarise()` et son argument `.by`, et la fonction `n()` :

```
Lizard |>
  summarize(Effectif = n(), .by = Survival)
```

```
# A tibble: 2 x 2
  Survival Effectif
  <fct>         <int>
1 living       154
2 killed        30
```

On constate ici que les tailles d'échantillons sont très différentes. C'est normal compte tenu de la difficulté de repérer

des individus morts dans la nature, et ce n'est pas gênant pour nos analyses puisque la taille des deux échantillons reste élevée.

Enfin, on peut calculer des indices d'incertitude. C'est d'autant plus important qu'il est difficile de se faire une idée de la signification d'une différence moyenne de longueur de cornes de 2 millimètres. Est-ce important ou négligeable ? Est-ce que ces estimations sont précises ou non ? Comme dans les chapitres précédents, nous allons calculer les intervalles de confiance à 95% de la moyenne de chaque groupe :

```
Lizard |>
  reframe(mean_cl_normal(Horn_len), .by = Survival)
```



```
# A tibble: 2 x 4
  Survival      y ymin ymax
  <fct>      <dbl> <dbl> <dbl>
1 living    24.3  23.9  24.7
2 killed    22.0  21.0  23.0
```

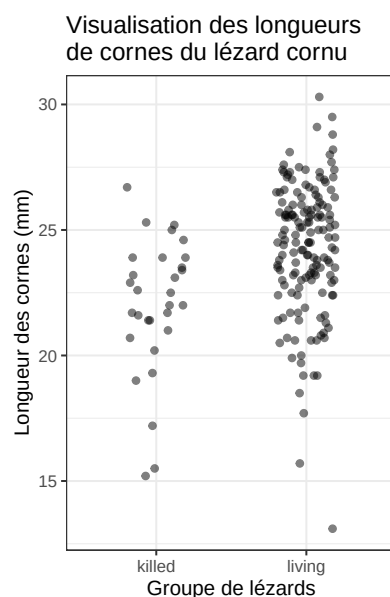
La colonne `y` nous présente à nouveau la moyenne de chaque groupe, la colonne `ymin` contient les bornes inférieures des intervalles de confiance à 95%, et la colonne `ymax` les bornes supérieures. On constate ici que les intervalles de confiance à 95% des longueurs de cornes des 2 groupes ne se chevauchent pas du tout : la borne inférieure du groupe `living` est au-dessus de la borne supérieure du groupe `killed`. Autrement dit, dans la population générale, la longueur moyenne des cornes chez les lézards vivants a de bonnes chances de se trouver dans l'intervalle [23.9 ; 24.7] millimètres, alors qu'elle a de bonnes chances de se trouver dans l'intervalle [21 ; 23] millimètres chez les lézards morts. La différence de moyennes entre ces 2 groupes vaut donc probablement entre 0.9 millimètres au moins, et 3.7 millimètres au plus. Le test statistique que nous ferons ensuite devrait donc confirmer que ces différences sont significatives, autrement dit, qu'elles ne sont pas liées au simple hasard de l'échantillonnage.

3.5 Exploration graphique des données

Comme toujours, nous pouvons réaliser plusieurs types de graphiques pour en apprendre plus sur **la distribution des données** dans les deux groupes. Si nous faisons un nuage de points, il est évidemment impossible ici de relier les points deux à deux. Non seulement cela n'aurait aucun sens puisque les échantillons sont indépendants, mais en outre, nous ne disposons pas du même nombre d'individus dans les 2 échantillons. Nous nous contenterons donc de faire un stripchart.

3.5.1 Avec un stripchart

```
Lizard |>
  ggplot(aes(x = Survival, y = Horn_len)) +
  geom_jitter(height = 0, width = 0.2, alpha = 0.5) +
  labs(x = "Groupe de lézards",
       y = "Longueur des cornes (mm)",
       title = "Visualisation des longueurs\nde cornes du lézard cornu")
```

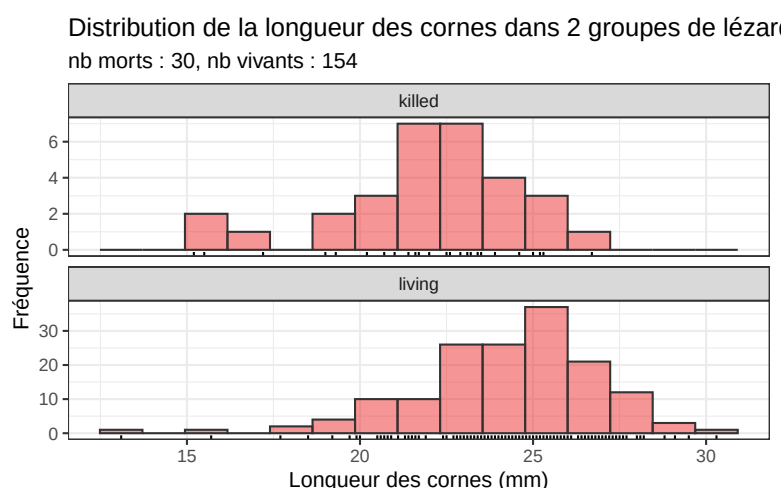


Ce premier graphique permet de visualiser très clairement les différences de tailles d'échantillons entre les deux groupes. Il permet également de voir que l'étendue des longueurs de

cornes est plus importante dans le groupe des individus vivants que dans celui des individus morts. En outre, le nuage de points des vivants semble être plus haut sur l'axe des y que celui des morts, confirmant les statistiques descriptives qui montraient des tailles de cornes en moyenne plus importantes dans le groupe des vivants.

3.5.2 Avec des histogrammes facettés

```
Lizard |>
  ggplot(aes(x = Horn_len)) +
  geom_histogram(bins = 15, fill = "firebrick2", color = "grey20", alpha = 0.5)+
  geom_rug() +
  facet_wrap(~Survival, ncol = 1, scales = "free_y") +
  labs(x = "Longueur des cornes (mm)",
       y = "Fréquence",
       title = "Distribution de la longueur des cornes dans 2 groupes de lézards cornus",
       subtitle = "nb morts : 30, nb vivants : 154")
```

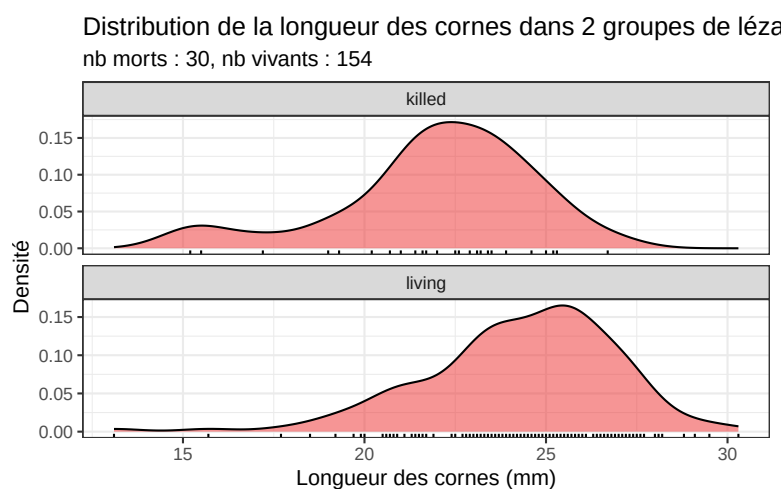


Notez ici l'utilisation de l'argument `scales = "free_y"` dans la fonction `facet_wrap()`. Cet argument permet de ne pas imposer la même échelle pour l'axe des ordonnées des 2 graphiques. Ce choix est ici pertinent puisque les effectifs des 2 groupes sont très différents. Faites un essai sans cet argument pour voir la différence. Il est en revanche important de conserver le même axe des x afin de faciliter la comparaison des 2 groupes.

Cette visualisation nous montre que les données doivent suivre à peu près une distribution Normale dans les 2 groupes, et que globalement la longueur des cornes semble légèrement plus élevée dans le groupe des vivants (avec un mode autour de 25-26 mm) que dans le groupe des morts (avec un mode autour de 23-24 mm). L'étendue des données semble légèrement plus grande dans le groupe des vivants, mais cela n'est peut-être dû qu'à la différence marquée des tailles d'échantillons.

3.5.3 Avec des diagrammes de densité facettés

```
Lizard |>
  ggplot(aes(x = Horn_len)) +
  geom_density(fill = "firebrick2", alpha = 0.5) +
  geom_rug() +
  facet_wrap(~Survival, ncol = 1, scales = "free_y") +
  labs(x = "Longueur des cornes (mm)",
       y = "Densité",
       title = "Distribution de la longueur des cornes dans 2 groupes de lézards cornus",
       subtitle = "nb morts : 30, nb vivants : 154")
```



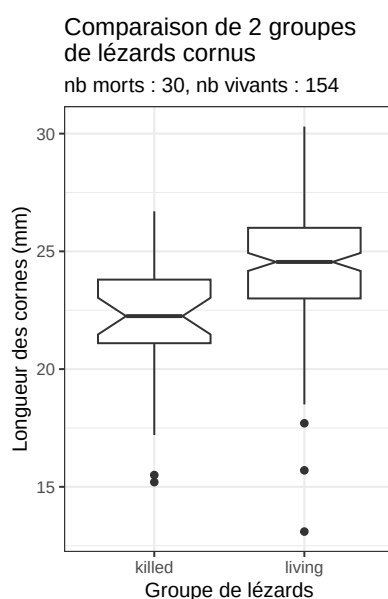
Les diagrammes de densité ressemblent ici beaucoup aux histogrammes. C'est normal car la taille des échantillons est importante (30 et 154 pour les groupes `killed` et `living` respectivement). C'était moins vrai dans les chapitres précédents car les tailles d'échantillons étaient plus faibles, et la forme des histogrammes dépendait alors beaucoup du nombre

de classes que l'on choisissait de représenter. Avec des échantillons de grande taille ($n \geq 30$), c'est moins problématique.

En général, il est donc inutile de faire à la fois les histogrammes et les diagrammes de densité. Choisissez l'un ou l'autre selon vos préférences et la situation.

3.5.4 Avec des boîtes à moustaches

```
Lizard |>
  ggplot(aes(x = Survival, y = Horn_len)) +
  geom_boxplot(notch = TRUE) +
  labs(x = "Groupe de lézards",
       y = "Longueur des cornes (mm)",
       title = "Comparaison de 2 groupes\nde lézards cornus",
       subtitle = "nb morts : 30, nb vivants : 154")
```

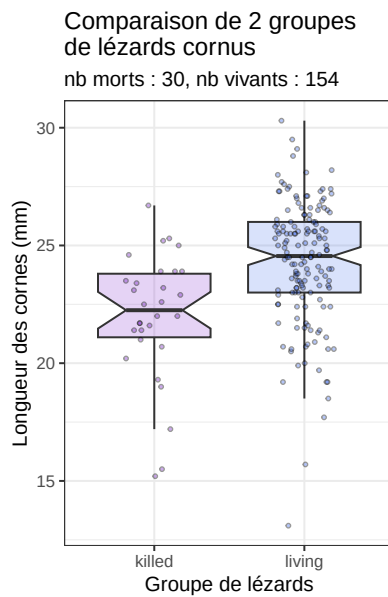


Nous visualisons ici encore plus clairement que sur les histogrammes le fait que les longueurs de cornes des individus vivants sont légèrement plus longues que celles des individus morts. D'ailleurs, puisque les intervalles de confiance à 95% des médianes des 2 groupes (les encoches) ne se chevauchent pas, un test de comparaison des moyennes devrait logiquement conclure à une différence significative en faveur des individus vivants. On peut également noter que la largeur de

l'encoche pour les individus morts est plus importante que celle des vivants. Cela traduit une incertitude plus grande autour de la médiane estimée dans le groupe des individus morts. C'est tout à fait logique compte tenu des effectifs plus faibles dans ce groupe.

Enfin, il est tout à fait possible de représenter sur le même graphique les boîtes à moustaches et les données brutes sous forme de stripchart. On a ainsi à la fois (i) une visualisation simplifiée de la position et de la dispersion des données avec les boîtes à moustaches, et (ii) accès à l'ensemble des données brutes, ce qui permet parfois de voir des structures invisibles sur les boîtes à moustaches (regroupement de points par exemple). Afin de ne pas dupliquer les valeurs les plus extrêmes du jeu de données, nous indiquerons à `geom_boxplot()` de ne pas afficher les outliers sur le graphique : tous les points seront en effet déjà affichés par `geom_jitter()` :

```
Lizard |>
  ggplot(aes(x = Survival, y = Horn_len, fill = Survival)) +
  geom_boxplot(notch = TRUE, color = "grey20", alpha = 0.2,
               outlier.color = NA, show.legend = FALSE) +
  geom_jitter(height = 0, width = 0.2, alpha = 0.4, shape = 21,
              show.legend = FALSE, size = 0.8) +
  labs(x = "Groupe de lézards",
       y = "Longueur des cornes (mm)",
       title = "Comparaison de 2 groupes\nde lézards cornus",
       subtitle = "nb morts : 30, nb vivants : 154") +
  scale_fill_manual(values = c("purple3", "royalblue2"))
```



3.6 Le test paramétrique

Le test paramétrique le plus puissant que nous puissions faire pour comparer la moyenne de 2 populations est le test de Student. Ce test étant paramétrique, nous devons nous assurer que ses conditions d'application sont vérifiées avant de pouvoir le réaliser.

3.6.1 Conditions d'application

Les conditions d'application de ce test sont au nombre de 3 :

1. Chacun des deux échantillons est issu d'un échantillonnage aléatoire de la population générale. Comme toujours, en l'absence d'indication contraire, on considère que cette condition est toujours vérifiée.
2. La variable numérique étudiée est distribuée normalement dans les deux populations. Il nous faudra donc faire deux tests de Shapiro-Wilk, un pour chaque échantillon.
3. La variance de la variable numérique étudiée est la même dans les deux populations. C'est ce que l'on appelle l'homoscédasticité.

En réalité, le test du t de Student sur deux échantillons indépendants est assez robuste face au non respect de cette troisième condition d'application. Cela signifie que si cette troisième condition d'application n'est pas strictement vérifiée, les résultats du test peuvent malgré tout rester valides. Lorsque les 2 échantillons comparés ont des tailles supérieures ou égales à 30, ce test fonctionne bien même si l'écart-type d'un groupe est jusqu'à 3 fois supérieur ou inférieur à l'écart-type du second groupe, à condition que la taille des 2 échantillons soit proche (ce qui n'est pas le cas ici !). En revanche, si les écarts-types diffèrent de plus d'un facteur 3, ou si les tailles d'échantillons sont très différentes, le test du t de Student ne devrait pas être utilisé. De même, si la taille des échantillons est inférieure à 30 et que les variances ne sont pas homogènes, ce test ne devrait pas être réalisé. En conclusion, les résultats du test du t de Student à deux échantillons indépendants peuvent rester valides si la troisième condition d'homoscédasticité n'est pas respectée, mais dans certains cas seulement.

Le test du t de Student sur deux échantillons indépendants est également assez robuste face à des écarts mineurs à la distribution Normale, tant que la forme des deux distributions comparées reste similaire et unimodale. En outre, la robustesse de ce test augmente avec la taille des échantillons.

Robustesse

La robustesse d'un test statistique est sa capacité à rester valide même lorsque certaines de ses conditions d'application ne sont pas parfaitement respectées. Plus un test est robuste, plus il est capable de supporter des "entorses" importantes à ses conditions d'application.

Au final, avec un peu d'habitude, même lorsque les conditions d'application ne sont pas toutes vérifiées, on peut parfois passer outre. Mais à ce stade, on préfère s'en tenir à des choses plus simples et claires.

La procédure à suivre

1. Faites un test de Normalité pour chacune des deux séries de données. Si elles suivent la loi Normale

- toutes les deux, passez au point 2. Sinon, rendez-vous au point 4.
2. Faites un test d'homoscédasticité (homogénéité des variances). Si les variances sont homogènes, passez au point 3. Sinon, rendez-vous au point 5.
 3. Faites un test de comparaison des moyennes paramétrique : le test de Student. Examinez la p -value pour conclure, et rendez-vous au point 6.
 4. Faites un test de comparaison des moyennes non paramétrique : le test de Wilcoxon de la somme des rangs. Examinez la p -value pour conclure, et rendez-vous au point 6.
 5. Faites un test de comparaison des moyennes non paramétrique : le test t de Welch. Examinez la p -value pour conclure, et rendez-vous au point 6.
 6. Si la p -value est supérieure à α , on ne peut pas rejeter l'hypothèse nulle et on conclut alors à une absence de différence significative entre les 2 populations. À l'inverse, si la p -value est inférieure ou égale à α , on rejette l'hypothèse nulle et on valide l'hypothèse alternative. Les deux populations ont des moyennes significativement différentes, et pour savoir laquelle est supérieure ou inférieure à l'autre, on revient aux estimations des moyennes et des intervalles de confiance à 95%, calculés dans la partie consacrée aux statistiques descriptives.

3.6.1.1 Normalité des données

Nous commençons donc par tester la Normalité des 2 populations dont sont issus les échantillons, c'est le point 1 de la procédure détaillée ci-dessus. Pour les individus morts, les hypothèses sont les suivantes :

- H_0 : la taille des cornes suit une distribution Normale dans la population des lézards cornus morts.
- H_1 : la taille des cornes ne suit pas une distribution Normale dans la population des lézards cornus morts.

```
Lizard |>  
  filter(Survival == "killed") |>
```

```
pull(Horn_len) |>
shapiro.test()
```

Shapiro-Wilk normality test

```
data: pull(filter(Lizard, Survival == "killed"), Horn_len)
W = 0.93452, p-value = 0.06482
```

La p -value est supérieure à $\alpha = 0.05$, donc on ne peut pas rejeter l'hypothèse nulle de normalité pour la taille des cornes de la population des lézards cornus morts (test de Shapiro-Wilk, $W = 0.93$, $p = 0.065$).

Pour les individus vivants :

- H_0 : la taille des cornes suit une distribution Normale dans la population des lézards cornus vivants.
- H_1 : la taille des cornes ne suit pas une distribution Normale dans la population des lézards cornus vivants.

```
Lizard |>
  filter(Survival == "living") |>
  pull(Horn_len) |>
  shapiro.test()
```

Shapiro-Wilk normality test

```
data: pull(filter(Lizard, Survival == "living"), Horn_len)
W = 0.96055, p-value = 0.0002234
```

La p -value est inférieure à $\alpha = 0.05$, donc on rejette l'hypothèse nulle de normalité pour la taille des cornes de la population des lézards cornus vivants (test de Shapiro-Wilk, $W = 0.96$, $p < 0.001$).

L'une des 2 séries de données ne suit pas la loi Normale, nous sommes donc censés passer directement au point 4 de la procédure.

Toutefois, si l'on examine les histogrammes (Section 3.5.2) ou les diagrammes de densité (Section 3.5.3) des 2 échantillons, on constate que la forme des distributions des 2 séries de données est très proche. Pour les 2 échantillons, la distribution est en effet unimodale, avec une asymétrie gauche assez marquée (une longue queue de distribution du côté gauche). La forme des distributions étant similaire (on parle bien de la forme des histogrammes et non de la position du pic), et les histogrammes étant proches de la forme typique d'une courbe en cloche, le test de Student devrait rester valide car il est robuste dans cette situation. Ici, pour l'exemple, on va donc passer au point 2 de la procédure. Notez toutefois que passer directement au point 4 de la procédure serait tout à fait correct : on ne pourrait rien vous reprocher si vous passez directement au test non paramétrique de comparaison des moyennes lorsque vous constatez que l'une des 2 séries de données ne suit pas la distribution Normale.

3.6.1.2 Homogénéité des variances

Le test le plus simple pour comparer la variance de 2 populations est le test F :

- H_0 : la variance des 2 populations est égale, leur ratio vaut 1 $\left(\frac{\sigma_{killed}^2}{\sigma_{living}^2} = 1\right)$.
- H_1 : la variance des 2 populations est différente, leur ratio ne vaut pas 1 $\left(\frac{\sigma_{killed}^2}{\sigma_{living}^2} \neq 1\right)$.

```
var.test(Horn_len ~ Survival, data = Lizard)
```

F test to compare two variances

```
data: Horn_len by Survival
F = 1.0607, num df = 29, denom df = 153, p-value = 0.7859
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.6339331 1.9831398
sample estimates:
ratio of variances
 1.060711
```

Ici, le ratio des variances (la variance des individus morts divisée par la variance des individus vivants) est très proche de 1 ($F = 1.06$, IC 95% : $[0.63 ; 1.98]$). Le test F nous montre qu'il est impossible de rejeter H_0 : au seuil $\alpha = 0.05$, le ratio des variances n'est pas significativement différent de 1 (ddl = 29 et 153, $p = 0.79$), les variances sont homogènes.

Le test de Bartlett est un autre test qui permet de comparer la variance de plusieurs populations (2 ou plus). Lorsque le nombre de populations est égal à 2 (comme ici), ce test est absolument équivalent au test F ci-dessus.

- H_0 : toutes les populations ont même variance ($\sigma_A^2 = \sigma_B^2 = \sigma_C^2 = \dots = \sigma_N^2$).
- H_1 : au moins une population a une variance différente des autres.

```
bartlett.test(Horn_len ~ Survival, data = Lizard)
```

Bartlett test of homogeneity of variances

data: Horn_len by Survival

Bartlett's K-squared = 0.042411, df = 1, p-value = 0.8368

Enfin, le test de Levene (attention, le package `car` doit être chargé) devrait être préféré la plupart du temps. Comme le test de Bartlett, il permet de comparer la variance de plusieurs populations, mais il est plus robuste vis-à-vis de la non-normalité des données.

- H_0 : toutes les populations ont même variance ($\sigma_A^2 = \sigma_B^2 = \sigma_C^2 = \dots = \sigma_N^2$).
- H_1 : au moins une population a une variance différente des autres.

```
# Le test de Levene fait partie du package car. Il doit être chargé en mémoire
# library(car)
leveneTest(Horn_len ~ Survival, data = Lizard)
```

```
Levene's Test for Homogeneity of Variance (center = median)
      Df F value Pr(>F)
group   1  0.0035  0.953
      182
```

Ici encore, les conclusions sont les mêmes :

Il est impossible de rejeter l'hypothèse nulle d'homogénéité des variances au seuil $\alpha = 0.05$ (test de Levene, $F = 0.004$, ddl = 1, $p = 0.953$).

À vous de choisir lequel de ces 3 tests vous souhaitez réaliser : il est évident qu'on ne fait jamais les 3 !

Ici, puisque l'homoscédasticité est vérifiée, on passe au point 3 de la procédure.

3.6.2 Réalisation du test et interprétation

Puisque la taille des cornes du lézard cornu suit approximativement la même distribution “presque Normale” dans les 2 populations (lézards morts et vivants) et que ces 2 populations ont des variances homogènes, on peut réaliser le test du t de Student sur deux échantillons indépendants.

- H_0 : la moyenne des 2 populations est égale, leur différence vaut 0 ($\mu_{killed} - \mu_{living} = 0$).
- H_1 : la moyenne des 2 populations est différente, leur différence ne vaut pas 0 ($\mu_{killed} - \mu_{living} \neq 0$).

```
t.test(Horn_len ~ Survival, data = Lizard, var.equal = TRUE)
```

Two Sample t-test

```
data: Horn_len by Survival
t = -4.3494, df = 182, p-value = 2.27e-05
alternative hypothesis: true difference in means between group killed and group living is not equal to 0
95 percent confidence interval:
 -3.335402 -1.253602
sample estimates:
mean in group killed mean in group living
      21.98667          24.28117
```

Notez bien la syntaxe :

- Nous utilisons ici la syntaxe du type “formule” faisant appel au symbole “~” (Longueur des cornes en fonction de la Survie) et à l’argument “data =”.
- L’argument “paired = TRUE” a disparu puisque nous avons ici deux échantillons indépendants
- L’argument “var.equal = TRUE” doit obligatoirement être spécifié : nous nous sommes assurés que l’homogénéité des variances était vérifiée. Il faut donc l’indiquer afin que le test de Student classique soit réalisé. Si on omet de le spécifier, c’est un autre test qui est réalisé (voir plus bas).

Au seuil α de 5%, on rejette l’hypothèse nulle d’égalité des moyennes de la longueur des cornes entre lézards vivants et morts (test t de Student sur deux échantillons indépendants, $t = -4.35$, ddl = 182, $p < 0.001$). Les lézards morts ont en moyenne des cornes plus courtes ($\hat{\mu}_{killed} = 21.99$ millimètres) que les lézards vivants ($\hat{\mu}_{living} = 24.28$ millimètres). La gamme des valeurs les plus probables pour la différence de moyenne entre les deux populations est fournie par l’intervalle de confiance à 95% de la différence de moyennes : [-3.34 ; -1.25].

Ce test confirme donc bien l’impression des chercheurs : les lézards principalement pris pour cibles par les pies-grièches migratrices ont des cornes en moyenne plus courtes (probablement entre 1.25 et 3.34 millimètres de moins) que les lézards de la population générale. Avoir des cornes plus longues semble donc protéger les lézards de la prédation, du moins dans une certaine mesure.

Notez bien que l’intervalle de confiance à 95% qui est fourni avec les résultats du test est l’intervalle de confiance à 95% de la différence de moyenne entre les 2 groupes. Cet intervalle nous donne donc une idée de la **magnitude de l’effet**, de son ampleur. En effet, dire que les lézards morts ont des cornes en moyenne plus courtes est intéressant, mais cela n’aura pas la même portée si leurs cornes sont plus courtes de 0.02 millimètres ou si elles sont plus courtes de 5 millimètres. Un test statistique permet de rejeter ou non une hypothèse nulle, mais c’est bien l’estimation (la moyenne de chaque groupe et

l'intervalle de confiance à 95% de la différence) qui nous dit ce qu'on doit penser des résultats, et de leur pertinence (écologique, biologique, physiologique, comportementale, etc.).

3.7 L'alternative non paramétrique

Si les conditions d'application du test de Student ne sont pas vérifiées (c'est bien le cas ici puisque la longueur des cornes ne suit pas une distribution Normale dans la population des lézards vivants), notre procédure nous conduit à l'étape 4 : nous devons utiliser un équivalent non paramétrique au test de Student. C'est le cas du **test de Wilcoxon sur la somme des rangs** (également appelé test de Mann-Whitney). Comme pour tous les tests de Wilcoxon, la comparaison porte alors non plus sur les moyennes mais sur les médianes.

- H_0 : la médiane des 2 populations est égale, leur différence vaut 0 ($med_{killed} - med_{living} = 0$).
- H_1 : la médiane des 2 populations est différente, leur différence ne vaut pas 0 ($med_{killed} - med_{living} \neq 0$).

```
wilcox.test(Horn_len ~ Survival, data = Lizard, conf.int = TRUE)
```

Wilcoxon rank sum test with continuity correction

```
data: Horn_len by Survival
W = 1181.5, p-value = 2.366e-05
alternative hypothesis: true location shift is not equal to 0
95 percent confidence interval:
 -3.200076 -1.300067
sample estimates:
difference in location
 -2.200031
```

L'argument `var.equal = TRUE` n'existe pas pour ce test, puisque c'est justement un test non paramétrique qui ne requiert pas l'homogénéité des variances. En revanche, comme pour tous les autres tests de Wilcoxon que nous avons réalisés jusqu'ici, l'argument `conf.int = TRUE` permet

d'afficher les estimateurs pertinents, ici, la différence de médiane entre les 2 populations et l'intervalle de confiance à 95% de cette différence de médiane.

La conclusion est ici la même que pour le test de Student : puisque la p -value est très inférieure à α , on rejette l'hypothèse nulle : les médianes sont bel et bien différentes.

3.8 L'autre alternative non paramétrique

Enfin, dans le cas où la variable étudiée suit la loi Normale dans les deux populations mais qu'elle n'a pas la même variance dans les deux populations (donc si vous arrivez au point 5 de la procédure décrite plus haut), il est toujours possible de réaliser un test de Wilcoxon, il est préférable de réaliser un test de Student modifié : le **test approché du t de Welch**. Ce test est moins puissant que le test de Student classique, mais il reste plus puissant que le test de Wilcoxon, et surtout, il permet de comparer les moyennes et non les médianes.

- H_0 : la moyenne des 2 populations est égale, leur différence vaut 0 ($\mu_{killed} - \mu_{living} = 0$).
- H_1 : la moyenne des 2 populations est différente, leur différence ne vaut pas 0 ($\mu_{killed} - \mu_{living} \neq 0$).

```
t.test(Horn_len ~ Survival, data = Lizard)
```

Welch Two Sample t-test

```
data: Horn_len by Survival
t = -4.2634, df = 40.372, p-value = 0.0001178
alternative hypothesis: true difference in means between group killed and group living is not equal to 0
95 percent confidence interval:
-3.381912 -1.207092
sample estimates:
mean in group killed mean in group living
21.98667 24.28117
```

La seule différence par rapport à la syntaxe du test t de Student paramétrique est la suppression de l'argument `var.equal = TRUE`. Attention donc, à bien utiliser la syntaxe correcte. Le test du t de Welch ne devrait être réalisé que lorsque la Normalité est vérifiée pour les 2 populations, mais pas l'homoscédasticité. Par rapport au test de Student classique, on constate que le nombre de degrés de liberté est très différent, et donc la p -value également. Les bornes de l'intervalle de confiance à 95% de la différence de moyenne sont différentes également puisque leur calcul a été fait en supposant que les 2 populations n'avaient pas même variance.

3.9 Exercices d'application

3.9.1 La taille des hommes et des femmes

On s'intéresse à la différence de taille supposée entre hommes et femmes. Le fichier `HommesFemmes.xls` contient les tailles en centimètres de 38 hommes et 43 femmes choisis au hasard parmi les étudiants de première année à La Rochelle Université. Importez, mettez en forme et analysez ces données. Vous prendrez soin de retirer les éventuelles valeurs manquantes, vous prendrez le temps d'examiner les données à l'aide de statistiques descriptives et de représentations graphiques adaptées, puis vous tenterez de répondre à la question suivante : les hommes et les femmes inscrits en première année à La Rochelle Université ont-ils la même taille ? Si non, caractérisez cette différence de taille.

3.9.2 La longueur du bec des manchots Adélie

Dans le jeu de données `penguins` du package `palmerpenguins`, récupérez les lignes du tableau qui correspondent aux manchots Adélie, et comparez la longueur des becs entre mâles et femelles. Comme toujours, avant de vous lancer dans les tests, vous prendrez soin de retirer les éventuelles valeurs manquantes, et vous prendrez le temps d'examiner les données à l'aide de statistiques descriptives et de représentations graphiques adaptées. Faites l'effort d'expliquer votre démarche, de préciser les hypothèses nulles et alternatives

de chaque test, et de rédiger l'interprétation que vous faites de chaque résultat.

3.10 Tests bilatéraux et unilatéraux

3.10.1 Principe

Jusqu'à maintenant, tous les tests que nous avons réalisés sont des **tests bilatéraux**. Pour chaque test, l'hypothèse nulle est imposée. En revanche, pour certains tests, l'hypothèse alternative est à choisir (et à spécifier) par l'utilisateur parmi 3 possibilités :

- Une hypothèse bilatérale. C'est celle qui est utilisée par défaut si l'utilisateur ne précise rien.
- Deux hypothèses unilatérales possibles, qui doivent être spécifiées explicitement par l'utilisateur.

Les tests unilatéraux peuvent concerner tous les tests pour lesquels les hypothèses sont de la forme suivante :

- H_0 : la valeur d'un paramètre de la population est égale à k (k peut être une valeur fixe, arbitraire, choisie par l'utilisateur, ou la valeur d'un paramètre d'une autre population).
- H_1 : la valeur d'un paramètre de la population **n'est pas égale à k** .

En réalité, si nous remplaçons l'hypothèse H_1 par :

- H_1 : la valeur d'un paramètre de la population **est supérieure à k** .

ou par :

- H_1 : la valeur d'un paramètre de la population **est inférieure à k** .

nous réalisons un test unilatéral.

Dans **RStudio**, la syntaxe permettant de spécifier l'hypothèse alternative que nous souhaitons utiliser est toujours la même. Il faut préciser, au moment de faire le test l'argument suivant :

- `alternative = "two.sided"` : pour faire un test bilatéral. Si on ne le fait pas explicitement, c'est de toutes façons cette valeur qui est utilisée par défaut.
- `alternative = "greater"` : pour choisir l'hypothèse unilatérale ">".
- `alternative = "less"` : pour choisir l'hypothèse unilatérale "<".

Attention : le choix d'utiliser "greater" ou "less" dépend donc de l'ordre dans lequel les échantillons sont spécifiés. Cette syntaxe est valable pour tous les tests de Student vus jusqu'ici (un échantillon, deux échantillons appariés, deux échantillons indépendants) et pour leurs alternatives non paramétriques (test de Wilcoxon des rangs signés, test de Wilcoxon de la somme des rangs, test du t de Welch).

! Attention

L'utilisation de tests unilatéraux doit être réservée exclusivement aux situations pour lesquelles le choix de l'hypothèse unilatérale est possible à justifier par un mécanisme quelconque (biologique, physiologique, comportemental, écologique, génétique, évolutif, biochimique, etc.). Observer que l'un des échantillons a une moyenne plus grande ou plus faible qu'un autre lors de la phase des statistiques descriptives des données **n'est pas du tout une raison suffisante**. Il faut pouvoir justifier le choix de l'hypothèse alternative par une explication valable. D'ailleurs, si on veut être rigoureux, il faudrait toujours formuler les hypothèses que l'on souhaite tester **avant** de mettre en place le protocole expérimental et **avant** d'acquérir les données.

Pourquoi s'embêter avec les tests unilatéraux puisqu'il est si rare qu'on ait le droit de les faire ? Tout simplement parce que toutes choses étant égales par ailleurs, un test unilatéral est toujours plus puissant (parfois, beaucoup plus puissant) qu'un test bilatéral. Or, la puissance est quelque chose qu'on cherche à maximiser (voir sec-puiss). Lorsqu'il est pertinent de réaliser un test unilatéral, on doit donc toujours le faire.

Reprenons l'un des exemples examinés précédemment pour mieux comprendre comment tout cela fonctionne.

3.10.2 Un exemple pas à pas

Reprenons l'exemple des lézards cornus. L'étude a été réalisée parce que les chercheurs supposaient que la longueur des cornes des lézards était susceptible de leur fournir une protection face à la prédation. Autrement dit, les chercheurs supposaient que des cornes *plus longues* devaient fournir une meilleure protection vis-à-vis de la prédation. Ainsi, les lézards morts devaient avoir des cornes moins longues en moyenne que les lézards vivants, simplement parce que porter des cornes courtes expose plus fortement les individus à la prédation. Nous avons donc une bonne raison "écologique/évolutive" de considérer un test unilatéral (la susceptibilité face à la prédation qui a entraîné une pression de sélection sur la longueur des cornes des lézards), avant même de collecter les données.

Lorsque nous avons examiné cette question, nous avons fait le test du t de Student sur échantillons indépendants de la façon suivante :

```
t.test(Horn_len ~ Survival, data = Lizard, var.equal = TRUE)
```

Two Sample t-test

```
data: Horn_len by Survival
t = -4.3494, df = 182, p-value = 2.27e-05
alternative hypothesis: true difference in means between group killed and group living is not equal to 0
95 percent confidence interval:
 -3.335402 -1.253602
sample estimates:
mean in group killed mean in group living
      21.98667         24.28117
```

Comme l'indiquent les résultats fournis, l'hypothèse alternative utilisée pour faire le test est : "La vraie différence de moyenne n'est pas égale à 0". Autrement dit, nous avons fait un test bilatéral avec les hypothèses suivantes :

- H_0 : la moyenne des 2 populations est égale, leur différence vaut 0 ($\mu_{killed} - \mu_{living} = 0$).

- H_1 : la moyenne des 2 populations est différente, leur différence ne vaut pas 0 ($\mu_{killed} - \mu_{living} \neq 0$).

Ce test est donc rigoureusement équivalent à celui-ci :

```
t.test(Horn_len ~ Survival, data = Lizard, var.equal = TRUE,
       alternative = "two.sided")
```

Two Sample t-test

```
data: Horn_len by Survival
t = -4.3494, df = 182, p-value = 2.27e-05
alternative hypothesis: true difference in means between group killed and group living is not equal to 0
95 percent confidence interval:
-3.335402 -1.253602
sample estimates:
mean in group killed mean in group living
      21.98667          24.28117
```

Ici, nous souhaitons en fait réaliser un **test unilatéral** avec les hypothèses suivantes :

- H_0 : la moyenne de longueur des cornes de la population des lézards morts est égale à celle des lézards vivants. Leur différence vaut 0 ($\mu_{killed} - \mu_{living} = 0$).
- H_1 : la moyenne de longueur des cornes de la population des lézards morts est **inférieure** à celle des lézards vivants. Leur différence est inférieure à 0 ($\mu_{killed} - \mu_{living} < 0$).

```
t.test(Horn_len ~ Survival, data = Lizard, var.equal = TRUE,
       alternative = "less")
```

Two Sample t-test

```
data: Horn_len by Survival
t = -4.3494, df = 182, p-value = 1.135e-05
alternative hypothesis: true difference in means between group killed and group living is less than 0
95 percent confidence interval:
-Inf -1.422321
```

```
sample estimates:
mean in group killed mean in group living
      21.98667          24.28117
```

Puisque la p -value de ce test est inférieure à $\alpha = 0.05$, on rejette l'hypothèse nulle de l'égalité des moyennes. On valide donc l'hypothèse alternative : les lézards cornus morts ont en moyenne des cornes plus courtes que les lézards vivants. Cette différence de longueur de cornes est en faveur des lézards vivants et vaut très probablement au moins 1.4 millimètres (c'est l'intervalle de confiance à 95% de la différence de moyennes qui nous le dit).

Dernière chose importante : il ne faut pas se tromper dans le choix de l'hypothèse alternative. En effet, nous aurions pu tenter de tester exactement la même chose en formulant les hypothèses suivantes :

- H_0 : la moyenne de longueur des cornes de la population des lézards **vivants** est égale à celle des lézards **morts**. Leur différence vaut 0 ($\mu_{living} - \mu_{killed} = 0$).
- H_1 : la moyenne de longueur des cornes de la population des lézards **vivants** est **supérieure** à celle des lézards **morts**. Leur différence est **supérieure** à 0 ($\mu_{living} - \mu_{killed} > 0$).

Ce test est normalement exactement le même que précédemment. Toutefois, si on essaie de le réaliser, on rencontre un problème :

```
t.test(Horn_len ~ Survival, data = Lizard, var.equal = TRUE,
       alternative = "greater")
```

Two Sample t-test

```
data: Horn_len by Survival
t = -4.3494, df = 182, p-value = 1
alternative hypothesis: true difference in means between group killed and group living is greater
95 percent confidence interval:
 -3.166684      Inf
sample estimates:
mean in group killed mean in group living
      21.98667          24.28117
```

Ici, la p -value est très supérieure à α puisqu'elle vaut 1. Une p -value de 1 devrait toujours attirer votre attention. La conclusion devrait donc être que l'on ne peut pas rejeter H_0 : les lézards morts et vivants ont en moyenne des cornes de même longueur. Nous savons pourtant que c'est faux.

Le problème est ici lié à l'ordre des catégories "vivant" ou "mort" dans le facteur **Survival** du tableau **Lizard**. Les dernières lignes des tests que nous venons de faire indiquent la moyenne de chaque groupe, mais le groupe "killed" apparaît toujours avant le groupe "living". C'est l'ordre des niveaux dans le facteur **Survival** qui doit dicter la syntaxe appropriée :

```
Lizard$Survival
```

```
[1] living living living living living living living living living living
[11] living living living living living living living living living living
[21] living living living living living living living living living living
[31] living living living living living living living living living living
[41] living living living living living living living living living living
[51] living living living living living living living living living living
[61] living living living living living living living living living living
[71] living living living living living living living living living living
[81] living living living living living living living living living living
[91] living living living living living living living living living living
[101] living living living living living living living living living living
[111] living living living living living living living living living living
[121] living living living living living living living living living living
[131] living living living living living living living living living living
[141] living living living living living living living living living living
[151] living living living living killed killed killed killed killed killed
[161] killed killed killed killed killed killed killed killed killed killed
[171] killed killed killed killed killed killed killed killed killed killed
[181] killed killed killed killed
Levels: killed living
```

Par défaut, dans **RStudio**, les niveaux d'un facteur sont classés par ordre alphabétique sauf si on spécifie manuellement un ordre différent. Ici, le niveau "killed" est donc le premier niveau du facteur, et "living" le second. Attention, on parle bien ici des niveaux, ou modalités, et non des données elles-mêmes. Ici, le premier lézard mesuré appartient à la catégorie

`living`. Ça n'est pas ça qui est important : c'est bien l'ordre des niveaux qui compte, et on peut le voir tout en bas, après `Levels:` Lorsque l'on réalise un test de Student avec ces données (ou un test de Wilcoxon d'ailleurs), la différence de moyenne qui est examinée par le test est donc “moyenne des `killed` - moyenne des `living`”. Lorsque nous avons tapé ceci :

```
t.test(Horn_len ~ Survival, data = Lizard, var.equal = TRUE,
       alternative = "greater")
```

nous avons donc en réalité posé les hypothèses suivantes :

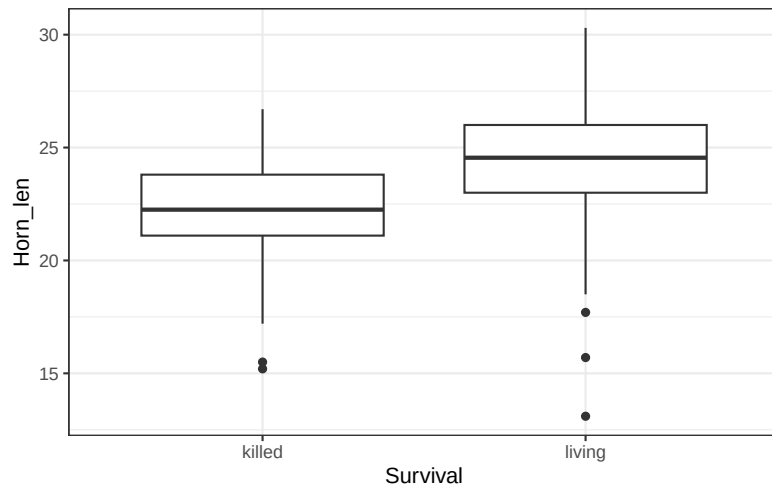
- H_0 : la moyenne de longueur des cornes de la population des lézards **morts** est égale à celle des lézards **vivants**. Leur différence vaut 0 ($\mu_{killed} - \mu_{living} = 0$).
- H_1 : la moyenne de longueur des cornes de la population des lézards **morts** est **supérieure** à celle des lézards **vivants**. Leur différence est **supérieure** à 0 ($\mu_{killed} - \mu_{living} > 0$).

Ce test est donc erroné, ce qui explique qu'il nous renvoie un résultat faux et une p -value de 1. Ici, puisque l'ordre des catégories est “`killed`” d'abord et “`living`” ensuite, la seule façon correcte de faire un test unilatéral qui a du sens est donc celle que nous avons réalisée en premier :

```
t.test(Horn_len ~ Survival, data = Lizard, var.equal = TRUE,
       alternative = "less")
```

Faites donc toujours attention à l'ordre des catégories de vos facteurs pour ne pas vous tromper. Une façon simple de vérifier cet ordre est d'observer vos graphiques (par exemple, les boîtes à moustaches). L'ordre dans lequel les catégories apparaissent sur l'axe des `x` reflète l'ordre des catégories du facteur porté par cet axe :

```
Lizard |>
  ggplot(aes(x = Survival, y = Horn_len)) +
  geom_boxplot()
```



3.10.3 Exercice d'application

Reprenez chaque exemple et exercice traité depuis le premier chapitre et identifiez les situations où un test unilatéral aurait du sens. Si vous en trouvez, faites ce test et assurez-vous que les hypothèses choisies sont bien celles qui sont utilisées lors du test.

4 Comparaison de proportions

4.1 Pré-requis

Comme pour chaque nouveau chapitre, je vous conseille de démarrer une nouvelle session de travail (Menu **Session** > **Restart R**), et de créer un nouveau script que vous placerez dans votre répertoire de travail. Inutile en revanche de créer un nouveau **Rproject** : vous pouvez tout à fait avoir plusieurs scripts dans le même répertoire de travail et pour un même **Rproject**. Comme toujours, consultez les sections du livre du semestre 3 consacrées [aux scripts](#) et [aux projets](#) si vous ne savez plus comment faire.

Si vous êtes dans une nouvelle session de travail (ou que vous avez quitté puis relancé **RStudio**), vous devrez penser à recharger en mémoire les packages utiles. Dans ce chapitre, vous aurez besoin d'utiliser :

- le **tidyverse** (Wickham 2023), qui comprend notamment **readr** (Wickham, Hester, et Bryan 2026), **dplyr** (Wickham, François, et al. 2026), **tidyr** (Wickham, Vaughan, et Girlich 2025) et **ggplot2** (Wickham, Chang, et al. 2026).
- **skimr** (Waring et al. 2026), qui permet de calculer des résumés de données très informatifs.
- **binom** (Dorai-Raj et Cascone 2026), qui permet de calculer plusieurs types d'intervalles de confiance pour une proportion.
- **ggmosaic** (Jeppson, Hofmann, et Cook 2026), pour réaliser des graphiques en mosaïque avec la syntaxe de **ggplot2**.
- **janitor** (Firke 2024), pour nettoyer facilement les noms de variables d'un tableau.
- **scales** (Wickham, Pedersen, et Seidel 2025), pour formater les axes des graphiques.

```
library(tidyverse)
library(skimr)
library(binom)
library(ggmosaic)
library(janitor)
library(scales)
```

⚠ Attention

Si certains packages ne sont pas encore installés sur votre ordinateur (par exemple, **binom** et **ggmosaic**), vous devrez les installer avant de pouvoir les charger. Consultez [la section consacrée aux packages dans le livre du semestre 3](#) pour savoir comment faire.

Vous aurez également besoin du jeu de données suivant que vous pouvez dès maintenant télécharger dans votre répertoire de travail :

- [squid.txt](#)

Enfin, je spécifie ici une fois pour toutes le thème que j'utiliserai pour tous les graphiques de ce chapitre. Libre à vous de choisir un thème différent ou de vous contenter du thème proposé par défaut.

```
theme_set(theme_bw())
```

4.2 Contexte

Jusqu'à présent, nous avons surtout comparé des **moyennes**. Pourtant, dans de très nombreuses études en biologie, en écologie, en médecine ou en agronomie, la variable mesurée n'est pas numérique continue, mais **binaire** :

- un individu est infecté ou non ;
- une graine germe ou ne germe pas ;
- un nid réussit ou échoue ;
- un animal est mâle ou femelle.

Dans ce type de situation, ce n'est plus la moyenne d'une grandeur continue que l'on cherche à comparer, mais une **proportion**. Les questions posées sont alors du type :

- la proportion de femelles dans une population diffère-t-elle de 50 % ?
- la proportion d'individus infectés est-elle la même dans deux groupes ?
- le succès de germination est-il identique dans deux traitements ?

Le cas le plus simple est celui d'une variable binaire observée sur chaque individu. Mais bientôt, des problèmes plus complexes pourront aussi être décrits sous la forme de **tables de contingence**, car les données issues de comptages sont souvent regroupées et ne sont pas toujours limitées à deux catégories. Les questions que nous abordons ici pourront donc être étendues aux facteurs possédant un nombre quelconque de modalités, et pas seulement deux. Dans l'immédiat, il faut donc comprendre à la fois :

1. comment passer des données brutes individuelles aux effectifs utiles pour les tests ;
2. comment distinguer l'**estimation** d'une proportion du **test** d'une hypothèse ;
3. comment choisir entre les principaux tests disponibles.

Dans ce chapitre, nous allons voir successivement :

1. pourquoi une proportion est une moyenne ;
2. comment estimer une proportion et son incertitude ;
3. comment comparer une proportion observée à une valeur théorique ;
4. comment comparer deux proportions dans deux groupes indépendants ;
5. comment relier ces comparaisons aux tables de contingence ;
6. quand utiliser un test exact plutôt qu'une approximation.

4.3 Importation et mise en forme des données

Nous allons travailler ici sur le fichier `squid.txt`, qui contient des données sur des calmars échantillonnés dans plusieurs sites. Les variables disponibles sont :

- **Sample** : identifiant de l'observation ;
- **Year** et **Month** : date de collecte ;
- **Location** : site d'échantillonnage ;
- **Sex** : sexe de l'individu ;
- **GSI** : Gonado-Somatic Index, ou indice gonado-somatique. C'est un rapport qui rend compte de la masse des gonades par rapport à la masse de l'individu et qui renseigne notamment sur le statut reproducteur des individus, leur maturité sexuelle, etc. Ici, il est exprimé en pourcents.

Nous n'utiliserons dans ce chapitre que les variables **Location** et **Sex**. Importez les données dans un objet nommé `squid_raw` (si vous ne savez plus comment faire, consultez [la section consacrée à l'importation dans le livre du semestre 3](#)) puis examinez le tableau obtenu :

```
squid_raw
```



```
# A tibble: 2,644 x 6
  Sample Year Month Location Sex  GSI
  <dbl> <dbl> <dbl>   <dbl> <dbl> <dbl>
1     1     1     1       1     2 10.4
2     2     1     1       3     2  9.83
3     3     1     1       1     2  9.74
4     4     1     1       1     2  9.31
5     5     1     1       1     2  8.99
6     6     1     1       1     2  8.77
7     7     1     1       1     2  8.26
8     8     1     1       3     2  7.40
9     9     1     1       3     2  7.22
10    10     1     2       1     2  6.84
# i 2,634 more rows
```

Comme dans d'autres chapitres du livre, il est utile d'harmoniser ou de simplifier les intitulés des colonnes dès

l'importation (par exemple, pour supprimer les majuscules ou les caractères spéciaux). Le package `janitor` permet de faire cela très facilement avec la fonction `clean_names()` :

```
squid_raw <- squid_raw |>
  clean_names()

names(squid_raw)
```

```
[1] "sample"    "year"      "month"     "location"  "sex"       "gsi"
```

Les sexes sont ici codés 1 et 2. Pour rendre le tableau plus lisible, nous allons transformer cette variable en facteur, et créer une variable logique `female` qui vaudra `TRUE` pour les femelles et `FALSE` pour les mâles :

```
squid <- squid_raw |>
  mutate(
    location = factor(location),
    sex = factor(sex, labels = c("male", "female")),
    female = sex == "female"
  )

squid
```

```
# A tibble: 2,644 x 7
   sample year month location sex    gsi female
   <dbl> <dbl> <dbl> <fct> <fct> <dbl> <lgl>
1      1     1     1     1 1     female 10.4  TRUE
2      2     1     1     3     female  9.83  TRUE
3      3     1     1     1     female  9.74  TRUE
4      4     1     1     1     female  9.31  TRUE
5      5     1     1     1     female  8.99  TRUE
6      6     1     1     1     female  8.77  TRUE
7      7     1     1     1     female  8.26  TRUE
8      8     1     1     3     female  7.40  TRUE
9      9     1     1     3     female  7.22  TRUE
10    10     1     2     1     female  6.84  TRUE
# i 2,634 more rows
```

Nous disposons maintenant d'un tableau **rangé**, au format long :

- chaque ligne correspond à un individu ;
- `location` et `sex` sont des variables qualitatives ;
- `female` est une variable logique qui code le sexe de façon binaire.

C'est ce type de tableau qui doit toujours constituer votre point de départ. Les tests sur proportions utilisent ensuite des **effectifs agrégés**, mais ces effectifs doivent être obtenus à partir de données individuelles correctement structurées.

4.4 Cas 1 : comparer une proportion observée à une proportion théorique

Nous allons commencer par la situation la plus simple : un seul échantillon, une variable binaire, et une proportion observée à comparer à une valeur théorique. Ici, nous allons nous demander si la proportion de femelles parmi les calmars du site 3 est compatible avec un sex-ratio équilibré, c'est-à-dire 50 % de femelles et 50 % de mâles.

Comme dans les chapitres précédents, nous allons procéder dans l'ordre :

1. définir la question statistique ;
2. examiner les données à l'aide de statistiques descriptives ;
3. réaliser une exploration graphique ;
4. effectuer le test d'hypothèse approprié.

4.4.1 Question statistique et données utiles

Avant de parler de test statistique, il faut d'abord isoler les données utiles à la question posée.

Prenons donc les calmars du site 3 :

```
squid_3 <- squid |>
  filter(location == "3")

squid_3
```

```
# A tibble: 447 x 7
  sample year month location sex    gsi female
  <dbl> <dbl> <dbl> <fct>   <fct> <dbl> <lgl>
1      2     1     1  1 3     female 9.83 TRUE
2      8     1     1  1 3     female 7.40 TRUE
3      9     1     1  1 3     female 7.22 TRUE
4     17     1     6  3     female 5.78 TRUE
5     19     1     5  3     female 5.65 TRUE
6     21     1     5  3     female 5.60 TRUE
7     40     1     6  3     female 4.66 TRUE
8     45     1     5  3     female 4.49 TRUE
9     48     1     5  3     male   4.30 FALSE
10    56     1     5  3     female 3.91 TRUE
# i 437 more rows
```

Nous disposons maintenant d'un échantillon unique, pour lequel chaque individu est soit une femelle, soit un mâle. La question statistique peut donc être formulée ainsi : la proportion de femelles dans cet échantillon est-elle compatible avec une proportion théorique de 0.5 ?

4.4.2 Exploration statistique des données

4.4.2.1 Une proportion est une moyenne

Avant même de parler de test statistique, il faut bien comprendre qu'une proportion est tout simplement la moyenne d'une variable codée TRUE / FALSE ou 1 / 0.

Le nombre de femelles et de mâles s'obtient facilement :

```
squid_3 |>
  count(sex)

# A tibble: 2 x 2
  sex      n
  <fct> <int>
1 male   261
2 female 186
```

On peut également demander un résumé rapide du tableau :

```
skim(squid_3)
```

```
-- Data Summary -----
```

	Values
Name	squid_3
Number of rows	447
Number of columns	7

```
-----  
Column type frequency:
```

factor	2
logical	1
numeric	4

```
-----  
Group variables      None
```

```
-- Variable type: factor -----
```

	skim_variable	n_missing	complete_rate	ordered	n_unique
1	location	0	1	FALSE	1
2	sex	0	1	FALSE	2
	top_counts				
1	3:	447,	1:	0,	2:
2	mal:	261,	fem:	186	

```
-- Variable type: logical -----
```

	skim_variable	n_missing	complete_rate	mean	count
1	female	0	1	0.416	FAL: 261, TRU: 186

```
-- Variable type: numeric -----
```

	skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50
1	sample	0	1	819.	526.	2	352.	809
2	year	0	1	1.75	0.676	1	1	2
3	month	0	1	5.00	2.20	1	4	5
4	gsi	0	1	2.06	2.16	0.0157	0.288	1.44
	p75							
1	1145							
2	2							
3	7							
4	2.40							

Vous pouvez noter dès maintenant que pour la variable `female`, qui est une variable logique, une moyenne est renvoyée par `skim()` : 0.416. Lorsqu'une moyenne de

TRUE/FALSE est calculée, les TRUE sont remplacés par 1, les FALSE par 0. Calculer la moyenne d'une variable logique revient donc à calculer la proportion de TRUE. C'est une idée essentielle, car elle permet de faire le lien entre les chapitres précédents consacrés aux moyennes et ce nouveau chapitre consacré aux proportions.

La proportion de femelles peut donc être obtenue en calculant la moyenne de la variable logique `female` :

```
squid_3 |>
  summarise(proportion_femelles = mean(female))
```

```
# A tibble: 1 x 1
  proportion_femelles
              <dbl>
1                0.416
```

Et si l'on préfère, on peut présenter le résultat en pourcentage :

```
squid_3 |>
  summarise(pourcentage_femelles = 100 * mean(female))
```

```
# A tibble: 1 x 1
  pourcentage_femelles
              <dbl>
1                 41.6
```

Bien entendu, il est aussi possible de calculer les proportions de femelles et de mâles à partir du comptage des effectifs grâce aux fonctions `count()` et `mutate()` :

```
squid_3 |>
  count(sex)
```

```
# A tibble: 2 x 2
  sex      n
  <fct> <int>
1 male   261
2 female 186
```

```
squid_3 |>
  count(sex) |>
  mutate(proportion = n / sum(n))
```

```
# A tibble: 2 x 3
  sex      n proportion
<fct> <int>   <dbl>
1 male   261    0.584
2 female 186    0.416
```

Notez que la somme des proportions est évidemment égale à 1.

4.4.2.2 L'incertitude d'une proportion

Comme pour une moyenne, une proportion observée dans un échantillon n'est qu'une **estimation** de la proportion réelle dans la population générale. Il faut donc s'intéresser à son incertitude.

Ici, nous allons estimer la proportion de femelles parmi les calmars échantillonnés au site 3 à l'aide de la fonction `binom.test()`. Cette fonction réalise un test binomial exact, dont les résultats ne nous intéressent pas encore à ce stade. En revanche, outre les résultats du test, elle fournit aussi l'intervalle de confiance de la proportion observée. `binom.test()` prend deux arguments : `x` est le nombre de succès (ici, c'est le nombre de femelles) et `n`, le nombre total d'essais (ici, le nombre total de calmars échantillonnés à la station 3) :

```
binom.test(
  x = sum(squid_3$female),
  n = nrow(squid_3)
)
```

Exact binomial test

```
data:  sum(squid_3$female) and nrow(squid_3)
number of successes = 186, number of trials = 447, p-value = 0.0004523
```

```

alternative hypothesis: true probability of success is not equal to 0.5
95 percent confidence interval:
 0.3699820 0.4633504
sample estimates:
probability of success
      0.4161074

```

Plusieurs éléments nous intéressent donc ici :

- **number of successes** correspond ici au nombre de femelles observées ;
- **number of trials** correspond au nombre total de calmars échantillonnés au site 3 ;
- **sample estimates** donne la proportion observée ;
- **95 percent confidence interval** fournit un intervalle de confiance à 95 % pour cette proportion.

La proportion observée de femelles vaut donc 0.416, soit environ 41.6 %. L'intervalle de confiance de cette proportion nous indique quelles sont les valeurs plausibles de la proportion de femelles dans la population générale, compte tenu de notre échantillon. Ici, dans la population générale, la proportion de femelles a de bonnes chances d'être comprise entre 37 % et 46.3 %.

On peut accéder directement aux valeurs de l'intervalle de confiance en utilisant l'élément `conf.int` de l'objet renvoyé par `binom.test()` :

```

test_exact <- binom.test(
  x = sum(squid_3$female),
  n = nrow(squid_3)
)

test_exact$conf.int[1]

```

```
[1] 0.369982
```

```
test_exact$conf.int[2]
```

```
[1] 0.4633504
```

La méthode utilisée pour le calcul de cet intervalle de confiance est la méthode **exacte** (on parle aussi souvent d'intervalle exact binomial, ou intervalle de Clopper-Pearson). Elle n'est toutefois pas la seule possible. Le package **binom**, que nous avons chargé en début de chapitre, contient la fonction **binom.confint()**, qui permet de calculer facilement l'intervalle de confiance d'une proportion selon plusieurs méthodes de calcul. Les arguments de cette fonction sont les mêmes que ceux de **binom.test()** :

```
binom.confint(
  x = sum(squid_3$female),
  n = nrow(squid_3)
)
```

	method	x	n	mean	lower	upper
1	agresti-coull	186	447	0.4161074	0.3713117	0.4623328
2	asymptotic	186	447	0.4161074	0.3704129	0.4618018
3	bayes	186	447	0.4162946	0.3708289	0.4619696
4	cloglog	186	447	0.4161074	0.3701677	0.4613510
5	exact	186	447	0.4161074	0.3699820	0.4633504
6	logit	186	447	0.4161074	0.3712528	0.4623958
7	probit	186	447	0.4161074	0.3710776	0.4622673
8	profile	186	447	0.4161074	0.3709891	0.4621846
9	lrt	186	447	0.4161074	0.3710090	0.4621738
10	prop.test	186	447	0.4161074	0.3702257	0.4634535
11	wilson	186	447	0.4161074	0.3713172	0.4623272

La méthode **exact** correspond à celle que nous retrouvons dans les résultats de **binom.test()**, les bornes de l'intervalle de confiance sont donc strictement identiques entre les deux fonctions. En général, c'est la méthode recommandée. Toutefois, la méthode **ac** (pour méthode d'**Agresti-Coull**) est parfois utile, car c'est l'une des rares méthodes de la liste qui permet un calcul des bornes de l'intervalle de confiance à 95% à la main. C'est donc la méthode approchée qui était privilégiée historiquement lorsque les moyens de calculs modernes n'existaient pas.

L'intervalle d'Agresti-Coull à 95 % peut être calculé de façon approchée en ajoutant 2 succès et 2 échecs. Si **x** est le nombre de succès observés et **n** la taille de l'échantillon, on calcule d'abord :

$$\tilde{p} = \frac{x+2}{n+4}$$

Puis l'erreur standard associée à cette proportion ajustée :

$$SE_{\tilde{p}} = \sqrt{\frac{\tilde{p}(1-\tilde{p})}{n+4}}$$

Enfin, l'intervalle de confiance à 95 % est approximativement :

$$\tilde{p} \pm 1.96 \times SE_{\tilde{p}}$$

Pour les calmars du site 3, on obtient ceci :

```
x <- sum(squid_3$female)
n <- nrow(squid_3)

p_tilde <- (x + 2) / (n + 4)
se_tilde <- sqrt(p_tilde * (1 - p_tilde) / (n + 4))

tibble(
  p_tilde = p_tilde,
  IC95_inf = p_tilde - 1.96 * se_tilde,
  IC95_sup = p_tilde + 1.96 * se_tilde
)
```

```
# A tibble: 1 x 3
  p_tilde IC95_inf IC95_sup
  <dbl>    <dbl>    <dbl>
1  0.417    0.371    0.462
```

On retrouve des valeurs très proches de celles renvoyées par `binom.confint(..., methods = "ac")`. Autrement dit :

- `binom.test()` fournit ici un **intervalle exact** ;
- `binom.confint(methods = "exact")` retrouve ce même intervalle ;
- `binom.confint(methods = "ac")` donne l'intervalle d'Agresti-Coull ;

- et cet intervalle d'Agresti-Coull peut être approximativement recalculé à la main à partir de la formule ci-dessus.

! Estimer n'est pas tester

Il faut bien distinguer deux opérations intellectuelles différentes :

- **estimer une proportion**, c'est chercher quelle valeur du paramètre est plausible dans la population générale. Cela revient donc à calculer la proportion observée grâce aux données disponibles ainsi que son intervalle de confiance à 95% ;
- **tester une proportion**, c'est chercher si une hypothèse précise sur ce paramètre peut être rejetée ou non.

L'estimation répond à la question "**combien ?**".

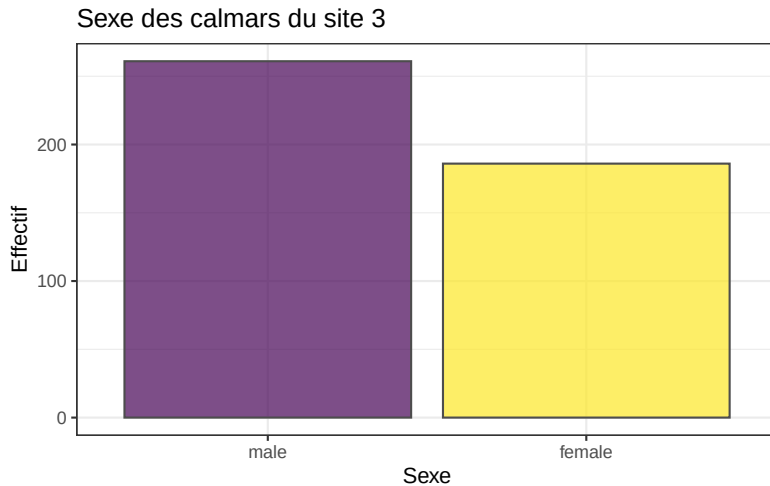
Le test répond à la question "**compatible avec quelle hypothèse ?**".

Cette dernière remarque est très importante et elle aurait pu être faite dans les chapitres précédents consacrés aux moyennes. Il ne faut jamais confondre l'estimation d'une grandeur (moyenne, proportion, etc.) avec le test d'une hypothèse à son sujet. L'estimation doit toujours précéder le test, et elle doit être examinée attentivement avant de se lancer dans le test. Si un test rejette l'hypothèse nulle, cela ne nous dit absolument rien sur la valeur réelle du paramètre dans la population générale. De même, si un test ne rejette pas l'hypothèse nulle, cela ne signifie pas que l'hypothèse nulle est vraie. C'est pour cela que les estimations et leurs intervalles de confiance sont tout aussi importants que les p -values. Et c'est la raison pour laquelle la plupart des fonctions de test dans R, renvoient à la fois les résultats du test, les estimations et les intervalles de confiance pertinents pour l'interprétation des résultats.

4.4.3 Exploration graphique des données

Avant de faire un test, il est toujours utile de visualiser les effectifs observés :

```
squid_3 |>
  ggplot(aes(x = sex, fill = sex)) +
  geom_bar(show.legend = FALSE, alpha = 0.7, color = "grey30") +
  labs(x = "Sexe", y = "Effectif", title = "Sexe des calmars du site 3") +
  scale_fill_viridis_d()
```



Ce graphique permet de voir immédiatement si l'on observe un déséquilibre marqué ou non entre les deux sexes. Mais un déséquilibre visuel ne suffit jamais à lui seul : il faut encore déterminer s'il peut être attribué au simple hasard de l'échantillonnage.

Puisqu'on sait maintenant comment calculer l'intervalle de confiance d'une proportion, on peut aussi visualiser l'incertitude associée à la proportion de mâles et de femelles. Pour cela, on récupère les effectifs et les proportions, puis on calcule l'intervalle de confiance à 95 % pour chaque proportion :

```
squid_3_summary <- squid_3 |>
  count(sex) |>
  mutate(proportion = n / sum(n)) |>
  rowwise() |>
  mutate(
    ci_low = binom.test(n, nrow(squid_3))$conf.int[1],
    ci_high = binom.test(n, nrow(squid_3))$conf.int[2]
  )
```

```
squid_3_summary
```

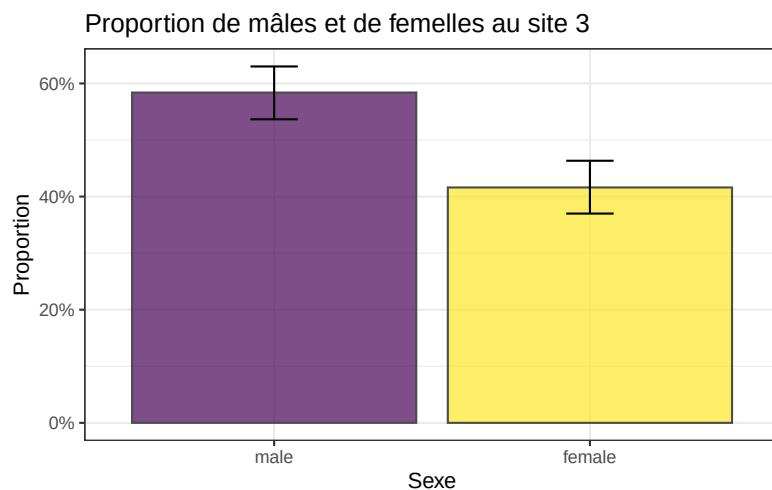
```
# A tibble: 2 x 5
# Rowwise:
  sex      n proportion ci_low ci_high
<fct> <int>      <dbl> <dbl> <dbl>
1 male    261      0.584  0.537  0.630
2 female  186      0.416  0.370  0.463
```

La fonction `rowwise()` permet de traiter chaque ligne du tableau individuellement, un peu comme si on faisait un `group_by()` et que chaque ligne du tableau se retrouvait dans un groupe distinct. Cela permet entre autres choses d'appliquer la même fonction à chaque ligne, en lui fournissant des arguments différents pour chaque ligne. Ici, pour chaque ligne, on calcule l'intervalle de confiance pour la proportion correspondante grâce à `binom.test()`. Pour plus de détail, consultez [cette vignette en ligne](#), ou en tapant ceci dans la console :

```
vignette("rowwise")
```

On utilise ensuite ce tableau pour la représentation graphique :

```
squid_3_summary |>
  ggplot(aes(x = sex, y = proportion, fill = sex)) +
  geom_col(show.legend = FALSE, alpha = 0.7, color = "grey30") +
  geom_errorbar(aes(ymin = ci_low, ymax = ci_high), width = 0.15) +
  scale_y_continuous(labels = percent_format()) +
  labs(
    x = "Sexe",
    y = "Proportion",
    title = "Proportion de mâles et de femelles au site 3"
  ) +
  scale_fill_viridis_d()
```



4.4.4 Test d'hypothèse : comparer une proportion à une valeur théorique

Supposons maintenant que l'hypothèse théorique soit celle d'un sex-ratio équilibré, c'est-à-dire 50 % de femelles et 50 % de mâles. Nous voulons donc savoir si la proportion observée de femelles au site 3 est compatible avec cette hypothèse.

Les hypothèses testées sont les suivantes :

- H_0 : la proportion de femelles dans la population vaut 0.5.
- H_1 : la proportion de femelles dans la population est différente de 0.5.

Le test adapté est le **test binomial exact** :

```
binom.test(
  x = sum(squid_3$female),
  n = nrow(squid_3),
  p = 0.5
)
```

Exact binomial test

```
data: sum(squid_3$female) and nrow(squid_3)
number of successes = 186, number of trials = 447, p-value = 0.0004523
alternative hypothesis: true probability of success is not equal to 0.5
```

95 percent confidence interval:

0.3699820 0.4633504

sample estimates:

probability of success

0.4161074

La p -value de ce test nous permet de décider si l'écart observé à 50 % est trop grand pour être attribué au seul hasard de l'échantillonnage. Ici, on rejette sans ambiguïté l'hypothèse nulle car la p -value est nettement inférieure au seuil alpha de 0.05. Le test nous indique que si la proportion de femelles dans la population générale est effectivement de 50%, observer par hasard (sous le seul effet de la fluctuation d'échantillonnage) la proportion que nous avons obtenue dans notre échantillon (0.416) ou une proportion encore plus éloignée de 0.5 est très peu probable : cette probabilité vaut 0.00045 (c'est la définition de la p -value). On rejette donc l'hypothèse nulle du test binomial : l'écart observé entre la proportion observée et 0.5 n'est pas seulement lié au hasard. D'ailleurs, l'intervalle de confiance à 95% de la proportion ne contient pas la valeur de 0.5 spécifiée dans l'hypothèse nulle. Cela signifie que 0.5 ne fait pas partie des valeurs les plus probables pour la proportion de femelles dans la population générale.

Ne pas confondre 0.5 et 0.05

Il est fréquent que de fausses conclusions soient tirées des résultats de tests sur des proportions, car il est facile de comparer, à tort, la p -value à 0.5 au lieu de la comparer à 0.05 comme d'habitude :

- 0.5 est la proportion supposée que l'on spécifie dans l'hypothèse nulle, et à laquelle on souhaite comparer la proportion observée
- 0.05 est le seuil α du test binomial

Soyez vigilants !

Le point important ici est le suivant : nous avons bien affaire à un **échantillon unique**, pour lequel nous comparons une proportion observée (ici, 0.416) à une **valeur théorique** (ici, 0.5, c'est-à-dire un sex-ratio équilibré). C'est exactement la situation dans laquelle il faut utiliser `binom.test()`.

4.4.4.1 Test unilatéral

Dans certains cas, l'hypothèse de recherche n'est pas "différente de", mais "supérieure à" ou "inférieure à". On peut alors réaliser un test unilatéral.

Par exemple, si nous avons de bonnes raisons biologiques de penser qu'au site 3 les femelles sont **moins fréquentes** que les mâles, nous pourrions formuler les hypothèses suivantes :

- H_0 : la proportion de femelles vaut 0.5.
- H_1 : la proportion de femelles est **inférieure** à 0.5.

Le code devient :

```
binom.test(  
  x = sum(squid_3$female),  
  n = nrow(squid_3),  
  p = 0.5,  
  alternative = "less"  
)
```

Exact binomial test

```
data: sum(squid_3$female) and nrow(squid_3)  
number of successes = 186, number of trials = 447, p-value = 0.0002262  
alternative hypothesis: true probability of success is less than 0.5  
95 percent confidence interval:  
 0.0000000 0.4559052  
sample estimates:  
probability of success  
 0.4161074
```

Un test unilatéral ne doit toutefois être utilisé que si l'hypothèse directionnelle a été formulée **avant** de regarder les données. Il ne faut jamais choisir après coup le sens du test qui arrange le mieux les résultats. Et il faudrait donc pouvoir justifier ce choix de façon objective, sur la base de travaux antérieurs, ou de connaissances acquises dans la littérature scientifique par exemple. N'hésitez pas à relire la Section 3.10 si vous n'êtes pas sûr d'avoir compris (ou

retenu) les situations dans lesquelles un test unilatéral est possible.

Ici, la conclusion du test unilatéral est la même que celle du test bilatéral : on rejette l'hypothèse nulle. Mais l'interprétation est différente : avec le test bilatéral, on concluait simplement que la proportion de femelles différait de 0.5, sans préciser dans quel sens. Avec le test unilatéral, on conclut que la proportion de femelles est **inférieure** à 0.5. Avec le test bilatéral, arriver à cette conclusion demandait une étape supplémentaire : le test montre que la proportion de femelles n'est pas égale à 0.5, et **l'estimation de la proportion** (et de son incertitude), nous indique que la proportion de femelles vaut 0.416, et qu'elle est donc significativement inférieure à 0.5. Lorsqu'on n'a pas de raison valable de faire un test unilatéral, le test bilatéral est presque toujours la meilleure option, et il permet presque toujours d'aboutir à la même conclusion que le test unilatéral.

Notez toutefois que la p -value du test unilatéral est exactement la moitié de celle du test bilatéral. C'est logique : le test bilatéral prend en compte les deux extrémités de la distribution nulle de la statistique du test, alors que le test unilatéral ne prend en compte qu'une seule extrémité. C'est ce qui explique le gain de puissance statistique d'un test unilatéral par rapport à son homologue bilatéral. Pour nos exemples, cela n'a pas d'importance, car les p -values des deux tests sont très éloignées du seuil α de 0.05. Mais dans d'autres situations, ce gain de puissance peut faire la différence entre rejeter ou non l'hypothèse nulle. C'est pour cela que le choix entre test bilatéral et test unilatéral doit être fait **avant** de regarder les données, et qu'il doit être justifié par des arguments biologiques ou scientifiques solides, et pas simplement par commodité ou parce que cela vous arrange sur le moment. C'est une question d'honnêteté intellectuelle et de rigueur scientifique.

4.5 Cas 2 : comparer deux proportions indépendantes

La deuxième situation classique consiste à comparer une proportion entre deux groupes indépendants. La logique reste la

même que dans le premier cas, mais il faut maintenant examiner deux échantillons, deux proportions, et leur incertitude respective.

Nous allons conserver la même organisation :

1. définir la question statistique ;
2. examiner les données à l'aide de statistiques descriptives ;
3. réaliser une exploration graphique ;
4. effectuer le test d'hypothèse approprié.

4.5.1 Question statistique et données utiles

Nous allons comparer ici la proportion de femelles entre les sites 1 et 3. Biologiquement, la question est simple : les sex-ratios observés dans ces deux zones d'échantillonnage sont-ils compatibles avec une même proportion sous-jacente, ou bien les deux sites se distinguent-ils ?

Le point important est le suivant : les données sont toujours observées **individu par individu**. Le tableau rangé reste donc notre point de départ pour extraire les données des sites 1 et 3 :

```
squid_13 <- squid |>
  filter(location %in% c("1", "3"))

squid_13
```

A tibble: 2,438 x 7

	sample	year	month	location	sex	gsi	female
	<dbl>	<dbl>	<dbl>	<fct>	<fct>	<dbl>	<lgl>
1	1	1	1	1	female	10.4	TRUE
2	2	1	1	3	female	9.83	TRUE
3	3	1	1	1	female	9.74	TRUE
4	4	1	1	1	female	9.31	TRUE
5	5	1	1	1	female	8.99	TRUE
6	6	1	1	1	female	8.77	TRUE
7	7	1	1	1	female	8.26	TRUE
8	8	1	1	3	female	7.40	TRUE
9	9	1	1	3	female	7.22	TRUE
10	10	1	2	1	female	6.84	TRUE

```
# i 2,428 more rows
```

4.5.2 Exploration statistique des données

Chaque ligne correspond à un individu. Pour examiner les effectifs observés, on peut commencer par croiser le site et le sexe :

```
squid_13 |>
  count(location, sex)
```

```
# A tibble: 4 x 3
  location sex      n
  <fct>    <fct> <int>
1 1      male   1031
2 1     female   960
3 3      male    261
4 3     female   186
```

À partir de ce tableau long, nous pouvons ensuite construire un résumé par site :

```
sex_ratio <- squid_13 |>
  summarise(
    effectif = n(),
    femelles = sum(female),
    proportion_femelles = mean(female),
    .by = location
  ) |>
  rowwise() |>
  mutate(
    ci_low = binom.test(femelles, effectif)$conf.int[1],
    ci_high = binom.test(femelles, effectif)$conf.int[2]
  ) |>
  ungroup()

sex_ratio
```

```
# A tibble: 2 x 6
  location effectif femelles proportion_femelles ci_low ci_high
  <fct>      <int>    <int>          <dbl>    <dbl>    <dbl>
1 1         1991     1991          0.9999999 0.9999999 0.9999999
2 3          261     186          0.7126437 0.5888889 0.8333333
```

1	1	1991	960	0.482	0.460	0.504
2	3	447	186	0.416	0.370	0.463

Nous avons donc ici effectué un passage fondamental :

1. nous partons de données **individuelles** au format long ;
2. nous calculons les **effectifs** et **proportions** utiles pour chaque groupe ;
3. nous ajoutons des **intervalles de confiance** pour visualiser l'incertitude.

Autrement dit, le test de comparaison de proportions ne remplace pas le travail préalable de mise en forme et d'exploration des données. Il vient **après**.

Ici, on constate que les intervalles de confiance à 95% des proportions des sites 1 et 3 ne se chevauchent presque pas, ce qui suggère déjà que les deux proportions seront très certainement significativement différentes. L'intervalle de confiance à 95% du site 1 est d'ailleurs beaucoup moins large que celui du site 3, probablement en partie en raison d'une taille d'échantillon très supérieure dans ce site. Cela nous confirme que l'estimation de la proportion de femelles dans le site 1 est plus précise que celle du site 3.

4.5.3 Exploration graphique des données

Comme toujours, avant de faire le test, une étape d'exploration graphique est toujours utile. Pour comparer des proportions, on peut visualiser les données sous plusieurs angles. Les [diagrammes bâtons empilés en proportions](#) et les [mosaic plots](#) présentés dans le livre du semestre 3 sont particulièrement adaptés à ce type de question.

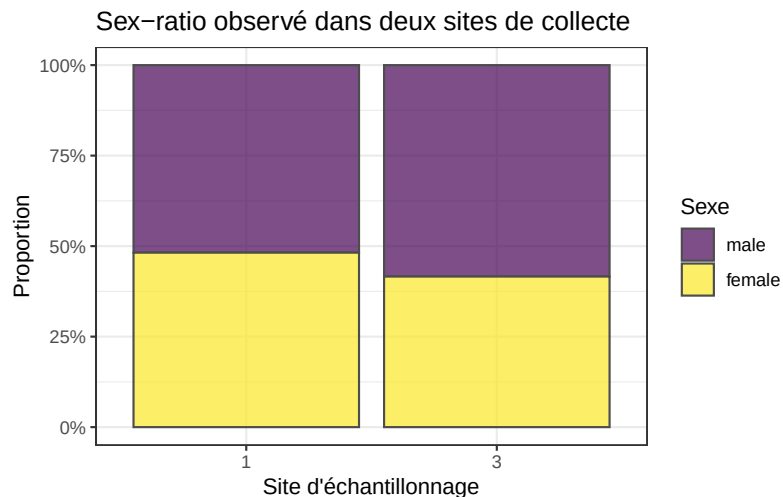
4.5.3.1 Avec un diagramme bâtons empilé en proportions

```
squid_13 |>
  ggplot(aes(x = location, fill = sex)) +
  geom_bar(position = "fill", color = "grey30", alpha = 0.7) +
  scale_y_continuous(labels = percent_format()) +
  labs(
```

```

x = "Site d'échantillonnage",
y = "Proportion",
fill = "Sexe",
title = "Sex-ratio observé dans deux sites de collecte"
) +
scale_fill_viridis_d()

```



Ce graphique permet de comparer rapidement les proportions de femelles et de mâles entre les deux sites. Il montre bien les proportions, mais il ne donne pas directement accès aux effectifs absolus.

4.5.3.2 Avec un mosaic plot

```

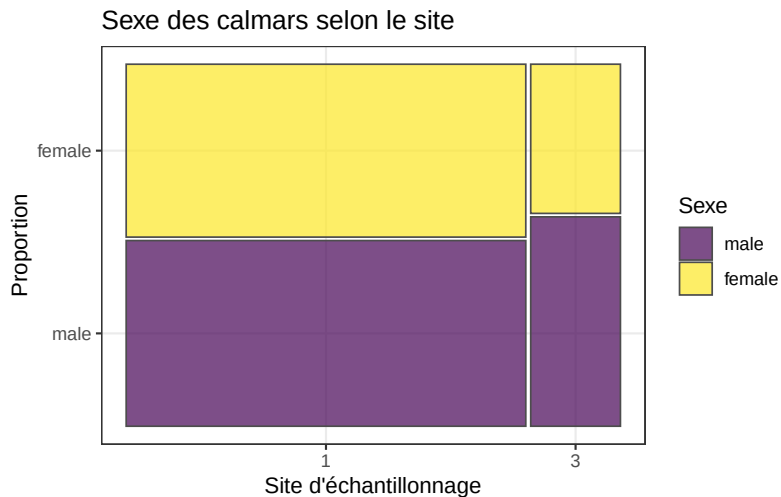
squid_13 |>
  mutate(location = fct_drop(location)) |> # Pour que les sites 2 et 4 n'apparaissent pas
  ggplot() +
  geom_mosaic(
    aes(x = location, fill = sex),
    color = "grey30",
    linewidth = 0.4,
    alpha = 0.7
  ) +
  scale_fill_viridis_d() +
  scale_x_productlist(name = "Site d'échantillonnage") +
  scale_y_productlist(name = "Proportion") +
  labs(

```

```

    fill = "Sexe",
    title = "Sexe des calmars selon le site"
  )

```



Le graphique en mosaïque rappelle encore mieux qu'il s'agit d'un problème de **table de contingence** :

- la largeur des rectangles dépend des effectifs ;
- la hauteur des subdivisions dépend des proportions de chaque sexe dans chaque site.

Il est donc particulièrement utile quand on veut garder en tête simultanément les proportions **et** les tailles d'échantillon.

Ici, le code présente quelques subtilités :

- la fonction `fct_drop()` permet de supprimer les modalités inutilisées du facteur `location` (ici, les sites 2 et 4). Sans cette ligne, l'axe des abscisses ferait apparaître les chiffres 1, 2, 3 et 4, avec un espace vide au niveau des sites 2 et 4 ;
- `geom_mosaic` transforme la nature des axes du graphique. C'est la raison pour laquelle le changement des titres des axes des abscisses et des ordonnées se fait avec `scale_x_productlist()` et `scale_y_productlist()`, et non avec `labs(x = ..., y = ...)` comme d'habitude.

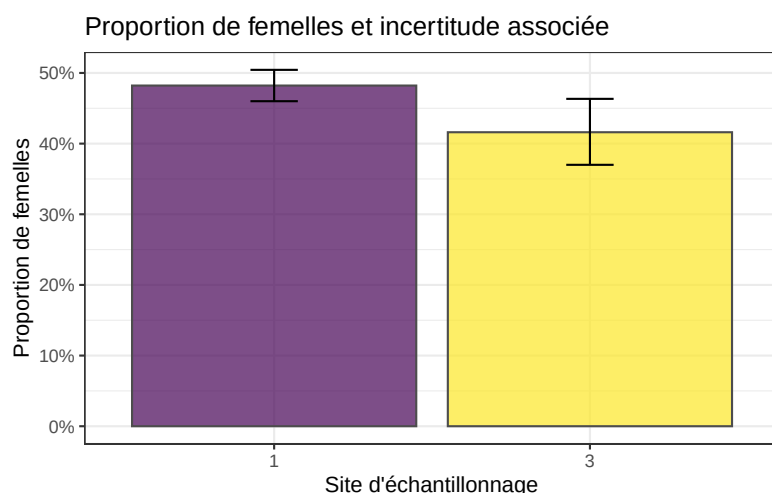
L'inconvénient de ce type de graphique est que la valeur numérique des proportions n'est pas directement lisible sur un

axe. Il faudrait les calculer au préalable et les ajouter manuellement sur le graphique avec `geom_text()` par exemple, mais c'est nettement plus complexe et ça n'est pas absolument indispensable.

4.5.3.3 Avec des intervalles de confiance

Enfin, comme pour toute estimation, il est utile de visualiser l'incertitude :

```
sex_ratio |>
  ggplot(aes(x = location, y = proportion_femelles, fill = location)) +
  geom_col(show.legend = FALSE, alpha = 0.7, color = "grey30") +
  geom_errorbar(aes(ymin = ci_low, ymax = ci_high), width = 0.15) +
  scale_y_continuous(labels = percent_format()) +
  labs(
    x = "Site d'échantillonnage",
    y = "Proportion de femelles",
    title = "Proportion de femelles et incertitude associée"
  ) +
  scale_fill_viridis_d()
```



Ce graphique est plus informatif qu'une simple comparaison de hauteurs de barres. Il montre à la fois :

- la proportion observée de femelles dans chaque site ;
- l'incertitude associée à ces proportions ;
- l'importance ou non du chevauchement entre les intervalles de confiance.

Ici, le graphique confirme que la proportion de femelles semble plus élevée dans le site 1 que dans le site 3, et que les intervalles de confiance à 95 % ne se chevauchent presque pas. Cela suggère que les deux proportions sont probablement significativement différentes. C'est ce que le test statistique devra confirmer.

4.5.4 Test d'hypothèse : comparer deux proportions

Le test le plus simple pour comparer deux proportions est le test de comparaison de proportions avec correction de continuité, que l'on effectue dans R avec la fonction `prop.test()`. Cette fonction ne travaille pas à l'échelle individuelle, mais bien à l'échelle des **effectifs agrégés** :

- `x` contient le nombre de succès dans chaque groupe ;
- `n` contient la taille de chaque groupe.

Les hypothèses testées sont :

- H_0 : les proportions sont identiques dans les deux populations.
- H_1 : les proportions diffèrent entre les deux populations.

```
prop.test(  
  x = sex_ratio$femelles,  
  n = sex_ratio$effectif  
)
```

2-sample test for equality of proportions with continuity correction

```
data: sex_ratio$femelles out of sex_ratio$effectif  
X-squared = 6.133, df = 1, p-value = 0.01327  
alternative hypothesis: two.sided  
95 percent confidence interval:  
 0.01400025 0.11812452  
sample estimates:  
  prop 1    prop 2  
0.4821698 0.4161074
```

La fonction renvoie entre autres :

- une estimation de la proportion dans chaque groupe ;
- une p-value pour le test d'égalité des proportions ;
- un intervalle de confiance à 95 % pour la différence entre les proportions.

Comme pour la plupart des tests statistiques vus jusqu'ici, il est possible d'afficher les résultats de tests dans un tableau rangé grâce à la fonction `tidy()` du package `broom` :

```
prop.test(
  x = sex_ratio$femelles,
  n = sex_ratio$effectif
) |>
  broom::tidy()

# A tibble: 1 x 9
  estimate1 estimate2 statistic p.value parameter conf.low conf.high method
    <dbl>      <dbl>      <dbl>  <dbl>      <dbl>    <dbl>    <dbl> <chr>
1     0.482     0.416       6.13  0.0133         1     0.0140     0.118 2-sample t~
# i 1 more variable: alternative <chr>
```

Il faut noter ici un point de méthode important : contrairement à `binom.test()`, qui repose sur un calcul exact, `prop.test()` repose sur une **approximation du χ^2** et applique par défaut une **correction de continuité**. Dans les grands échantillons, cela ne pose en général aucun problème. Avec de petits effectifs, en revanche, il faudra se tourner vers un test exact.

Le point essentiel, ici, est de bien comprendre que le test porte sur des **comptages agrégés**, mais que ces comptages proviennent de données individuelles convenablement structurées. Si vous sautez directement à la table de comptage sans réfléchir au tableau de départ, vous vous privez d'une bonne partie du raisonnement statistique.

! À retenir !

Pour comparer deux proportions, il ne faut pas partir d'un tableau résumé trouvé "tout fait" par hasard. Il faut raisonner dans l'ordre :

1. données individuelles au format long ;
2. définition claire de ce qu'est un "succès" ;

3. calcul des effectifs et proportions par groupe ;
4. exploration graphique ;
5. visualisation de l'incertitude ;
6. test statistique adapté.

4.5.5 Du test de deux proportions à la table de contingence

Le même problème peut être présenté sous forme de **table de contingence** :

```
tab_squid <- squid_13 |>
  count(location, sex) |>
  pivot_wider(names_from = sex, values_from = n) |>
  column_to_rownames("location") |>
  as.matrix()
```

tab_squid

	male	female
1	1031	960
3	261	186

```
class(tab_squid)
```

```
[1] "matrix" "array"
```

Ce tableau n'est plus un **tibble**, mais une matrice. Il croise **deux variables qualitatives** (le site d'échantillonnage et le sexe) et contient des **effectifs** : c'est une table de contingence construite avec exactement les mêmes données que précédemment. Notez qu'il ne contient pas les nombres de femelles et les effectifs totaux par site comme nous l'avons vu plus haut, mais bien les nombres de mâles et de femelles dans les deux sites d'intérêt. Ici, les sites sont présentés en lignes et les sexes en colonnes, mais on aurait tout à fait pu choisir l'inverse : le résultat aurait été strictement le même.

On peut alors utiliser un test du χ^2 d'indépendance :

```
chisq.test(tab_squid)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data: tab_squid  
X-squared = 6.133, df = 1, p-value = 0.01327
```

Dans une table 2×2 , `chisq.test()` et `prop.test()` sont très proches dans leur logique et conduisent ici aux mêmes conclusions. Par défaut, les deux fonctions appliquent une correction de continuité de Yates. Les statistiques de test et les p -values renvoyées par les deux fonctions sont donc très similaires, voire identiques selon la façon dont les données sont fournies.

Le langage diffère légèrement :

- avec `prop.test()`, on raisonne en termes de **comparaison de deux proportions** ;
- avec `chisq.test()`, on raisonne en termes **d'association entre deux variables qualitatives**.

Dans le cas présent, où les deux variables qualitatives possèdent chacune deux modalités, la comparaison de proportions et le test d'indépendance permettent d'aborder la même question. Lorsque l'on compare une réponse binaire entre plusieurs groupes, `prop.test()` reste utilisable. Le test du χ^2 d'indépendance permet, plus généralement, d'étudier l'association entre deux variables qualitatives possédant plusieurs modalités.

Il faut donc retenir l'idée suivante :

- le **cas binaire simple** (succès / échec dans deux groupes) peut être vu comme une **table de contingence 2×2** ;
- les tests sur proportions font donc partie d'un cadre plus général, celui de l'analyse des tableaux d'effectifs ou tables de contingences.

4.5.6 Conditions d'application du test du chi carré

Le test du χ^2 d'indépendance repose sur une approximation. Il ne faut donc pas seulement regarder les effectifs observés dans la table de contingence, mais aussi les **effectifs attendus** sous l'hypothèse nulle d'indépendance. Ces effectifs attendus correspondent aux valeurs que l'on devrait observer dans chaque case si les deux variables qualitatives étaient indépendantes.

On peut les afficher directement à partir du résultat de `chisq.test()` :

```
test_chi_squid <- chisq.test(tab_squid)

test_chi_squid$expected
```

```
      male   female
1 1055.1157 935.8843
3  236.8843 210.1157
```

i Conditions d'application

Pour utiliser un test du χ^2 d'indépendance, on vérifie généralement trois points :

1. les observations doivent être indépendantes. Sauf indication contraire, on considère que c'est le cas si l'échantillonnage a été réalisé correctement ;
2. aucun effectif attendu ne doit être inférieur à 1 ;
3. pas plus de 20 % des effectifs attendus ne doivent être inférieurs à 5.

Lorsque ces conditions sont vérifiées, l'approximation du χ^2 est généralement considérée comme acceptable. Lorsqu'elles ne le sont pas, en revanche, la p-value fournie par le test du χ^2 peut devenir peu fiable. Dans une table 2 x 2, la solution la plus classique consiste alors à utiliser un **test exact de Fisher**, qui ne repose pas sur cette approximation.

4.5.7 Le cas des petits effectifs : test exact de Fisher

Les fonctions `prop.test()` et `chisq.test()` reposent sur des approximations. Lorsque les effectifs sont faibles, ces approximations peuvent devenir médiocres. Dans ce cas, il est préférable d'utiliser le **test exact de Fisher**.

Nous allons reprendre ici les données sur les calmars, mais en comparant cette fois les sites 2 et 4. Le site 4 ne contient que 49 individus, ce qui en fait un bon exemple de situation où un test exact mérite d'être envisagé. Évidemment, avant de faire ce test, il faudrait passer par les étapes préalables d'exploration statistique et graphique, et vérifier les conditions d'application du test du χ^2 . Si on faisait tout ça (n'hésitez pas !), on verrait que le test du χ^2 est adapté dans ce cas également. Mais pour l'exemple, on fait comme si ça n'était pas le cas et on procède donc avec le test exact de Fisher pour en illustrer le fonctionnement.

```
squid_24 <- squid |>
  filter(location %in% c("2", "4")) |>
  mutate(location = fct_drop(location))

tab_squid_small <- table(squid_24$location, squid_24$sex)

tab_squid_small
```

	male	female
2	88	69
4	22	27

On réalise le test exact de Fisher avec la fonction `fisher.test()` à laquelle on fournit la table de contingence :

```
fisher.test(tab_squid_small)
```

Fisher's Exact Test for Count Data

```

data:  tab_squid_small
p-value = 0.1917
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
  0.7813271 3.1504077
sample estimates:
odds ratio
  1.561749

```

Le test exact de Fisher repose, comme son nom l'indique, sur un calcul exact. Il est particulièrement utile lorsque la taille totale de l'échantillon est faible ou lorsque les conditions d'application du test du χ^2 ne sont pas respectées.

Ici, la p -value est supérieure à 0.05, ce qui signifie que l'on ne peut pas rejeter l'hypothèse nulle d'égalité des proportions entre les sites 2 et 4. L'écart observé entre les deux proportions peut donc être attribué au hasard de l'échantillonnage.

Le test fournit aussi une estimation du **ratio des chances** (*odds ratio*). Dans notre table, les lignes correspondent aux sites et les colonnes aux sexes, avec les mâles en première colonne et les femelles en seconde colonne. L'odds ratio compare donc les chances d'observer un mâle plutôt qu'une femelle dans le site 2, aux chances d'observer un mâle plutôt qu'une femelle dans le site 4. Une valeur de 1 indiquerait que ces chances sont identiques dans les deux sites. Une valeur supérieure à 1 indique que les mâles sont relativement plus fréquents dans le premier site que dans le second ; une valeur inférieure à 1 indiquerait l'inverse.

Ici, l'odds ratio estimé vaut environ 1.56. Les chances d'observer un mâle plutôt qu'une femelle sont donc estimées comme environ 1.56 fois plus élevées dans le site 2 que dans le site 4. Cette estimation doit toutefois être interprétée avec prudence, car son intervalle de confiance à 95 % est large et contient la valeur 1. Autrement dit, les données sont compatibles aussi bien avec une différence modérée entre les sites qu'avec une absence de différence réelle. Cette conclusion est cohérente avec la p -value : on ne dispose pas ici d'éléments suffisants pour conclure à une association significative entre le site et le sexe des calmars.

Dans la pratique :

- pour des effectifs suffisants, `prop.test()` ou `chisq.test()` conviennent très bien ;
- pour de petits effectifs, il faut penser à `fisher.test()` ;
- la signification de “petits effectifs” et “d’effectifs suffisants” dépend des conditions d’application du test du χ^2 .

4.6 Comment choisir le bon test ?

On peut résumer les cas les plus fréquents de la façon suivante :

- **un seul échantillon** et une proportion à comparer à une valeur théorique : `binom.test()`
- **deux groupes indépendants** et une variable binaire : `prop.test()`
- **table de contingence** entre deux variables qualitatives : `chisq.test()`
- **petits effectifs** dans une table de contingence : `fisher.test()`

Autrement dit, il faut d’abord se demander :

1. combien de groupes ou de variables sont en jeu ?
2. est-ce que je compare une proportion à une valeur théorique, ou est-ce que je compare plusieurs groupes ?
3. les effectifs sont-ils suffisamment grands pour une approximation, ou faut-il un test exact ?

! À retenir !

Les comparaisons de proportions concernent toujours des variables binaires ou, plus généralement, des tableaux d’effectifs.

Avant de lancer le test :

- identifiez clairement ce qu’est un “succès” et ce qu’est un “échec” ;
- partez des données individuelles quand vous les avez ;
- calculez les effectifs et les proportions observées ;
- examinez l’incertitude des proportions ;

- choisissez un test exact (`binom.test()` ou `fisher.test()`) lorsque les effectifs sont faibles.

Comme pour les comparaisons de moyennes, l'interprétation ne doit jamais reposer uniquement sur la p -value. Les proportions observées, leurs différences, et leurs intervalles d'incertitude sont tout aussi importants.

4.7 Exercices d'application

1. À partir du jeu de données `squid`, testez si la proportion de femelles au site 2 diffère de 50 %.
2. Comparez la proportion de femelles entre les sites 1 et 2. Commencez impérativement par un tableau long, puis construisez le tableau de comptage utile au test.
3. À partir du tableau `squid`, construisez une table de contingence croisant `location` et `sex`, puis commentez l'intérêt d'un mosaic plot pour représenter ce tableau.
4. Pour la comparaison entre les sites 3 et 4, comparez les résultats donnés par `prop.test()` et `fisher.test()`. Pourquoi est-il utile d'examiner les deux ici ?

5 Qualité de l’ajustement

ATTENTION : ÉCRITURE EN COURS. Ne prenez pas pour argent comptant ce qui figure dans les sections 16.12 et suivantes : elles contiennent de nombreuses maladroresses et imprécisions qui seront corrigées prochainement.

5.1 Pré-requis

Comme pour chaque nouveau chapitre, je vous conseille de travailler dans un nouveau script que vous placerez dans votre répertoire de travail, et dans une nouvelle session de travail (Menu **Session** > **Restart R**). Inutile en revanche de créer un nouveau **Rproject** : vous pouvez tout à fait avoir plusieurs scripts dans le même répertoire de travail et pour un même **Rproject**. Comme toujours, consultez les sections du livre du semestre 3 consacrées [aux scripts](#) et [aux projets](#) si vous ne savez plus comment faire.

Si vous êtes dans une nouvelle session de travail (ou que vous avez quitté puis relancé **RStudio**), vous devrez penser à recharger en mémoire les packages utiles. Dans ce chapitre, vous aurez besoin d’utiliser :

- le **tidyverse** (Wickham 2023), qui comprend notamment **readr** (Wickham, Hester, et Bryan 2026), **dplyr** (Wickham, François, et al. 2026), **tidyr** (Wickham, Vaughan, et Girlich 2025) et **ggplot2** (Wickham, Chang, et al. 2026) ;
- **skimr** (Waring et al. 2026), qui permet de calculer des résumés de données très informatifs ;
- **janitor** (Firke 2024), pour nettoyer facilement les noms de variables d’un tableau.

```
library(tidyverse)
library(skimr)
```

```
library(janitor)
```

Vous aurez également besoin des jeux de données suivants que vous pouvez dès maintenant télécharger dans votre répertoire de travail :

- [poissons_recifaux_lune.csv](#)
- [calendrier_lunaire_recifal.csv](#)
- [loups.csv](#)

Enfin, je spécifie ici une fois pour toutes le thème que j'utiliserai pour tous les graphiques de ce chapitre. Libre à vous de choisir un thème différent ou de vous contenter du thème proposé par défaut :

```
theme_set(theme_bw())
```

5.2 Contexte

Dans le chapitre précédent, nous avons comparé des **proportions**. Dans le cas le plus simple, la variable étudiée ne possède que deux modalités : succès / échec, femelle / mâle, vivant / mort, présent / absent, etc. Le test binomial permet alors de comparer une proportion observée à une valeur théorique, et `prop.test()` permet de comparer deux proportions.

Mais de nombreuses questions ne se résument pas à deux catégories. On peut vouloir savoir si :

- les observations sont réparties de façon équilibrée entre plusieurs catégories ;
- une distribution observée correspond à une distribution théorique connue (par exemple une loi de Poisson ou une loi binomiale) ;
- les écarts entre effectifs observés et effectifs attendus sont compatibles avec la fluctuation d'échantillonnage ou si des associations préférentielles existent entre certaines modalités de deux variables qualitatives.

Dans ce chapitre, nous allons donc passer d'une proportion isolée à une **distribution de fréquences entière**.

L'idée générale reste pourtant très proche de celle du chapitre précédent. Il faut toujours comparer ce que l'on observe à ce que l'on aurait attendu si l'hypothèse nulle était vraie. La différence, c'est que cette comparaison ne porte plus sur une seule proportion, mais sur plusieurs effectifs simultanément.

! À retenir !

Un test de qualité de l'ajustement compare une distribution observée d'effectifs à une distribution attendue sous un modèle théorique.

Le raisonnement est toujours le même :

1. définir clairement le modèle attendu sous H_0 ;
2. calculer les effectifs attendus ;
3. mesurer l'écart entre observé et attendu ;
4. décider si cet écart est compatible avec les fluctuations d'échantillonnage.

5.3 Contexte scientifique et données

Nous allons commencer avec un exemple issu d'une étude sur la reproduction d'un poisson récifal, le labre à six bandes, ou Girelle paon de Hardwicke (*Thalassoma hardwicke*), dans le lagon de Moorea en Polynésie française. Chez de nombreux organismes marins, les rythmes lunaires peuvent influencer la reproduction, le développement larvaire, le recrutement et les interactions avec les prédateurs. La lumière nocturne associée aux différentes phases de la lune peut par exemple modifier les conditions rencontrées par les larves dans la colonne d'eau.

Dans une étude consacrée à la phénologie reproductive de ce poisson récifal, Shima et al. (2020b) ont montré que la reproduction n'était pas répartie au hasard au cours du cycle lunaire. Les données brutes associées à cette étude sont disponibles sur Dryad (Shima et al. 2020a). Elles contiennent de nombreuses variables décrivant la croissance journalière des otolithes, les dates de fertilisation estimées, les dates de recrutement, la durée de vie larvaire, les sites de collecte et plusieurs variables lunaires.

Le fichier brut contient plus de 40 000 lignes, car chaque ligne correspond à un incrément de croissance d'un otolithe. Pour ce chapitre, nous n'avons pas besoin d'un tel niveau de détail. Nous allons donc utiliser deux fichiers simplifiés :

- `poissons_recifaux_lune.csv`, qui contient une ligne par poisson ;
- `calendrier_lunaire_recifal.csv`, qui décrit les jours lunaires disponibles pendant la fenêtre temporelle étudiée.

Avant d'importer les données, nous allons définir l'ordre dans lequel nous voulons afficher les phases lunaires. Dans les fichiers `.csv`, les phases sont codées avec des noms courts, sans accents et avec des tirets bas. C'est pratique pour les noms de variables, mais moins lisible dans un tableau ou sur un graphique.

On crée donc deux petits vecteurs :

- `phase_levels`, qui spécifie l'ordre dans lequel on veut ordonner les modalités dans les données (pour les graphiques notamment) ;
- `phase_labels`, qui spécifie la façon dont les noms seront affichés dans les sorties.

```
# Ordre chronologique des phases lunaires
# On reprend exactement les mêmes noms que dans le fichier CSV
# pour que R comprenne qu'il s'agit des mêmes modalités
phase_levels <- c(
  "nouvelle_lune",
  "premier_quartier",
  "pleine_lune",
  "dernier_quartier"
)

# Noms affichés dans les tableaux et les graphiques
phase_labels <- c(
  "Nouvelle lune",
  "Premier quartier",
  "Pleine lune",
  "Dernier quartier"
)
```

On peut maintenant importer le fichier contenant les données des poissons récifaux. Au passage, on transforme `birth_lunar_phase` en facteur ordonné selon les niveaux définis ci-dessus. On transforme aussi `stage` en facteur, car il s'agit d'une variable qualitative.

```
reef_fish <- read_csv(
  "data/poissons_recifaux_lune.csv"
) |>
  mutate(
    birth_lunar_phase = factor(
      birth_lunar_phase,
      levels = phase_levels,
      labels = phase_labels
    ),
    stage = factor(stage)
  )

reef_fish
```

```
# A tibble: 701 x 11
  fish_id stage   site reef birth_date pelagic_larval_durat~1 birth_lunar_day
  <dbl> <fct>   <chr> <dbl> <date>           <dbl>           <dbl>
1      19 settler VME      76 2016-12-16           46           17
2      20 settler VME      76 2016-12-07           48            8
3      21 settler VME      76 2016-12-08           47            9
4      23 settler VOW     NA 2016-12-19           46           20
5      27 settler VOW     NA 2016-12-19           47           20
6      28 settler VOW     NA 2016-12-14           42           15
7      30 settler VME     NA 2016-12-10           47           11
8      33 settler VMW     NA 2016-12-15           45           16
9      37 settler VMW     NA 2016-12-15           47           16
10     39 settler VOW     NA 2016-12-14           49           15
# i 691 more rows
# i abbreviated name: 1: pelagic_larval_duration
# i 4 more variables: birth_lunar_month <dbl>, birth_lunar_phase <fct>,
#   birth_moon_first_half <dbl>, birth_moon_second_half <dbl>
```

Ce premier tableau contient une ligne par poisson. Les variables principales sont :

- `fish_id` : identifiant du poisson ;

- **stage** : étape du cycle de vie du poisson au moment de sa collecte ;
- **site** et **reef** : informations sur le site de collecte ;
- **birth_date** : date de fertilisation estimée ;
- **pelagic_larval_duration** : durée de vie larvaire pélagique ;
- **birth_lunar_day** : jour lunaire correspondant à la date de fertilisation estimée ;
- **birth_lunar_month** : cycle lunaire correspondant à la date de fertilisation estimée ;
- **birth_lunar_phase** : phase lunaire correspondant à la date de fertilisation estimée.

Dans ce jeu de données, l'unité statistique est donc le **poisson** : chaque ligne correspond à un individu échantillonné. Pour chaque poisson, les auteurs ont estimé une **date de fertilisation** à partir de l'âge larvaire reconstruit grâce aux otolithes. Cette date n'a donc pas été observée directement sur le terrain : elle est déduite a posteriori à partir de la croissance de l'individu.

Dans la suite du chapitre, nous ne compterons donc pas directement des fertilisations observées dans la nature. Nous compterons des **poissons**, en les classant selon la phase lunaire correspondant à leur date de fertilisation estimée. Dire qu'il y a 320 poissons dans la phase "premier quartier" signifie donc que 320 poissons du jeu de données ont une date de fertilisation estimée située pendant le premier quartier.

Par simplicité, on pourra parfois parler de "dates de fertilisation estimées", mais il faut garder en tête que les effectifs analysés sont des effectifs de poissons classés selon leur date de fertilisation estimée.

On importe ensuite le calendrier lunaire correspondant à la fenêtre temporelle étudiée. On applique le même recodage à **lunar_phase**, afin que les phases lunaires aient le même ordre et les mêmes noms dans les deux tableaux.

```
moon_calendar <- read_csv(
  "data/calendrier_lunaire_recifal.csv"
) |>
  mutate(
    lunar_phase = factor(
```

```

    lunar_phase,
    levels = phase_levels,
    labels = phase_labels
  )
)

moon_calendar

```

```

# A tibble: 163 x 4
  date      lunar_day lunar_month lunar_phase
<date>      <dbl>      <dbl> <fct>
1 2016-11-10         11        1161 Premier quartier
2 2016-11-11         12        1161 Premier quartier
3 2016-11-12         13        1161 Premier quartier
4 2016-11-13         14        1161 Premier quartier
5 2016-11-14         15        1161 Pleine lune
6 2016-11-15         16        1161 Pleine lune
7 2016-11-16         17        1161 Pleine lune
8 2016-11-17         18        1161 Pleine lune
9 2016-11-18         19        1161 Pleine lune
10 2016-11-19        20        1161 Pleine lune
# i 153 more rows

```

Ce second tableau contient une ligne par jour lunaire disponible dans la période étudiée. Ses variables sont :

- **date** : date calendaire ;
- **lunar_day** : jour dans le cycle lunaire ;
- **lunar_month** : numéro de lunaison ;
- **lunar_phase** : phase lunaire correspondant à cette date.

La variable **lunar_month** correspond au numéro de lunaison selon la numérotation astronomique de Brown (*Brown Lunation Number*). Dans ce système, la lunaison 1 commence avec la première nouvelle lune de 1923. Les valeurs 1161, 1162, etc. ne sont donc pas des mois calendaires classiques, mais des identifiants de cycles lunaires successifs. La variable **lunar_day**, elle, indique le jour au sein de ce cycle lunaire.

Ce calendrier nous permettra de calculer les proportions attendues dans chaque phase lunaire. Autrement dit, nous allons comparer les phases lunaires associées aux dates de ferti-

lisation estimées des poissons aux phases lunaires qui étaient disponibles pendant la période étudiée.

5.4 Le modèle proportionnel

Le modèle de probabilité le plus simple est le **modèle proportionnel**. Dans ce modèle, la fréquence attendue d'un événement est proportionnelle au nombre d'occasions qu'il avait de se produire. Par exemple, si on dispose de deux catégories, et que la première est disponible deux fois plus souvent que l'autre, on s'attend à ce que l'événement d'intérêt se produise deux fois plus souvent dans la première catégorie que dans la seconde, si cet événement se produit effectivement au hasard.

Dans notre exemple, on veut savoir si les dates de fertilisation estimées des poissons sont réparties au hasard au cours du temps, au fil des cycles lunaires, ou si certaines phases lunaires sont au contraire associées à davantage de poissons. Si la phase lunaire n'intervenait pas dans le processus, on pourrait d'abord penser que 25 % des poissons devraient avoir une date de fertilisation estimée pendant la nouvelle lune, 25 % pendant le premier quartier, 25 % pendant la pleine lune et 25 % pendant le dernier quartier. Puisqu'il y a quatre phases lunaires distinctes, il semble donc logique d'attendre un quart des poissons dans chaque phase si le cycle lunaire ne joue aucun rôle dans ce phénomène biologique. Malheureusement, **ce raisonnement simple est erroné** : dans la fenêtre temporelle couverte par l'étude, les quatre phases lunaires ne représentent pas exactement le même nombre de jours. Certaines phases sont disponibles un peu plus longtemps que d'autres. Si une phase lunaire dure plus longtemps dans la période étudiée, elle offre mécaniquement plus d'occasions d'observer un poisson dont la date de fertilisation estimée tombe dans cette phase. Sous l'hypothèse nulle, les dates de fertilisation estimées sont indépendantes du cycle lunaire. Cela ne signifie donc pas que l'on attend exactement le même nombre de poissons dans chaque phase (un quart). Cela signifie plutôt que le nombre attendu de poissons doit être proportionnel au nombre de jours disponibles dans chaque phase lunaire.

Autrement dit, si les dates de fertilisation estimées sont indépendantes du cycle lunaire, une phase lunaire qui représente

30 % des jours disponibles devrait contenir environ 30 % des poissons. Une phase qui représente 20 % des jours disponibles devrait contenir environ 20 % des poissons. C'est ce modèle proportionnel que nous allons tester.

Attention, cette hypothèse est volontairement simple (c'est presque toujours le cas pour les hypothèses nulles). Elle ne tient pas compte des mécanismes biologiques fins, ni du comportement des adultes, ni des différences de survie larvaire entre phases lunaires. Elle nous servira ici de modèle nul.

i Modèle proportionnel

Dans un modèle proportionnel, les effectifs attendus ne sont pas nécessairement identiques dans toutes les catégories. Ils sont proportionnels au nombre d'occasions disponibles dans chaque catégorie.

Les hypothèses testées sont donc les suivantes :

- H_0 : les dates de fertilisation estimées sont indépendantes des phases lunaires. Leur répartition est proportionnelle au nombre de jours disponibles dans chaque phase.
- H_1 : les dates de fertilisation estimées ne sont pas indépendantes des phases lunaires. Leur répartition n'est pas proportionnelle au nombre de jours disponibles dans chaque phase.

Pour tester cette hypothèse, il faut comparer deux grandeurs : les effectifs observés et les effectifs attendus dans chaque phase lunaire.

- Les **effectifs observés** correspondent simplement au nombre de poissons dont la date de fertilisation estimée tombe dans chaque phase lunaire.
- Les **effectifs attendus** correspondent au nombre de poissons que l'on aurait dû obtenir dans chaque phase, si les dates de fertilisation estimées étaient indépendantes du cycle lunaire. Dans ce cas, les poissons devraient se répartir proportionnellement au nombre de jours disponibles dans chaque phase lunaire.

5.5 Exploration statistique des données

L'exploration statistique des données est ici très simple. La seule variable d'intérêt est `birth_lunar_phase`, qui indique la phase lunaire correspondant à la date de fertilisation estimée de chaque individu. On peut commencer par vérifier que cette variable ne contient pas de valeur manquante et que les modalités sont bien codées :

```
reef_fish |>
  skim(birth_lunar_phase)
```

Table 5.1: Data summary

Name	reef_fish
Number of rows	701
Number of columns	11
Column type frequency:	
factor	1
Group variables	None

Variable type: factor

skim_variable	n_missing	n_complete	ordered	unique	top_counts
birth_lunar_phase	0	1	FALSE	4	Pre: 320, Ple: 155, Nou: 152, Der: 74

On compte ensuite le nombre de poissons observés dans chaque phase lunaire, et on calcule les proportions correspondantes :

```
lunar_counts <- reef_fish |>
  count(birth_lunar_phase, name = "observed_count") |>
  mutate(
    observed_prop = observed_count / sum(observed_count),
  )

lunar_counts
```

```
# A tibble: 4 x 3
  birth_lunar_phase observed_count observed_prop
  <fct>              <int>          <dbl>
1 Nouvelle lune      152          0.217
2 Premier quartier   320          0.456
3 Pleine lune        155          0.221
4 Dernier quartier    74          0.106
```

La colonne `observed_count` indique l'effectif observé dans chaque phase lunaire. La colonne `observed_prop` indique la proportion observée de poissons dans chaque phase. À ce stade, on décrit uniquement les données : on constate clairement des déséquilibres entre les phases. Les poissons dont la date de fertilisation estimée tombe pendant le premier quartier semblent environ deux fois plus nombreux que ceux dont la date de fertilisation estimée tombe pendant la nouvelle lune ou la pleine lune. À l'inverse, les poissons dont la date de fertilisation estimée tombe pendant le dernier quartier semblent deux fois moins nombreux que ceux dont la date de fertilisation estimée tombe pendant la nouvelle lune ou la pleine lune. On ne sait pas encore si ces écarts entre phases lunaires sont plus importants que ce que l'on pourrait obtenir par hasard. C'est le test statistique de qualité de l'ajustement qui nous permettra de le vérifier.

Pour construire le modèle attendu sous H_0 , il faut maintenant regarder le calendrier lunaire. Si les dates de fertilisation estimées étaient indépendantes du cycle lunaire, les poissons devraient se répartir proportionnellement au nombre de jours disponibles dans chaque phase.

On compte donc le nombre de jours disponibles dans chaque phase lunaire, et on calcule les proportions correspondantes :

```
phase_opportunities <- moon_calendar |>
  count(lunar_phase, name = "n_days") |>
  mutate(
    expected_prop = n_days / sum(n_days)
  )

phase_opportunities
```

```
# A tibble: 4 x 3
```

	lunar_phase	n_days	expected_prop
	<fct>	<int>	<dbl>
1	Nouvelle lune	35	0.215
2	Premier quartier	44	0.270
3	Pleine lune	42	0.258
4	Dernier quartier	42	0.258

La colonne `n_days` indique le nombre de jours disponibles dans chaque phase lunaire pendant la fenêtre temporelle étudiée. La colonne `expected_prop` indique la proportion de jours correspondant à chaque phase lunaire.

Sous H_0 , ces proportions de jours deviennent les proportions attendues de poissons. Par exemple, si une phase lunaire représente 25 % des jours disponibles, on attend environ 25 % des poissons dans cette phase.

Il reste alors à transformer ces proportions attendues en effectifs attendus. La formule est simple :

$$\text{Effectif attendu} = \text{Proportion attendue} \times \text{Effectif total}$$

On peut donc calculer les effectifs attendus de la façon suivante :

```
lunar_expected <- phase_opportunities |>
  mutate(
    expected_count = expected_prop * nrow(reef_fish)
  )

lunar_expected
```

```
# A tibble: 4 x 4
  lunar_phase    n_days expected_prop expected_count
  <fct>         <int>         <dbl>         <dbl>
1 Nouvelle lune     35         0.215         151.
2 Premier quartier  44         0.270         189.
3 Pleine lune       42         0.258         181.
4 Dernier quartier  42         0.258         181.
```

La colonne `expected_count` indique le nombre de poissons que l'on devrait observer dans chaque phase lunaire si l'hypothèse nulle était vraie. Ces valeurs ne sont pas nécessairement des nombres entiers, car il s'agit d'effectifs théoriques calculés à partir de proportions.

Nous pouvons maintenant réunir les effectifs observés et les effectifs attendus dans un même tableau descriptif :

```
lunar_summary <- lunar_counts |>
  left_join(
    lunar_expected,
    by = c("birth_lunar_phase" = "lunar_phase")
  )

lunar_summary
```

```
# A tibble: 4 x 6
  birth_lunar_phase observed_count observed_prop n_days expected_prop
  <fct>             <int>           <dbl>   <int>         <dbl>
1 Nouvelle lune      152           0.217     35          0.215
2 Premier quartier   320           0.456     44          0.270
3 Pleine lune        155           0.221     42          0.258
4 Dernier quartier    74           0.106     42          0.258
# i 1 more variable: expected_count <dbl>
```

Ici, la jointure n'est pas absolument indispensable : on aurait pu utiliser `bind_cols()` pour coller les deux tableaux. Mais la jointure est plus sûre, car elle fonctionnera même si les lignes ne sont pas ordonnées de la même manière dans les deux tableaux. Elle permet en outre de vérifier que les modalités des phases lunaires sont bien strictement identiques dans les deux tableaux.

`lunar_summary` rassemble les grandeurs nécessaires pour comprendre la comparaison qui sera faite par le test. On y trouve, pour chaque phase lunaire, le nombre de poissons observés, la proportion observée, le nombre de jours disponibles, la proportion attendue et l'effectif attendu.

5.6 Exploration graphique

Avant de réaliser le test, il est toujours utile de représenter les effectifs observés et attendus. Cela permet de voir si certaines catégories semblent très différentes du modèle attendu sous H_0 . Pour chaque phase lunaire, on commence donc par rassembler les effectifs observés et attendus dans une même colonne d'un tableau long :

```
lunar_long <- lunar_summary |>
  select(birth_lunar_phase, observed_count, expected_count) |>
  pivot_longer(
    cols = c(observed_count, expected_count),
    names_to = "type",
    values_to = "count"
  )

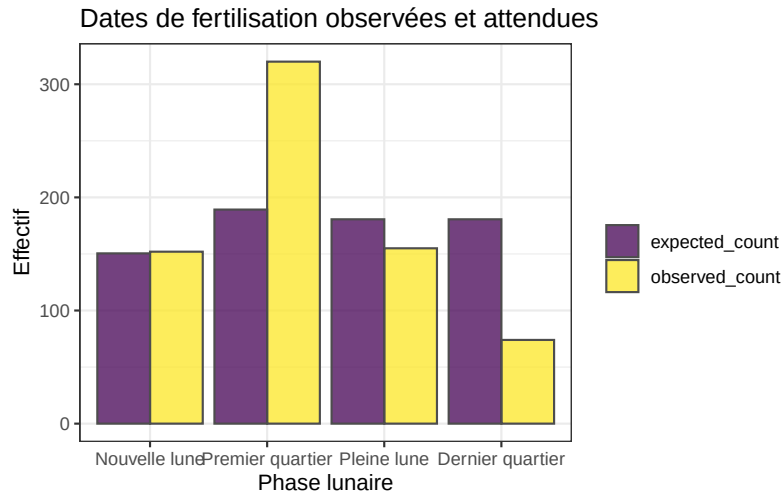
lunar_long
```

```
# A tibble: 8 x 3
  birth_lunar_phase type      count
  <fct>             <chr>    <dbl>
1 Nouvelle lune     observed_count 152
2 Nouvelle lune     expected_count 151.
3 Premier quartier observed_count 320
4 Premier quartier expected_count 189.
5 Pleine lune       observed_count 155
6 Pleine lune       expected_count 181.
7 Dernier quartier observed_count  74
8 Dernier quartier expected_count 181.
```

On représente ensuite ces effectifs sur un diagramme bâtons avec `geom_col()`:

```
lunar_long |>
  ggplot(aes(x = birth_lunar_phase, y = count, fill = type)) +
  geom_col(position = "dodge", alpha = 0.75, color = "grey30") +
  labs(
    x = "Phase lunaire",
    y = "Effectif",
    fill = "",
    title = "Dates de fertilisation observées et attendues"
```

```
) +  
scale_fill_viridis_d()
```



On voit que les poissons ne semblent pas répartis entre les phases lunaires de façon proportionnelle au nombre de jours disponibles dans chaque phase, lorsque l'on considère leur date de fertilisation estimée. Si les écarts semblent faibles entre effectifs observés et attendus pour la nouvelle lune et la pleine lune, le premier quartier est beaucoup plus représenté qu'attendu, alors que le dernier quartier est beaucoup moins représenté. Le graphique ne constitue pas un test. Il permet simplement de décrire la structure des écarts entre les données observées et le modèle attendu. Si jamais le test statistique conclut que les écarts constatés ne sont pas compatibles avec les fluctuations d'échantillonnage, ce graphique permettra de discuter précisément quels écarts sont les plus importants et dans quel sens. Le test statistique ne dira en effet qu'une seule chose : si les données sont compatibles ou non avec le modèle proportionnel. Si le test rejette l'hypothèse nulle, il faudra ensuite examiner la nature des écarts constatés pour comprendre quels mécanismes biologiques peuvent expliquer ces écarts.

5.7 Le test du chi-deux de qualité de l'ajustement

Le test du χ^2 de qualité de l'ajustement compare des effectifs observés à des effectifs attendus sous l'hypothèse nulle.

La statistique de test est :

$$\chi^2 = \sum_i \frac{(\text{Observé}_i - \text{Attendu}_i)^2}{\text{Attendu}_i}$$

Dans cette formule :

- i désigne les différentes catégories ;
- Observé_i est l'effectif observé dans la catégorie i ;
- Attendu_i est l'effectif attendu dans cette même catégorie sous H_0 .

Ici, les catégories sont les quatre phases lunaires. Le test calcule donc un écart standardisé pour chaque phase lunaire, puis additionne ces écarts pour obtenir une statistique globale.

On peut calculer explicitement ces écarts dans un tableau. On le fait ici uniquement pour mieux comprendre le test. Dans la pratique, la fonction `chisq.test()` calcule directement la statistique et la p -value :

```
lunar_fit <- lunar_summary |>
  mutate(
    difference = observed_count - expected_count,
    relative_difference = difference / expected_count,
    contribution = (observed_count - expected_count)^2 / expected_count,
    contribution_prop = contribution / sum(contribution)
  ) |>
  relocate(
    difference,
    relative_difference,
    contribution,
    contribution_prop,
    .after = birth_lunar_phase
  )
```

```
lunar_fit
```

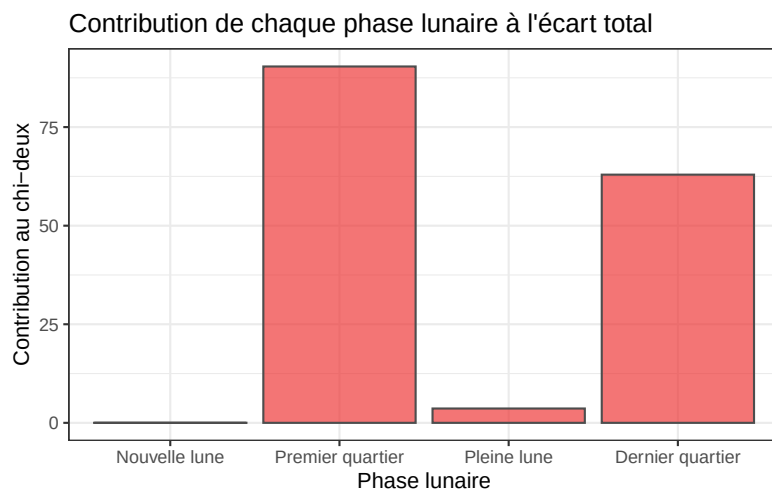
```
# A tibble: 4 x 10
  birth_lunar_phase difference relative_difference contribution
  <fct>             <dbl>             <dbl>             <dbl>
1 Nouvelle lune      1.48              0.00982           0.0145
2 Premier quartier  131.              0.691            90.4
3 Pleine lune       -25.6             -0.142            3.64
4 Dernier quartier -107.             -0.590            62.9
# i 6 more variables: contribution_prop <dbl>, observed_count <int>,
#   observed_prop <dbl>, n_days <int>, expected_prop <dbl>,
#   expected_count <dbl>
```

La colonne **difference** indique l'écart brut entre l'effectif observé et l'effectif attendu. Une valeur positive indique qu'il y a plus de poissons observés qu'attendus dans cette phase lunaire ; une valeur négative indique qu'il y en a moins qu'attendu.

La colonne **relative_difference** exprime le même écart relativement à l'effectif attendu. Elle permet de comparer plus facilement des catégories qui n'ont pas exactement les mêmes effectifs attendus. Enfin, la colonne **contribution** indique la contribution de chaque phase lunaire à la statistique du χ^2 , et **contribution_prop** indique la proportion de l'écart total expliquée par chaque phase. Plus cette contribution est grande, plus la phase lunaire concernée participe à l'écart global entre les données observées et le modèle attendu.

On peut maintenant représenter ces contributions :

```
lunar_fit |>
  ggplot(aes(x = birth_lunar_phase, y = contribution)) +
  geom_col(fill = "firebrick2", alpha = 0.65, color = "grey30") +
  labs(
    x = "Phase lunaire",
    y = "Contribution au chi-deux",
    title = "Contribution de chaque phase lunaire à l'écart total"
  )
```



Plus la valeur de χ^2 est grande, plus l'écart entre les effectifs observés et attendus est important. La p-value indique ensuite si un écart au moins aussi grand serait plausible si l'hypothèse nulle était vraie.

Dans R, le test se réalise avec la fonction `chisq.test()`. Comme le package `janitor` possède également une fonction nommée `chisq.test()`, nous allons appeler explicitement la fonction du package `stats`, qui est chargé automatiquement avec R :

```
test_lunar <- stats::chisq.test(
  x = lunar_summary$observed_count,
  p = lunar_summary$expected_prop
)

test_lunar
```

Chi-squared test for given probabilities

```
data:  lunar_summary$observed_count
X-squared = 156.97, df = 3, p-value < 2.2e-16
```

Ici, la p-value est très petite. On rejette donc l'hypothèse nulle : les dates de fertilisation estimées ne sont pas compatibles avec une répartition simplement proportionnelle au nombre de jours de chaque phase lunaire.

Cela ne signifie pas que la lune “détermine” à elle seule la reproduction. C’est une conclusion importante. Le test nous dit seulement que le modèle nul proposé ne décrit pas correctement les effectifs observés. L’écart peut venir de la phénologie reproductive des adultes, de la survie différentielle des larves, des conditions de recrutement, ou de plusieurs de ces mécanismes à la fois.

5.8 Degrés de liberté et p -value

Dans les résultats du test ci-dessus, on note que le nombre de degrés de liberté est égal à 3 ($df = 3$). Pour un test du χ^2 de qualité de l’ajustement, le nombre de degrés de liberté ne doit pas être accepté sans réfléchir. Il doit être vérifié et se calcule de la façon suivante :

$$df = k - 1 - m$$

où k est le nombre de catégories et m le nombre de paramètres estimés à partir des données pour construire les effectifs attendus.

Dans notre exemple, les proportions attendues, sous l’hypothèse nulle d’un modèle proportionnel, sont déterminées par le calendrier lunaire et par la durée de la fenêtre d’étude. Elles ne sont pas estimées à partir des effectifs observés de poissons. On a donc $k = 4$ phases lunaires et $m = 0$ paramètre estimé à partir des effectifs observés, soit un nombre de degrés de liberté égal à :

$$4 - 1 - 0$$

[1] 3

Puisque le nombre de degrés de liberté utilisé par `chisq.test()` correspond bien au nombre de degrés de liberté correct, la p -value du test est correcte également, et nos conclusions sont valides. Si en revanche le bon nombre de degrés de liberté n’avait pas été utilisé par `chisq.test()`, il aurait fallu recalculer la p -value manuellement. Nous aurons l’occasion de voir comment faire plus loin.

Attention

Un test de qualité de l'ajustement teste toujours l'ajustement de données observées à un modèle précis. Si le modèle nul est mal choisi, la p -value peut être très petite sans que l'interprétation biologique soit claire.

5.9 Conditions d'application

Le test du χ^2 repose sur une approximation. Pour que cette approximation soit correcte, les effectifs attendus ne doivent pas être trop faibles. Les règles usuelles sont les suivantes :

- aucun effectif attendu ne devrait être inférieur à 1 ;
- pas plus de 20 % des effectifs attendus ne devraient être inférieurs à 5.

Dans notre exemple, on peut examiner les effectifs attendus directement :

```
lunar_fit |>
  select(birth_lunar_phase, expected_count)

# A tibble: 4 x 2
  birth_lunar_phase expected_count
  <fct>             <dbl>
1 Nouvelle lune    151.
2 Premier quartier 189.
3 Pleine lune      181.
4 Dernier quartier 181.
```

Ici, tous les effectifs attendus sont largement supérieurs à 5. Les conditions d'application du test sont donc satisfaites.

Si ce n'était pas le cas, le test ne pourrait pas être réalisé immédiatement. Il faudrait essayer de regrouper des catégories voisines, à condition que ce regroupement ait un sens biologique.

5.10 Magnitude de l'effet

Comme toujours, la p -value ne mesure pas la taille de l'effet : elle indique seulement si les données observées sont compatibles avec l'hypothèse nulle, pas si l'écart observé est biologiquement important.

Pour évaluer la magnitude de l'écart, il faut revenir aux catégories elles-mêmes. Ici, trois informations sont particulièrement utiles :

- les différences entre effectifs observés et attendus ;
- les différences relatives entre effectifs observés et attendus ;
- les contributions de chaque catégorie au χ^2 .

On peut extraire les colonnes les plus utiles du tableau `lunar_fit` :

```
lunar_fit |>
  select(
    birth_lunar_phase,
    observed_count,
    expected_count,
    difference,
    relative_difference,
    contribution_prop
  )
```

```
# A tibble: 4 x 6
  birth_lunar_phase observed_count expected_count difference relative_difference
<fct>             <int>         <dbl>         <dbl>         <dbl>
1 Nouvelle lune      152           151.          1.48          0.00982
2 Premier quartier   320           189.         131.          0.691
3 Pleine lune        155           181.        -25.6         -0.142
4 Dernier quartier    74           181.       -107.         -0.590
# i 1 more variable: contribution_prop <dbl>
```

L'écart au modèle proportionnel n'est pas réparti de façon homogène entre les phases lunaires. Le premier quartier présente un excès net de poissons par rapport à l'effectif attendu : davantage de poissons que prévu ont une date de fertilisation estimée pendant cette phase. À l'inverse, le dernier quartier

présente un déficit net : moins de poissons que prévu ont une date de fertilisation estimée pendant cette phase.

Les écarts observés pour la nouvelle lune et la pleine lune sont plus faibles. Ils participent donc beaucoup moins à l'écart global mesuré par le test du χ^2 . L'effet détecté par le test correspond donc surtout à une surreprésentation du premier quartier et à une sous-représentation du dernier quartier, plutôt qu'à un décalage uniforme de toutes les phases lunaires.

5.11 Le cas de deux catégories

Le test du χ^2 de qualité de l'ajustement fonctionne aussi avec seulement deux catégories. Dans ce cas, il devient très proche du test binomial vu au chapitre précédent.

Reprenons l'exemple précédent, mais posons une question binaire : la proportion de poissons dont la date de fertilisation estimée tombe pendant le premier quartier est-elle compatible avec la proportion de jours de premier quartier dans la période étudiée ?

- H_0 : la proportion de poissons dont la date de fertilisation estimée tombe pendant le premier quartier est égale à la proportion de jours de premier quartier dans la période étudiée.
- H_1 : la proportion de poissons dont la date de fertilisation estimée tombe pendant le premier quartier est différente de cette proportion attendue.

Commençons par transformer la question en deux catégories :

```
# Calcul de la proportion attendue de fertilisation pendant le premier quartier
expected_prop_first_quarter <- phase_opportunities |>
  filter(lunar_phase == "Premier quartier") |>
  pull(expected_prop)

expected_prop_first_quarter
```

```
[1] 0.2699387
```

```

# Tableau synthétique
first_quarter_fit <- reef_fish |>
  mutate(
    birth = if_else(
      birth_lunar_phase == "Premier quartier",
      "Premier quartier",
      "Autres phases"
    )
  ) |>
  count(birth, name = "observed") |>
  mutate(
    expected_prop = if_else(
      birth == "Premier quartier",
      expected_prop_first_quarter,
      1 - expected_prop_first_quarter
    )
  )

first_quarter_fit

```

A tibble: 2 x 3

	birth <chr>	observed <int>	expected_prop <dbl>
1	Autres phases	381	0.730
2	Premier quartier	320	0.270

Le test du χ^2 de qualité de l'ajustement s'écrit :

```

stats::chisq.test(
  x = first_quarter_fit$observed,
  p = first_quarter_fit$expected_prop
)

```

Chi-squared test for given probabilities

```

data: first_quarter_fit$observed
X-squared = 123.79, df = 1, p-value < 2.2e-16

```

On peut comparer ce résultat à celui du test binomial exact :

```
stats::binom.test(  
  x = 320,  
  n = 701, # 381 + 320  
  p = expected_prop_first_quarter  
)
```

Exact binomial test

```
data: 320 and 701  
number of successes = 320, number of trials = 701, p-value < 2.2e-16  
alternative hypothesis: true probability of success is not equal to 0.2699387  
95 percent confidence interval:  
 0.4191546 0.4941945  
sample estimates:  
probability of success  
 0.4564907
```

Les deux tests conduisent ici à la même conclusion : la proportion de poissons dont la date de fertilisation estimée tombe pendant le premier quartier est nettement supérieure à la proportion attendue si les dates de fertilisation estimées étaient indépendantes du cycle lunaire.

La proportion observée est donc d'environ 45.6 %, avec une proportion réelle dans la population générale probablement comprise entre 41.9% et 49.4% (comme l'indique l'intervalle de confiance à 95% de cette proportion), alors que la proportion attendue de jours de premier quartier est d'environ 27 %. L'intérêt principal du test binomial est qu'il fournit directement un intervalle de confiance sur la proportion observée.

! À retenir !

Avec seulement deux catégories, le test du χ^2 de qualité de l'ajustement et le test binomial répondent à une question très proche.

En pratique, si la question porte sur une proportion unique et que vous disposez d'un ordinateur, `binom.test()` reste souvent préférable : il fournit un test exact et un intervalle de confiance directement interprétable. En revanche, si plus de 2 catégories vous

intéressant, le test de χ^2 de qualité de l'ajustement est le seul possible.

5.12 Ajustement à une loi binomiale

Le modèle proportionnel que nous venons d'utiliser permettait de comparer des effectifs observés à des effectifs attendus proportionnels au nombre d'occasions disponibles dans chaque catégorie. On peut aussi utiliser un test de qualité de l'ajustement pour comparer une distribution observée à une **loi de probabilité** particulière.

Commençons par la loi binomiale. L'exemple utilisé ici porte sur la composition de familles de deux enfants, à partir de données présentées par Rodgers et Doughty (2001). Pour chaque famille, on connaît le nombre de garçons parmi les deux enfants.

```
two_child_families <- tibble(  
  n_boys = 0:2,  
  observed_count = c(530, 1332, 582)  
)  
  
two_child_families
```

```
# A tibble: 3 x 2  
  n_boys observed_count  
  <int>         <dbl>  
1     0           530  
2     1          1332  
3     2           582
```

L'unité statistique est ici la **famille**. La variable étudiée est `n_boys`, c'est-à-dire le nombre de garçons dans une famille de deux enfants. Cette variable peut prendre seulement trois valeurs :

- 0 : aucun garçon, donc deux filles ;
- 1 : un garçon et une fille ;
- 2 : deux garçons.

À ce stade, il ne s'agit donc pas de tester une proportion unique, comme dans le chapitre précédent. On ne demande pas simplement si la proportion totale de garçons est égale à 0.5. On demande si la **distribution complète** du nombre de garçons par famille correspond à ce que prédit un modèle binomial.

La loi binomiale est adaptée ici parce que chaque famille de deux enfants peut être vue comme une série de deux essais :

- le nombre d'essais est fixé à l'avance : chaque famille contient exactement deux enfants ;
- chaque essai a deux issues possibles : garçon ou fille ;
- on s'intéresse au nombre de succès, ici le nombre de garçons ;
- sous le modèle binomial, la probabilité d'avoir un garçon est la même pour chaque enfant ;
- sous le modèle binomial, les issues sont indépendantes les unes des autres.

Si ces conditions sont raisonnables, le nombre de garçons dans une famille de deux enfants peut être décrit par une loi binomiale :

$$X \sim \mathcal{B}(n = 2, p)$$

où X est le nombre de garçons dans une famille, n est le nombre d'enfants par famille, et p est la probabilité qu'un enfant soit un garçon.

Les hypothèses testées sont donc les suivantes :

- H_0 : le nombre de garçons dans les familles de deux enfants suit une loi binomiale.
- H_1 : le nombre de garçons dans les familles de deux enfants ne suit pas une loi binomiale.

Pour calculer les effectifs attendus, il faut connaître les probabilités attendues pour les trois catégories : 0 garçon, 1 garçon et 2 garçons. Ces probabilités sont données par la loi binomiale.

Il reste toutefois un point important : la valeur de p n'est pas donnée dans l'hypothèse nulle. Nous devons donc l'estimer à

partir des données. Pour cela, on calcule la proportion totale de garçons dans l'ensemble des familles :

```
n_children <- sum(two_child_families$observed_count) * 2
n_boys_observed <- sum(
  two_child_families$n_boys * two_child_families$observed_count
)

p_boy <- n_boys_observed / n_children

p_boy
```

```
[1] 0.5106383
```

La valeur de `p_boy` est la proportion observée de garçons parmi tous les enfants de l'échantillon. On l'utilise comme estimation de p dans la loi binomiale.

On peut maintenant calculer les probabilités attendues avec `dbinom()` :

```
two_child_fit <- two_child_families |>
  mutate(
    expected_prob = dbinom(
      x = n_boys,
      size = 2,
      prob = p_boy
    )
  )

two_child_fit
```

```
# A tibble: 3 x 3
```

	n_boys	observed_count	expected_prob
	<int>	<dbl>	<dbl>
1	0	530	0.239
2	1	1332	0.500
3	2	582	0.261

La colonne `expected_prob` contient les probabilités attendues sous le modèle binomial. Par exemple, la probabilité attendue d'avoir exactement un garçon dans une famille de

deux enfants est calculée par la loi binomiale avec `size = 2` et `prob = p_boy`.

Pour obtenir les effectifs attendus, on multiplie ensuite ces probabilités par le nombre total de familles :

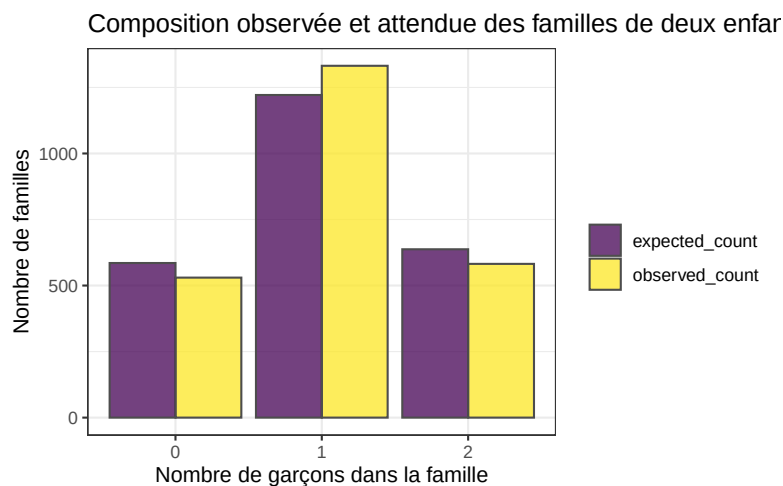
```
two_child_fit <- two_child_fit |>
  mutate(
    expected_count = sum(observed_count) * expected_prob
  )

two_child_fit
```

```
# A tibble: 3 x 4
  n_boys observed_count expected_prob expected_count
  <int>      <dbl>         <dbl>         <dbl>
1     0         530         0.239         585.
2     1        1332         0.500        1221.
3     2         582         0.261         637.
```

Le tableau compare maintenant les effectifs observés et les effectifs attendus sous le modèle binomial. On peut représenter cette comparaison graphiquement :

```
two_child_fit |>
  select(n_boys, observed_count, expected_count) |>
  pivot_longer(
    cols = c(observed_count, expected_count),
    names_to = "type",
    values_to = "count"
  ) |>
  ggplot(aes(x = factor(n_boys), y = count, fill = type)) +
  geom_col(position = "dodge", alpha = 0.75, color = "grey30") +
  labs(
    x = "Nombre de garçons dans la famille",
    y = "Nombre de familles",
    fill = "",
    title = "Composition observée et attendue des familles de deux enfants"
  ) +
  scale_fill_viridis_d()
```



Le graphique suggère que les familles ayant un garçon et une fille sont plus nombreuses qu'attendu sous le modèle binomial. À l'inverse, les familles ayant deux enfants du même sexe sont moins nombreuses qu'attendu.

On peut maintenant calculer les écarts et les contributions au χ^2 :

```
two_child_fit <- two_child_fit |>
  mutate(
    difference = observed_count - expected_count,
    contribution = (observed_count - expected_count)^2 / expected_count
  ) |>
  relocate(difference, contribution, .after = n_boys)

two_child_fit
```

A tibble: 3 x 6

	n_boys	difference	contribution	observed_count	expected_prob	expected_count
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0	-55.3	5.22	530	0.239	585.
2	1	111.	10.0	1332	0.500	1221.
3	2	-55.3	4.79	582	0.261	637.

Le test du χ^2 de qualité de l'ajustement s'écrit alors :

```
test_two_child <- stats::chisq.test(
  x = two_child_fit$observed_count,
  p = two_child_fit$expected_prob
```

```
)
test_two_child
```

Chi-squared test for given probabilities

```
data: two_child_fit$observed_count
X-squared = 20.021, df = 2, p-value = 4.492e-05
```

Le test indique un écart net entre les effectifs observés et les effectifs attendus sous le modèle binomial. Toutefois, il faut ici faire attention au nombre de degrés de liberté. La fonction `chisq.test()` utilise trois catégories, et renvoie donc par défaut :

$$df = 3 - 1 = 2$$

Mais ce n'est pas le bon nombre de degrés de liberté, car nous avons estimé un paramètre à partir des données : la probabilité p d'avoir un garçon. Il faut donc retirer un degré de liberté supplémentaire :

$$df = k - 1 - m = 3 - 1 - 1 = 1$$

La p -value doit donc être recalculée avec ce nombre de degrés de liberté :

```
p_value_two_child <- stats::pchisq(
  test_two_child$statistic,
  df = 1,
  lower.tail = FALSE
)

p_value_two_child
```

```
X-squared
7.657994e-06
```

La conclusion reste la même : on rejette l'hypothèse nulle. La distribution du nombre de garçons dans les familles de deux enfants ne correspond pas à celle attendue sous une loi binomiale simple.

Cela signifie qu'au moins une des hypothèses du modèle binomial n'est probablement pas vérifiée. La probabilité d'avoir un garçon peut varier d'une famille à l'autre, les sexes des enfants d'une même famille peuvent ne pas être strictement indépendants, ou l'échantillon de familles de deux enfants peut être influencé par des décisions familiales d'arrêt ou de poursuite de la reproduction. Par exemple, certaines familles ayant deux enfants du même sexe peuvent décider d'avoir un troisième enfant, ce qui les retire alors de l'ensemble des familles de deux enfants.

! À retenir !

La loi binomiale est adaptée lorsque l'on étudie un nombre de succès obtenu dans un nombre fixé d'essais indépendants, avec la même probabilité de succès à chaque essai.

Dans cet exemple, on ne teste pas une proportion unique. On teste si la distribution complète du nombre de garçons par famille correspond à celle attendue sous une loi binomiale.

5.13 Ajustement à une loi de Poisson

Le test de qualité de l'ajustement ne sert pas seulement à comparer des effectifs observés à des proportions théoriques fixées à l'avance. Il permet aussi de tester si des données de comptage suivent une distribution de probabilité particulière.

La loi de Poisson est particulièrement importante pour les variables de comptage. Elle peut décrire le nombre d'événements observés dans une unité d'espace ou de temps lorsque :

- les événements sont indépendants les uns des autres ;
- la probabilité d'un événement est constante dans l'espace ou dans le temps ;
- les événements ne se regroupent pas en paquets.

Nous allons utiliser le fichier `louples.csv`, qui contient des données sur le nombre de louveteaux observés dans différentes portées. Les variables disponibles sont :

- `inbreedCoef` : coefficient de consanguinité ;
- `nPups` : nombre de louveteaux dans la portée.

```
wolves <- read_csv("data/louples.csv", show_col_types = FALSE)

wolves

# A tibble: 24 x 2
  inbreedCoef nPups
      <dbl> <dbl>
1         0         6
2         0         6
3        0.13         7
4        0.13         5
5        0.13         4
6        0.19         8
7        0.19         7
8        0.19         4
9        0.25         6
10       0.24         3
# i 14 more rows
```

Ici, nous allons nous concentrer uniquement sur `nPups`, c'est-à-dire sur le nombre de louveteaux par portée. La question posée est la suivante : la distribution observée du nombre de louveteaux par portée est-elle compatible avec une loi de Poisson ?

Les hypothèses sont donc :

- H_0 : le nombre de louveteaux par portée suit une loi de Poisson.
- H_1 : le nombre de louveteaux par portée ne suit pas une loi de Poisson.

Commençons par quelques statistiques descriptives :

```
wolves |>
  summarise(
    n = n(),
```

```

    mean_pups = mean(nPups),
    var_pups = var(nPups),
    ratio_var_mean = var(nPups) / mean(nPups)
  )

```

```

# A tibble: 1 x 4
      n mean_pups var_pups ratio_var_mean
<int>   <dbl>   <dbl>         <dbl>
1     24     3.96     3.52         0.889

```

Pour une loi de Poisson, la moyenne et la variance sont égales. Le rapport variance / moyenne donne donc une première indication :

- s'il est proche de 1, la distribution est compatible avec une loi de Poisson ;
- s'il est nettement supérieur à 1, les données sont plus agrégées que prévu ;
- s'il est nettement inférieur à 1, les données sont plus régulières que prévu.

Ici, ce rapport est proche de 1, ce qui suggère que la loi de Poisson n'est pas une hypothèse absurde. Mais ce n'est qu'un examen descriptif. Il faut maintenant comparer les effectifs observés et attendus.

```

pup_counts <- wolves |>
  count(nPups, name = "observed") |>
  complete(nPups = 0:max(nPups), fill = list(observed = 0))

pup_counts

```

```

# A tibble: 9 x 2
  nPups observed
<dbl>   <int>
1     0         0
2     1         1
3     2         4
4     3         8
5     4         3
6     5         2
7     6         3

```

8	7	2
9	8	1

Pour calculer les effectifs attendus sous une loi de Poisson, nous devons connaître le paramètre de cette loi. Pour une loi de Poisson, ce paramètre est la moyenne, souvent notée λ . Comme il n'est pas connu, nous allons l'estimer à partir des données :

```
lambda_pups <- mean(wolves$nPups)

lambda_pups
```

```
[1] 3.958333
```

Nous pouvons maintenant calculer les probabilités attendues grâce à la fonction `dpois()` :

```
pup_fit <- pup_counts |>
  mutate(
    expected_prob = dpois(nPups, lambda = lambda_pups),
    expected = nrow(wolves) * expected_prob
  )

pup_fit
```

```
# A tibble: 9 x 4
  nPups observed expected_prob expected
<dbl>   <int>         <dbl>     <dbl>
1     0       0         0.0191     0.458
2     1       1         0.0756     1.81
3     2       4         0.150      3.59
4     3       8         0.197      4.74
5     4       3         0.195      4.69
6     5       2         0.155      3.71
7     6       3         0.102      2.45
8     7       2         0.0577     1.38
9     8       1         0.0285     0.685
```

Les effectifs attendus sont faibles pour plusieurs valeurs du nombre de louveteaux. Si l'on utilisait directement toutes les

catégories, les conditions d'application du test du χ^2 ne seraient pas bien respectées. Dans ce cas, une solution classique consiste à regrouper des catégories voisines, à condition que ce regroupement ait du sens.

Ici, nous allons distinguer les portées petites (0-2 louveteaux), intermédiaires (3-4 louveteaux), et grandes (5+ louveteaux) :

```
pup_fit_grouped <- pup_fit |>
  mutate(
    class = case_when(
      nPups <= 2 ~ "0-2",
      nPups <= 4 ~ "3-4",
      nPups >= 5 ~ "5+"
    )
  ) |>
  summarise(
    observed = sum(observed),
    expected = sum(expected),
    .by = class
  ) |>
  mutate(class = factor(class, levels = c("0-2", "3-4", "5+"))) |>
  arrange(class)

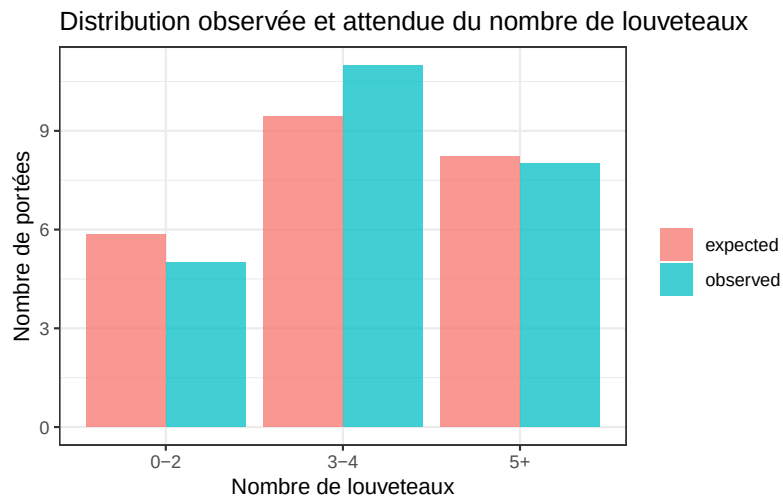
pup_fit_grouped
```

```
# A tibble: 3 x 3
  class observed expected
<fct>    <int>    <dbl>
1 0-2         5     5.86
2 3-4        11     9.42
3 5+         8     8.23
```

La comparaison graphique est souvent la meilleure façon de comprendre le test :

```
pup_fit_grouped |>
  pivot_longer(
    cols = c(observed, expected),
    names_to = "type",
    values_to = "count"
  ) |>
```

```
ggplot(aes(x = class, y = count, fill = type)) +
  geom_col(position = "dodge", alpha = 0.75) +
  labs(
    x = "Nombre de louveteaux",
    y = "Nombre de portées",
    fill = "",
    title = "Distribution observée et attendue du nombre de louveteaux"
  )
```



Les écarts entre effectifs observés et attendus semblent faibles. On peut maintenant réaliser le test. Comme nous avons estimé λ à partir des données, nous perdons un degré de liberté. La fonction `chisq.test()` ne le sait pas. Elle calcule donc une p-value avec un nombre de degrés de liberté trop grand. Nous allons utiliser la statistique du χ^2 renvoyée par le test, puis recalculer nous-mêmes la p-value avec le bon nombre de degrés de liberté.

```
test_pups <- stats::chisq.test(
  x = pup_fit_grouped$observed,
  p = pup_fit_grouped$expected / sum(pup_fit_grouped$expected)
)

test_pups
```

Chi-squared test for given probabilities

```
data: pup_fit_grouped$observed
X-squared = 0.37877, df = 2, p-value = 0.8275
```

Le nombre correct de degrés de liberté est :

$$df = k - 1 - m$$

où k est le nombre de catégories après regroupement et m le nombre de paramètres estimés à partir des données. Ici, $k = 3$ et $m = 1$, car nous avons estimé λ . On obtient donc :

```
df_pups <- nrow(pup_fit_grouped) - 1 - 1
df_pups
```

```
[1] 1
```

La p-value correcte est alors :

```
chi2_pups <- sum(
  (pup_fit_grouped$observed - pup_fit_grouped$expected)^2 /
  pup_fit_grouped$expected
)
1 - pchisq(chi2_pups, df = df_pups)
```

```
[1] 0.5288993
```

Ici, la p-value est grande. On ne rejette donc pas l'hypothèse nulle. La distribution du nombre de louveteaux par portée n'est pas significativement différente de celle attendue sous une loi de Poisson.

Cette conclusion doit toutefois rester prudente. L'échantillon ne contient que 24 portées, ce qui limite fortement la puissance du test. Ne pas rejeter H_0 ne signifie donc pas que la loi de Poisson est vraie. Cela signifie seulement que, avec ces données, nous n'avons pas de preuve claire d'un écart à cette loi.

! Important 1: Estimation des paramètres et degrés de liberté

Quand les probabilités attendues sont entièrement fixées par l'hypothèse nulle, le nombre de degrés de liberté vaut :

$$df = k - 1$$

où k est le nombre de catégories.

Quand un ou plusieurs paramètres sont estimés à partir des données, il faut retirer un degré de liberté supplémentaire pour chaque paramètre estimé :

$$df = k - 1 - m$$

Cette correction est indispensable pour obtenir la bonne p-value.

5.14 Comment choisir et interpréter un test de qualité de l'ajustement ?

On peut résumer les cas les plus fréquents de la façon suivante :

- **une distribution catégorielle** à comparer à des proportions fixées à l'avance : `chisq.test(x, p = ...)`
- **deux catégories seulement**, avec une proportion à comparer à une valeur théorique : `binom.test()`, ou `chisq.test()` si les effectifs sont grands
- **une variable de comptage** à comparer à une loi de Poisson : calcul des probabilités attendues avec `dpois()`, puis test du χ^2
- **une distribution de nombres de succès** à comparer à une loi binomiale : calcul des probabilités attendues avec `dbinom()`, puis test du χ^2

Dans tous les cas, il faut examiner les données avant le test. En particulier :

1. les catégories doivent avoir un sens biologique ou expérimental ;

2. les effectifs attendus doivent être suffisamment grands ;
3. les regroupements éventuels doivent être décidés de façon raisonnable ;
4. les paramètres estimés à partir des données doivent être pris en compte dans les degrés de liberté ;
5. l'interprétation doit porter sur le modèle testé, et pas sur une idée vague de “bonne” ou “mauvaise” distribution.

 Attention

Un test non significatif ne prouve pas que le modèle testé est vrai.

Il indique seulement que l'écart entre les données et le modèle n'est pas suffisamment grand pour rejeter l'hypothèse nulle, compte tenu de la taille de l'échantillon et des catégories utilisées.

5.15 Conclusion

Les tests de qualité de l'ajustement prolongent directement les tests sur proportions. Dans les deux cas, il s'agit de comparer des effectifs observés à des effectifs attendus sous un modèle nul.

La nouveauté de ce chapitre est que le modèle nul peut maintenant porter sur une distribution entière :

- distribution proportionnelle entre plusieurs catégories ;
- distribution binomiale ;
- distribution de Poisson ;
- plus généralement, toute distribution pour laquelle on peut calculer des probabilités attendues.

Comme toujours, le test ne suffit pas. Il faut aussi examiner les effectifs observés, les effectifs attendus, les contributions au χ^2 , les conditions d'application, et la magnitude des écarts observés.

5.16 Exercices d'application

1. À partir du jeu de données `poissons_recifaux_lune.csv`, testez si les dates de fertilisation estimées sont réparties de façon égale entre les quatre phases lunaires. Comparez ce test avec le modèle proportionnel au nombre de jours de chaque phase lunaire présenté dans ce chapitre.
2. À partir du jeu de données `poissons_recifaux_lune.csv`, regroupez les jours lunaires en deux périodes seulement : du jour 0 au jour 14, puis du jour 15 au jour 29. Les dates de fertilisation estimées sont-elles réparties proportionnellement au nombre de jours de ces deux périodes ?
3. Reprenez le jeu de données `loups.csv`. Réalisez le test d'ajustement à une loi de Poisson sans regrouper les catégories. Examinez les effectifs attendus. Pourquoi ce test n'est-il pas satisfaisant ?
4. Toujours avec `loups.csv`, proposez un autre regroupement des nombres de louveteaux. Recalculez le test et comparez la conclusion à celle du chapitre.
5. À partir du jeu de données `HornedLizards.csv`, testez si les lézards vivants et tués par prédation sont également fréquents dans l'échantillon. Comparez les résultats de `chisq.test()` et `binom.test()`.

Références

- Dorai-Raj, Sundar, et Arianna D. Cascone. 2026. *binom: Binomial Confidence Intervals for Several Parameterizations*. <https://doi.org/10.32614/CRAN.package.binom>.
- Firke, Sam. 2024. *janitor: Simple Tools for Examining and Cleaning Dirty Data*. <https://github.com/sfirke/janitor>.
- Fox, John, Sanford Weisberg, et Brad Price. 2026. *car: Companion to Applied Regression*. https://github.com/bpric/e2652/car_repo.
- Fuller, Andrea, Peter R. Kamerman, Shane K. Maloney, Graham Mitchell, et Duncan Mitchell. 2003. « Variability in brain and arterial blood temperatures in free-ranging ostriches in their natural habitat ». *Journal of Experimental Biology* 206 (7): 1171-81. <https://doi.org/10.1242/jeb.00230>.
- Hasselquist, Dennis, James A. Marsh, Paul W. Sherman, et John C. Wingfield. 1999. « Is avian humoral immunocompetence suppressed by testosterone? » *Behavioral Ecology and Sociobiology* 45 (3): 167-75. <https://doi.org/10.1007/s002650050550>.
- Horst, Allison, Alison Hill, et Kristen Gorman. 2022. *palmer-penguins: Palmer Archipelago (Antarctica) Penguin Data*. <https://allisonhorst.github.io/palmerpenguins/>.
- Jeppson, Haley, Heike Hofmann, et Di Cook. 2026. *ggmosaic: Mosaic Plots in the ggplot2 Framework*. <https://haleyjeppson.github.io/ggmosaic/>.
- Rodgers, Joseph Lee, et Debby Doughty. 2001. « Does Having Boys or Girls Run in the Family? » *Chance* 14 (4): 8-13. <https://doi.org/10.1080/09332480.2001.10542293>.
- Shima, Jeffrey S., Craig W. Osenberg, Erik G. Noonburg, Suzanne H. Alonzo, et Stephen E. Swearer. 2020a. « Lunar Rhythms in Growth of Larval Fish ». Dryad. <https://doi.org/10.5061/dryad.zgmsbcc93>.
- . 2020b. « Reproductive Phenology across the Lunar Cycle: Parental Decisions, Offspring Responses, and Consequences for Reef Fish ». *Ecology* 101 (9): e03086. <https://doi.org/10.1002/ecy.3086>.

- Waring, Elin, Michael Quinn, Amelia McNamara, Eduardo Arino de la Rubia, Hao Zhu, et Shannon Ellis. 2026. *skimr: Compact and Flexible Summaries of Data*. <https://docs.ropensci.org/skimr/>.
- Wickham, Hadley. 2023. *tidyverse: Easily Install and Load the Tidyverse*. <https://tidyverse.tidyverse.org>.
- Wickham, Hadley, et Jennifer Bryan. 2025. *readxl: Read Excel Files*. <https://readxl.tidyverse.org>.
- Wickham, Hadley, Winston Chang, Lionel Henry, Thomas Lin Pedersen, Kohske Takahashi, Claus Wilke, Kara Woo, Hiroaki Yutani, Dewey Dunnington, et Teun van den Brand. 2026. *ggplot2: Create Elegant Data Visualisations Using the Grammar of Graphics*. <https://ggplot2.tidyverse.org>.
- Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, et Davis Vaughan. 2026. *dplyr: A Grammar of Data Manipulation*. <https://dplyr.tidyverse.org>.
- Wickham, Hadley, Jim Hester, et Jennifer Bryan. 2026. *readr: Read Rectangular Text Data*. <https://readr.tidyverse.org>.
- Wickham, Hadley, Thomas Lin Pedersen, et Dana Seidel. 2025. *scales: Scale Functions for Visualization*. <https://scales.r-lib.org>.
- Wickham, Hadley, Davis Vaughan, et Maximilian Girlich. 2025. *tidyr: Tidy Messy Data*. <https://tidyr.tidyverse.org>.
- Young, Kevin V., Edmund D. Brodie, et Edmund D. Brodie. 2004. « How the Horned Lizard Got Its Horns ». *Science* 304 (5667): 65-65. <https://doi.org/10.1126/science.1094790>.